

.....

-
-
-

como hacer test automatizados E2E en una aplicación web con spring boot

Para hacer **tests E2E (end-to-end)** en una app web con **Spring Boot**, la idea es: levantar tu aplicación “como en producción” (HTTP real, DB real o parecida, frontend real) y automatizar un navegador para comprobar flujos completos (login, comprar, etc.).

A continuación tienes un enfoque práctico y muy usado hoy en día.

1) Elige la herramienta E2E (recomendación rápida)

Opción A (muy recomendada): Playwright

- Rápido, estable, buen soporte para Chrome/Firefox/WebKit.
- Muy bien para CI y para pruebas “flaky” (tiene auto-wait).
- Puedes escribir tests en **Java** (si quieres quedarte en el ecosistema JVM) o en **Node**.

Opción B: Cypress

- Experiencia DX excelente, muy popular en frontend.
- Ideal si tu equipo ya está en Node para tests.

Opción C: Selenium

- Clásico, muy extendido.
- Suele requerir más “esperas” y mantenimiento que Playwright.

Mi sugerencia si partes de cero: **Playwright**.

2) Define qué es “E2E” en tu proyecto

Un E2E típico prueba:

- **Frontend real** (tu SPA o plantillas server-side).
- **Backend real** (Spring Boot levantado).
- **Integraciones**: o bien reales (con entorno) o **mockeadas** (WireMock) si son externas.
- **Base de datos**: idealmente real (Postgres/MySQL) mediante **Testcontainers**.

3) Estructura recomendada en un proyecto Spring Boot

Dos patrones comunes:

Patrón 1: E2E como módulo aparte

- /e2e-tests con Playwright (Node o Java).
- Ventaja: separas dependencias y tiempos.

Patrón 2: E2E dentro del repo Spring como carpeta

- /src/test/e2e (si Playwright Java) o /e2e (si Node).
- Ventaja: más fácil de ejecutar en local.

4) Levantar Spring Boot para los E2E

Opción simple (local y CI):

1. Arrancas la app en un puerto (p. ej. 8080) con un perfil e2e.
2. Ejecutas el runner E2E apuntando a BASE_URL=http://localhost:8080.

Opción más “pro” (recomendable): Testcontainers

- Levantas DB real (Postgres) en contenedor.
- Configuras Spring con esas credenciales.
- Opcional: levantas también servicios (Redis, Keycloak, etc.) en contenedores.

5) Ejemplo con Playwright (Node) + Spring Boot

5.1. Perfil application-e2e.yml

Ejemplo de cosas típicas:

- DB dedicada
- logs más simples
- desactivar jobs/cron
- desactivar integraciones externas o apuntarlas a mocks

spring:

```
datasource:  
  url: jdbc:postgresql://localhost:5432/app_e2e  
  username: app  
  password: app  
jpa:  
  hibernate:  
    ddl-auto: validate
```

app:

```
externalPayments:  
  enabled: false
```

5.2. Script para CI/local

- Paso 1: levantar backend
- Paso 2: levantar frontend (si es SPA separada)
- Paso 3: correr Playwright

Ejemplo conceptual:

- `./mvnw spring-boot:run -Dspring-boot.run.profiles=e2e`
- `npx playwright test`

5.3. Test E2E básico (Playwright)

```
import { test, expect } from '@playwright/test';

test('login y ver dashboard', async ({ page }) => {
  await page.goto(process.env.BASE_URL + '/login');

  await page.getByLabel('Email').fill('demo@acme.com');
  await page.getByLabel('Password').fill('password');
  await page.getByRole('button', { name: 'Entrar' }).click();

  await expect(page).toHaveURL(/.*dashboard/);
  await expect(page.getByText('Bienvenido')).toBeVisible();
});
```

6) Datos de prueba: la clave para que E2E no sea un infierno

Lo que más rompe E2E es el estado.

Tres estrategias típicas (de mejor a peor):

A) Reset del estado por test (ideal)

- Antes de cada test: limpiar DB / volver a snapshot.
- Con Testcontainers puedes:
 - recrear esquema
 - usar @Sql en backend
 - o scripts truncate entre tests

B) Semilla fija + datos únicos por test

- Insertas “usuarios demo” (seed).
- Cada test crea datos con IDs aleatorios para evitar colisiones.

C) Reutilizar estado

- Suele acabar en tests frágiles (evítalo si puedes).

7) Autenticación: cómo hacer login sin flakiness

Depende del sistema:

- Si usas login clásico (form): perfecto, hazlo por UI.
 - Si usas OAuth/OIDC (Keycloak, Auth0):
 - O bien tienes un “modo e2e” con auth simplificada,
 - o haces login real contra un IdP de testing,
 - o guardas el “storageState” (Playwright) para no repetir login en cada test.
-

8) Buenas prácticas E2E (para que sea mantenible)

- Testea **flujos críticos**, no cada detalle (eso es para unit/integration).
 - Usa **selectores estables**:
 - data-testid="login-submit" (ideal)
 - evita selectores por CSS frágiles.
 - Paraleliza en CI, pero con DB aislada o data strategy sólida.
 - Captura evidencia:
 - screenshots / video / trace en fallos (Playwright lo hace genial).
 - Separa:
 - **Integration tests** (Spring + DB + mocks) para lógica
 - **E2E** para 5–20 caminos críticos.
-

9) Alternativa: E2E en Java con Playwright Java

Si prefieres quedarte 100% en Maven/Gradle, Playwright tiene binding Java y puedes ejecutar E2E desde mvn test (con un perfil).

Si me dices **qué tipo de frontend** tienes (Thymeleaf dentro de Spring, SPA React/Angular/Vue, o front separado) y **qué auth** usas (session, JWT, OAuth), te propongo una estructura exacta (carpetas, perfiles e2e, y pipeline de CI) para tu caso.

quiero hacerlo con esos que te graban la pantalla y te guarda el test

Thought for 8s

Vale: lo que describes es **“record & playback”** (grabas acciones en el navegador y te genera/guarda el test). Para Spring Boot te funciona igual, porque al final estás probando la app por HTTP como un usuario.

Aquí tienes las opciones más típicas (y cómo montarlas):

1) Playwright “codegen” (graba y genera el test)

Playwright trae un **generador de tests**: tú navegas/clicas/escribes y él te va creando el script y selectores bastante decentes.

Por qué suele ser la mejor opción: rápido, estable, y lo grabado luego lo puedes “arreglar” y mantener como código.

Cómo encaja con Spring Boot

1. Levantas tu app (`./mvnw spring-boot:run -Dspring-boot.run.profiles=e2e`)
2. Ejecutas el recorder apuntando a tu URL.
3. Te genera un test (normalmente en TS/JS si usas npx, o en Java si usas Playwright Java).

👉 Referencia oficial del generador:

2) Cypress Studio (graba dentro del runner y lo guarda en tu spec)

Cypress tiene **Cypress Studio**, que permite:

- **Record:** interactúas con la app y te añade comandos al test
- **Assert:** click derecho para añadir aserciones
- **Edit:** editar el código directamente

Es muy cómodo si tu equipo ya vive en Cypress.

👉 Doc oficial:

Nota realista: como cualquier herramienta visual, a veces hay detalles/versiones/entornos corporativos (proxy, etc.) que pueden afectar (no es lo común, pero existe).

3) Selenium IDE (extensión del navegador, record & playback)

Selenium IDE es literalmente “grabo pasos en Chrome/Firefox y los reproduzco”. Es open-source y pensado para autoría rápida.

Cuándo lo elegiría

- Para equipos QA/manuales que quieren empezar *ya*.
- Para flujos sencillos y demos.
- Si luego quieres exportar y migrar a código (depende del flujo y plugins, pero la idea es esa).

👉 Página oficial:

4) Herramientas no/low-code que graban (SaaS)

Si quieres algo aún más “sin código”, hay opciones tipo **mabl / Testim / Ghost Inspector / Katalon / etc.** (suelen grabar, dar UI para asserts, reintentos, reports, etc.). Ojo: normalmente **son de pago** y a veces acabas “peleándote” con mantenimiento cuando cambia la UI.

Lo que te recomiendo para Spring Boot (práctico)

Si quieres “grabar y que te quede un test mantenible”:

- | | | | | |
|----|--|--------------------|---------|------------------|
| 1) | Playwright | codegen (mi | primera | opción) |
| 2) | Cypress | Studio si | ya | estás en Cypress |
| 3) | Selenium IDE si priorizas lo visual y rápido para casos simples | | | |

Checklist para que lo grabado no sea frágil (muy importante)

- Añade en tu HTML atributos estables tipo `data-testid="login-submit"` y que el recorder los use.
- Evita selectores por CSS “bonitos” pero frágiles (clases que cambian).
- Define un perfil e2e en Spring (desactivar jobs, usar DB de test, mocks externos).

Si me dices en 1 línea:

1. ¿Tu front es **Thymeleaf** o **SPA (React/Angular/Vue)**?
2. ¿Tu auth es **form login**, **JWT** u **OAuth/OIDC**?

...te doy el “camino feliz” exacto (comandos + estructura de carpetas + cómo guardar el test grabado y correrlo en CI) para tu caso.

que es lo más sencillo de hacer

La respuesta corta y honesta

👉 lo más sencillo, rápido y con menos dolor es: **Playwright con “codegen”**. 👉

Si nunca has hecho E2E o no quieres pelearte con mil configs, **esto es el camino feliz**.

🟢 **Opción MÁS simple (recomendada): Playwright + grabador**

Por qué es lo más fácil

- No necesitas tocar Spring Boot casi nada
 - Grabas la pantalla → **te guarda el test**
 - Los selectores suelen salir bastante bien
 - Corre igual en tu máquina y en CI
 - Si algo falla, te da video + screenshots
-

🌱 Paso a paso ultra simple

1 Arranca tu Spring Boot

`./mvnw spring-boot:run`

Tu app debe verse en:

`http://localhost:8080`

2 Crea carpeta de tests E2E

```
mkdir e2e
cd e2e
npm init -y
npm install @playwright/test
npx playwright install
```

3 Graba el test (la magia ✨)

`npx playwright codegen http://localhost:8080`

Se abre:

- un navegador
- una ventana de código

👉 Tú **navegas normalmente**:

- login
- clicks
- formularios

👉 **Playwright va escribiendo el test solo**

Cuando terminas:

- guardas el archivo (`example.spec.ts` o `.js`)
- listo

Ejecuta el test grabado

npx playwright test

Si falla:

- te muestra **video**
- screenshots
- trace interactivo

Qué NO hacer (para que siga siendo sencillo)

  No dependas de grabes 50 flujos
 No pruebes cada pixel de datos "random" en DB

 Graba **solo flujos críticos**:

- login
- crear algo
- flujo principal de negocio

Alternativa igual de fácil (pero un poco menos robusta)

Selenium IDE

- extensión del navegador
- grabar / reproducir
- cero código

 Más simple al principio
 Más frágil a medio plazo

Conclusión clara

Si tu prioridad es **"quiero grabar, guardar el test y que funcione"**:

 **Playwright**

codegen

 Selenium IDE (si quieres cero setup)

Si quieres, dime:

- ¿tu login es usuario/password normal?
- ¿Spring Boot con Thymeleaf o SPA?

Y te digo **exactamente qué grabar** para que no se rompa mañana 😊

hazlo con selenium

Perfecto: con **Selenium** lo más sencillo para “grabar la pantalla y que te guarde el test” es **Selenium IDE** (extensión del navegador). Luego, si quieres, lo **exportas a Java + JUnit** para integrarlo con tu proyecto Spring Boot.

1) Opción más fácil: Selenium IDE (grabar y guardar)

Paso A — Instala Selenium IDE

- Instala la extensión **Selenium IDE** en **Chrome** o **Firefox** (desde la tienda de extensiones).

Paso B — Levanta tu Spring Boot

```
./mvnw spring-boot:run
```

Tu app normalmente estará en:

`http://localhost:8080`

Paso C — Graba el test

1. Abre **Selenium IDE**
2. **Create a new project**
3. Pon **Base URL**: `http://localhost:8080`
4. Dale a **Record**
5. Haz el flujo real (login, crear algo, etc.)
6. Para terminar, **Stop recording**
7. **Save** el proyecto (te genera un `.side`)

✅ Ya tienes el test guardado y reproducible desde Selenium IDE con “Play”.

2) Exportar el test grabado a Java (para correrlo con Maven/JUnit)

En Selenium IDE:

1. Abre tu proyecto `.side`
2. **Export**
3. Elige:
 - **Java JUnit** (común)
 - o **Java TestNG**

Te descargará un archivo .java con los pasos.

3) Integrarlo en tu proyecto Spring Boot (JUnit + Selenium WebDriver)

Dependencias Maven (ejemplo)

Añade (o equivalente) en tu pom.xml:

```
<dependencies>
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.18.1</version>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>io.github.bonigarcia</groupId>
  <artifactId>webdrivermanager</artifactId>
  <version>5.7.0</version>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.2</version>
  <scope>test</scope>
</dependency>
</dependencies>
```

Nota: las versiones pueden variar; si tu proyecto ya usa JUnit 5, seguramente solo necesitas Selenium + WebDriverManager.

Ejemplo de test E2E en Java (plantilla)

Crea src/test/java/.../E2ELoginTest.java:

```
import io.github.bonigarcia.wdm.WebDriverManager;
import org.junit.jupiter.api.*;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import java.time.Duration;

public class E2ELoginTest {

    private WebDriver driver;
```

```

private WebDriverWait wait;

@BeforeEach
void setUp() {
    WebDriverManager.chromedriver().setup();

    ChromeOptions options = new ChromeOptions();
    // Quita esta línea si quieres ver el navegador
    options.addArguments("--headless=new");

    driver = new ChromeDriver(options);
    wait = new WebDriverWait(driver, Duration.ofSeconds(10));
}

@AfterEach
void tearDown() {
    if (driver != null) driver.quit();
}

@Test
void loginYDashboard() {
    driver.get("http://localhost:8080/login");

    // Recomendación: usa data-testid en tu HTML para que no se rompan los selectores.
    driver.findElement(By.name("username")).sendKeys("demo@acme.com");
    driver.findElement(By.name("password")).sendKeys("password");
    driver.findElement(By.cssSelector("button[type='submit']")).click();

    wait.until(ExpectedConditions.urlContains("dashboard"));
    Assertions.assertTrue(driver.getPageSource().contains("Bienvenido"));
}
}

```

Ejecutar:

```
./mvnw test
```

4) Para que no se rompa mañana (muy importante)

Si grabas con Selenium IDE, tus tests serán mucho más estables si en tu UI pones selectores “fijos”, por ejemplo:

```

<button data-testid="login-submit">Entrar</button>
<input data-testid="login-email" />

```

Y en Selenium (o IDE) apuntas a `css=[data-testid="login-submit"]`.

Si me dices **cómo es tu login** (form normal / OAuth) y si tu frontend es **Thymeleaf o SPA**, te indico exactamente **qué selectores usar** y cómo dejarlo listo para CI (GitHub Actions/GitLab).

me pone que selenium ide no esta disponible para chrome, katalon esta bien?

Sí 👍 **Katalon está bien** — y, de hecho, **es la alternativa más sencilla ahora mismo** si Selenium IDE no te aparece para Chrome.

Te lo digo claro y sin humo 🙌

🟢 ¿Katalon sirve para Spring Boot?

Sí, **perfectamente.**
Katalon usa **Selenium por debajo**, así que:

- Graba acciones reales en el navegador
- Guarda los tests
- Puedes ejecutarlos en local o CI
- Funciona igual con apps Spring Boot (Thymeleaf o SPA)

🌿 La forma MÁS simple con Katalon (record & playback)

1 Instala Katalon

- Descarga **Katalon Studio** (Desktop)
- Plan **free** es suficiente para empezar

2 Arranca tu app Spring Boot

`./mvnw spring-boot:run`

URL típica:

`http://localhost:8080`

3 Graba el test (sin código)

1. Abre **Katalon Studio**
2. **New → Test Case**
3. **Record Web**
4. Navegador: **Chrome**
5. URL inicial: `http://localhost:8080`

6. Haz el flujo normal:

- login
- clicks
- formularios

7. Stop → Save

🎉 Listo. El test queda guardado y reproducible.

4 Ejecuta el test

- Botón **Run**
 - Ves el navegador moverse solo
 - Resultado PASS / FAIL
-

🧠 Consejos clave para que no se rompa

Esto es lo que diferencia “funciona hoy” de “sigue funcionando en 3 meses”:

✓ Usa atributos estables en tu HTML

```
<input data-testid="username" />
<button data-testid="login-submit">Entrar</button>
```

En Katalon:

- Apunta a css=[data-testid="login-submit"]
 - Evita XPath's largos y frágiles
-

✓ Mantén pocos tests E2E

- Login
- Flujo principal
- Caso feliz

👉 Todo lo demás: tests unitarios / integración en Spring

⚠ Cosas a tener en cuenta con Katalon

Lo bueno

- Muy fácil
- Grabación visual

- Ideal para empezar rápido

Lo menos bueno

- Es una herramienta “grande”
 - Algunas funciones avanzadas son de pago
 - A largo plazo suele convenir migrar a código puro (Selenium / Playwright)
-

🚩 Conclusión honesta

Si tu prioridad es:

“Quiero grabar, guardar el test y que funcione YA”

👉 **Katalon es una buena elección** ✅

Si más adelante quieres, puedes:

- exportar/migrar a Selenium en Java
 - o pasar a Playwright sin tirar todo
-

Si quieres, dime:

- ¿login normal o OAuth?
- ¿Thymeleaf o SPA?

Y te digo **exactamente qué grabar primero** para que no se rompa y te dé valor real 😊

3.type on id=formField:R6dpf55: with value prueba@prueba.com Failed: 21:31:20 call to eval() blocked by CSP 'test 1' ended with 1 error(s)

Thought for 9s

Ese error no es de tu test en sí: es tu **CSP (Content Security Policy)** bloqueando eval(). Y **Selenium IDE (y otros “recorders” basados en extensión)** a veces usan eval/inyección de JS para ejecutar comandos (por eso revienta justo al “type”).

Qué significa tu mensaje exactamente

- call to eval() blocked by CSP ⇒ tu web envía una cabecera CSP que **no permite** unsafe-eval.
 - El “recorder” intenta ejecutar algo con eval dentro de la página y el navegador lo bloquea.
-

Lo más sencillo para solucionarlo (de menos invasivo a más)

1) Ejecuta los tests NO desde la extensión, sino con Katalon/Selenium WebDriver

Esto suele evitar el problema porque WebDriver no depende del mismo mecanismo de “eval en la página” típico de algunas extensiones.

✅ Si ya estás usando **Katalon Studio**, haz esto:

- No uses “play” del test grabado en modo extensión/IDE si es el que falla por CSP.
- En Katalon, asegúrate de ejecutar como **WebUI** normal (WebDriver) y que el objeto apunte al input correcto.
- Si el test grabado usa un “target” raro como id=formField:R6dpf55: (con :), cambia el selector a CSS robusto, por ejemplo:
 - `css=input[id="formField:R6dpf55:"]` (ojo: en CSS los : hay que tratarlos bien; con comillas suele ir)
 - o mejor: `data-testid` (ver punto 4)

Nota: el mensaje que pegas parece muy “Selenium IDE style”. Si estás en Katalon y te sale exactamente eso, probablemente estás ejecutando algo grabado/exportado que sigue usando pasos/ejecución tipo IDE.

2) Solución rápida sólo para entorno de test: relajar CSP en perfil e2e

En **Spring Boot** puedes tener un perfil e2e donde tu CSP sea menos estricta **solo en test**.

Ejemplo conceptual:

- En prod: CSP estricta (sin unsafe-eval)
- En e2e: permitir lo mínimo para que el recorder funcione (si de verdad lo necesitas)

⚠ Importante: **no recomiendo** meter unsafe-eval en producción. Es una relajación de seguridad.

3) Desactivar CSP en el navegador (solo para debug local)

Hay extensiones/flags que “desactivan CSP” para pruebas, pero:

- no es representativo de producción
- en CI suele ser un lío
- y no lo recomiendo como solución final

4) Mejor práctica (te evita el 80% de problemas de grabadores): data-testid

Aunque arregles lo del CSP, esos IDs con cosas dinámicas (formField:R6dpf55:) son una receta para tests frágiles.

En tu HTML:

```
<input data-testid="email" type="text" />
<button data-testid="login-submit">Entrar</button>
```

En Katalon/Selenium apuntas a:

- `css=[data-testid="email"]`

Qué haría yo en tu caso (lo más sencillo y limpio)

1. **Katalon ejecutando por WebDriver** (no “IDE extension mode”)
2. Añadir data-testid en los elementos clave
3. Si aun así necesitas grabar y reproducir con el mecanismo que choca con CSP: **perfil e2e** que relaje CSP solo en test

Si me dices **cómo estás sirviendo CSP** (Spring Security headers, filtro propio, Nginx, etc.) te digo exactamente dónde tocarlo para que **solo el perfil e2e** cambie la cabecera y no afecte producción.

estoy usando selenium ide en firefox

Thought for 13s

Perfecto, entonces ya cuadra: **Selenium IDE en Firefox** suele fallar en páginas con **CSP estricta**, porque durante el “playback” intenta ejecutar JS con `eval()` dentro de la página, y tu CSP lo bloquea (`script-src sin 'unsafe-eval'`). Por eso el paso **type** revienta con:

call to eval() blocked by CSP

Tienes 3 salidas, de la más recomendable a la más “hack”:

1) Mejor salida (sin tocar seguridad de la web): ejecutar el .side con WebDriver (no dentro de la extensión)

En lugar de darle “Play” en Selenium IDE, ejecuta ese mismo test con un runner que usa **WebDriver real**, que normalmente **no depende del eval** de la extensión.

Opción A: Exportar a Java JUnit desde Selenium IDE

- Export → Java JUnit
- Pegas el test en `src/test/java`
- Lo ejecutas con `mvn test`

Opción B: selenium-side-runner (Node)

- Ejecuta el .side desde consola contra Firefox/GeckoDriver (esto ya no es “dentro” de la extensión).
- Si quieres, te paso los comandos exactos para tu caso.

✅ Ventaja: no debilitas CSP en tu app.

2) Solución “rápida” solo para entorno E2E: relajar CSP en *perfil de test*

Si tu objetivo es **seguir usando Play dentro de Selenium IDE**, entonces sí o sí necesitas permitir eval en ese entorno.

En un entorno E2E (local/QA), tu cabecera CSP debería incluir:

- script-src ... 'unsafe-eval' ...

⚠ **Nunca en producción.** Solo en application-e2e/local.

Si usas Spring Security (ejemplo típico)

Crear una config SOLO para el perfil e2e:

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@Profile("e2e")
public class SecurityE2EConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .headers(headers -> headers)
            .contentSecurityPolicy(csp -> csp
                .policyDirectives("default-src 'self';"
                    + "script-src 'self' 'unsafe-eval';"
                    + "object-src 'none';"
                    + "base-uri 'self';")
                )
            );
        return http.build();
    }
}
```

Y arrancas con:

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=e2e
```

Si tu CSP la pone **Nginx/Apache/CloudFront**, entonces el cambio se hace ahí, pero igual: **solo para entorno de test.**

3) Hack solo para probar: desactivar CSP en Firefox (NO recomendado)

Para confirmar que “es CSP y no otra cosa” puedes desactivarlo globalmente:

- about:config
- buscar security.csp.enable
- ponerlo en false

⚠ Esto desactiva CSP para TODO el navegador, no es buena práctica y no sirve para CI, pero ayuda a diagnosticar.

Nota extra: ese selector id=FormField:R6dpf55:

Aunque arregles CSP, esos IDs con : a veces dan guerra. Mejor usa selectores estables:

- data-testid="email" en tu HTML
- y en el test: css=[data-testid="email"]

Qué te recomiendo hacer YA (lo más simple con menos cambios)

1. Mantén tu CSP segura
2. **Exporta el test de Selenium IDE a Java JUnit** y ejecútalo con Maven

Si me pegas tu CSP actual (la cabecera Content-Security-Policy que ves en DevTools → Network → Response headers) o me dices si la pone Spring Security o un proxy, te digo exactamente qué línea tocar para que funcione solo en e2e.

como puedo ejecutar el junit que he exportado

Perfecto  vamos **paso a paso** y sin líos.
Si ya **exportaste el test desde Selenium IDE a Java JUnit**, ejecutarlo es bastante directo.

1 Coloca el archivo .java en el sitio correcto

El archivo que exporta Selenium IDE suele llamarse algo como:

MyTest.java

Ponlo dentro de tu proyecto Spring Boot en:

src/test/java/

Ejemplo completo:

src/test/java/com/tuempresa/e2e/LoginE2ETest.java

⚠️ Asegúrate de que el package del archivo coincida con la carpeta.

2 Añade las dependencias necesarias en pom.xml

Si tu proyecto Spring Boot es estándar, probablemente **ya tienes JUnit 5**.
Añade **Selenium y WebDriverManager** si no están:

```
<dependencies>
<!-- Selenium -->
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.18.1</version>
  <scope>test</scope>
</dependency>

<!-- Descarga automática del driver -->
<dependency>
  <groupId>io.github.bonigarcia</groupId>
  <artifactId>webdrivermanager</artifactId>
  <version>5.7.0</version>
  <scope>test</scope>
</dependency>
</dependencies>
```

👉 JUnit **NO hace falta** si usas spring-boot-starter-test.

3 Ajusta el test exportado (2 cambios casi siempre necesarios)

Los tests exportados suelen venir así (resumen):

A) Asegúrate de que usa JUnit 5

Debe tener:

```
import org.junit.jupiter.api.Test;
```

y **no**:

```
import org.junit.Test;
```

Si usa JUnit 4, dímelo y lo adaptamos (es trivial).

B) Inicializa el WebDriver correctamente

Selenium IDE **no siempre** pone WebDriverManager.

Añade arriba:

```
import io.github.bonigarcia.wdm.WebDriverManager;
```

Y en el setup:

```
WebDriverManager.firefoxdriver().setup();  
driver = new FirefoxDriver();
```

Ejemplo mínimo funcional:

```
@BeforeEach  
void setUp() {  
    WebDriverManager.firefoxdriver().setup();  
    driver = new FirefoxDriver();  
}
```

4 Ejecutar el test 🚀

Opción A — Desde consola (la más común)

Desde la raíz del proyecto:

```
./mvnw test
```

Maven:

- levanta JUnit
- ejecuta el test
- abre Firefox
- corre el flujo

Opción B — Ejecutar SOLO ese test

Muy útil al depurar:

```
./mvnw -Dtest=LoginE2ETest test
```

O con paquete:

```
./mvnw -Dtest=com.tuempresa.e2e.LoginE2ETest test
```

Opción C — Desde el IDE (IntelliJ / Eclipse)

- Click derecho sobre la clase
- Run 'LoginE2ETest'

👉 Ideal para ver qué falla paso a paso.

5 Asegúrate de que tu app esté levantada

Muy importante 📌

Antes de ejecutar el test:

`./mvnw spring-boot:run`

(O en otra terminal)

El test **no levanta Spring Boot solo** (a menos que lo hayas preparado para eso).

6 Problemas comunes (rápido)

❌ No abre el navegador

- | | | |
|-----------------------------|-------|------------------|
| ✓ | Falta | WebDriverManager |
| ✓ No está instalado Firefox | | |
-

❌ Falla por CSP

- ✓ Correcto: **WebDriver no usa eval**, esto ya no debería pasar 🎉
-

❌ No encuentra elementos

- | | | | | |
|-------------------|--------|------------|----------|-------------------------|
| ✓ | Cambia | selectores | frágiles | (id=FormField:R6dpf55:) |
| ✓ Usa data-testid | | | | |
-

🚩 Resumen rápido

1. `.java` → `src/test/java`
 2. Dependencias Selenium
 3. `WebDriverManager.firefoxdriver().setup()`
 4. `./mvnw test`
 5. App Spring Boot levantada
-

Si quieres, **pégame el test Java que exportó Selenium IDE** (aunque sea un trozo) y te lo dejo **limpio, estable y listo para CI** 💪

```
package com.upm.library; import static org.hamcrest.CoreMatchers.is; import static
org.hamcrest.core.IsNot.not; import org.aspectj.lang.annotation.After; import
org.aspectj.lang.annotation.Before; import org.junit.jupiter.api.Test; import java.awt.*; import
java.util.HashMap; import java.util.Map; public class ReservationTest { private WebDriver driver;
private Map<String, Object> vars; JavascriptExecutor js; @Before("") public void setUp() { driver
```

```

= new FirefoxDriver(); js = (JavascriptExecutor) driver; vars = new HashMap<String, Object>(); }
@After("") public void tearDown() { driver.quit(); } @Test public void test1() {
driver.get("https://us-east-1snnleupih.auth.us-east-
1.amazoncognito.com/login?client_id=1h2dcs34d8ucv154k44joac1bc&code_challenge=GLP5b
SnpjlkntR-
gRYCom2bXxZdZsbRfO2JAEU_qcuc&code_challenge_method=S256&nonce=uOQBpTFKR--
1Gkg9BGewVGa5QKb5AANvDnPebUDiXeo&redirect_uri=http://localhost:8080/login/oauth2/c
ode/cognito&response_type=code&scope=phone+openid+email&state=k9LV339_KKImRvC11k
ePad79i-fEj8rqS0lneHuu-JQ%3D"); driver.manage().window().setSize(new Dimension(1936,
1056)); driver.findElement(By.id("formField:R6dpf55:")).sendKeys("prueba@prueba.com");
driver.findElement(By.id("formField:R6dpf55:")).sendKeys(Keys.ENTER);
driver.findElement(By.id("formField:r1:")).sendKeys("1234567aA!");
driver.findElement(By.id("formField:r1:")).sendKeys(Keys.ENTER);
driver.findElement(By.cssSelector(".two > .card:nth-child(1) > .p")).click();
driver.findElement(By.name("q")).click(); driver.findElement(By.name("q")).sendKeys("a");
driver.findElement(By.cssSelector(".primary")).click(); driver.findElement(By.linkText("Ver
ejemplares →")).click(); driver.findElement(By.linkText("Reservar ejemplar")).click();
driver.findElement(By.cssSelector(".primary")).click();
driver.findElement(By.cssSelector(".danger")).click();
assertThat(driver.switchTo().alert().getText(), is("¿Cancelar esta reserva?"));
driver.switchTo().alert().accept(); driver.findElement(By.cssSelector(".ghost")).click(); } } AQUI
QUE IMPORT ME FALTA Y CUALES ESTAN MAL SI HAY ALGUNO MAL

```

Tu export está “mezclado”: tiene **anotaciones de AspectJ** (org.aspectj...Before/After) y **JUnit 5** (org.junit.jupiter.api.Test). Eso no va a funcionar.

Además te faltan imports de **Selenium** y tienes un **import de Dimension equivocado** (estás usando java.awt.Dimension, pero Selenium usa org.openqa.selenium.Dimension).

Aquí va lo que está mal y lo que falta:

Imports / anotaciones que están mal

 **Estos están mal (no son de JUnit)**

```

import org.aspectj.lang.annotation.After;
import org.aspectj.lang.annotation.Before;

```

Y también estas anotaciones:

```

@Before("")
@After("")

```

 En JUnit 5 deben ser:

- @BeforeEach
- @AfterEach

y sin parámetros.

✗ Este import está mal en tu caso

```
import java.awt.*;
```

Porque luego haces `new Dimension(...)` y eso te está trayendo `java.awt.Dimension`.

✅ Debes usar:

- `org.openqa.selenium.Dimension`

Así que **quita** `java.awt.*`.

Imports que te faltan (Selenium + Hamcrest)

Te faltan todos los típicos de Selenium:

- `WebDriver`
- `FirefoxDriver`
- `By`
- `Keys`
- `JavascriptExecutor`
- `Dimension` (el de Selenium)

Y te falta el `assertThat` (en JUnit 5 con Hamcrest se suele usar):

- `org.hamcrest.MatcherAssert.assertThat`
-

Versión corregida de imports (mínima y correcta)

Pon esto arriba (y elimina los malos):

```
package com.upm.library;

import static org.hamcrest.CoreMatchers.is;
import static org.hamcrest.MatcherAssert.assertThat;

import java.util.HashMap;
import java.util.Map;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import org.openqa.selenium.Alert;
import org.openqa.selenium.By;
import org.openqa.selenium.Dimension;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.Keys;
```



```
import org.openqa.selenium.WebDriver;  
import org.openqa.selenium.firefox.FirefoxDriver;
```

Nota: tu not no lo estás usando, así que puedes borrar:

```
import static org.hamcrest.core.IsNot.not;
```

Y Alert es opcional (puedes dejarlo o quitarlo), pero ayuda si luego quieres declarar Alert
alert =

Cambios en las anotaciones (muy importante)

Cambia esto:

```
@Before("")  
public void setUp() { ... }  
  
@After("")  
public void tearDown() { ... }
```

Por esto:

```
@BeforeEach  
void setUp() { ... }  
  
@AfterEach  
void tearDown() { ... }
```

Extra importante (te va a fallar si no lo tienes)

En Firefox necesitas **geckodriver** en PATH o usar **WebDriverManager**.

Si quieres la opción fácil, añade WebDriverManager y en setUp():

```
WebDriverManager.firefoxdriver().setup();  
driver = new FirefoxDriver();
```

(Esto requiere el import io.github.bonigarcia.wdm.WebDriverManager y la dependencia en Maven.)

Si me dices si usas **Maven o Gradle**, te pongo el bloque exacto de dependencias y te dejo el test completo compilando.

```
NO ME RECONOCE OPENQA HE IMPORTADO <dependency>  
<groupId>org.seleniumhq.selenium</groupId> <artifactId>selenium-java</artifactId>  
<version>4.18.1</version> <scope>test</scope> </dependency>
```

Tranquilo 😊 esto no es Selenium, es Maven/IDE.
Cuando pasa “no reconoce org.openqa.selenium” casi siempre es una de estas causas. Vamos en orden, sin saltarnos nada.

1 Lo MÁS común: no has recargado Maven en el IDE

Aunque el pom.xml esté bien, el IDE no ha descargado las dependencias.

En IntelliJ

1. Abre el pom.xml
2. Arriba a la derecha → **Load Maven Changes**
3. O en el panel **Maven** → **Reload All Maven Projects**

En Eclipse

1. Click derecho sobre el proyecto
2. **Maven** → **Update Project**
3. Marca *Force Update of Snapshots/Releases*

👉 Esto solo ya arregla el 80% de los casos

2 Verifica que el archivo está en src/test/java

Tu dependencia tiene:

```
<scope>test</scope>
```

Eso significa:

- ✓ Funciona en src/test/java
- ✗ **NO funciona** en src/main/java

📌 Tu clase **ReservationTest** debe estar en:

```
src/test/java/com/upm/library/ReservationTest.java
```

Si está en src/main/java, **OpenQA nunca va a aparecer**.

3 Fuerza Maven desde consola (para descartar IDE)

Desde la raíz del proyecto:

```
./mvnw clean test
```

O en Windows:

```
mvn clean test
```

Si esto **descarga Selenium** y empieza a ejecutar tests:
👉 el problema es SOLO del IDE (caché).

4 Borra caché del IDE (si sigue sin ir)

IntelliJ

File → Invalidate Caches / Restart

Marca:

- Clear file system cache
 - Clear VCS log caches
-

5 Verifica que el pom.xml NO está roto

Asegúrate de que la **dependencia está dentro de** <dependencies>, no fuera.

Ejemplo mínimo correcto:

```
<project>
...
<dependencies>

<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.18.1</version>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>io.github.bonigarcia</groupId>
  <artifactId>webdrivermanager</artifactId>
  <version>5.7.0</version>
  <scope>test</scope>
</dependency>

</dependencies>
</project>
```

6 Comprueba rápido si Maven lo ve

Ejecuta:

```
./mvnw dependency:tree | grep selenium
```

Si aparece algo como:

```
org.seleniumhq.selenium:selenium-java:jar:4.18.1:test
```

👉 Selenium **SÍ** está y el problema es el IDE.

7 Checklist final (rápido)

✓	Clase	en src/test/java
✓ pom.xml recargado		
✓ mvn		test funciona
✓ IDE reiniciado		

Si **después de esto** sigue sin reconocer org.openqa.selenium, dime:

1. ¿Usas **IntelliJ** o **Eclipse**?
2. ¿Tu proyecto es **Spring Boot** o Maven puro?
3. ¿Qué error exacto te sale (en rojo)?

Con eso te lo arreglo en un mensaje más 🛠️👉

```
org.openqa.selenium.NoSuchElementException: Unable to locate element:
#formField\:r1\: For documentation on this error, please visit:
https://www.selenium.dev/documentation/webdriver/troubleshooting/errors#nosuchelement
exception Build info: version: '4.37.0', revision: '3c1da1e342' System info: os.name: 'Windows
10', os.arch: 'amd64', os.version: '10.0', java.version: '25.0.1' Driver info:
org.openqa.selenium.firefox.FirefoxDriver Command: [dbaed380-1ad3-454b-aa1a-
a06ddda99914, findElement {using=id, value=formField\:r1\:}] Capabilities {acceptInsecureCerts:
true, browserName: firefox, browserVersion: 147.0.4, moz:accessibilityChecks: false,
moz:buildID: 20260212191108, moz:geckodriverVersion: 0.36.0, moz:headless: false,
moz:platformVersion: 10.0, moz:processID: 22516, moz:profile:
C:\Users\Usuario\AppData\Lo..., moz:shutdownTimeout: 60000, moz:webdriverClick: true,
moz:windowless: false, pageLoadStrategy: normal, platformName: windows, proxy: Proxy(),
setWindowRect: true, strictFileInteractability: false, timeouts: {implicit: 0, pageLoad: 300000,
script: 30000}, unhandledPromptBehavior: dismiss and notify, userAgent: Mozilla/5.0 (Windows
NT 10....} Session ID: dbaed380-1ad3-454b-aa1a-a06ddda99914
```

Ese error significa simplemente: **cuando Selenium intenta escribir la contraseña, ese elemento todavía no existe (o no es visible) en el DOM**. En páginas como **Amazon Cognito Hosted UI**, tras meter el email suele haber un **paso intermedio / render asíncrono**, y el id del password **puede cambiar** o tardar en aparecer. Resultado: NoSuchElementException.

Además, en tu log se ve que buscas **por id** formField\:r1: pero el mensaje “Unable to locate element: #formField\:r1\:” te confirma que Selenium lo está intentando como selector CSS equivalente (con los : escapados). El escape está bien: el problema es **timing / página distinta / id distinto**.

Arreglo rápido y correcto: espera explícita + selector más robusto

1) No uses findElement directo para el password: usa WebDriverWait

Ejemplo (JUnit 5), justo después de enviar el email:

```
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import java.time.Duration;

// ...

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));

// email =
WebElement email =
wait.until(ExpectedConditions.elementToBeClickable(By.id("formField:R6dpf55:")));
email.sendKeys("prueba@prueba.com");
email.sendKeys(Keys.ENTER);

// espera a que aparezca el password (Cognito suele usar name="password")
WebElement password =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("password")));
password.sendKeys("1234567aA!");
password.sendKeys(Keys.ENTER);
```

✅ **Clave:** en Cognito Hosted UI es mucho más estable buscar el password por name="password" que por un id tipo formField:r1: (que puede variar).

Si name="password" no funciona

Usa un selector por autocomplete (muy típico) o por type=password:

```
WebElement password =
wait.until(
ExpectedConditions.visibilityOfElementLocated(By.cssSelector("input[type='password']"))
);
```

Por qué te falla exactamente ahora

Causas típicas (una o varias):

1. **Todavía estás en la pantalla del email** (no avanzó), por error de login/validación.
2. El campo password aparece pero **tarda** (JS).
3. El id del password **no es siempre** formField:r1: (cambia).
4. Estás enviando ENTER y a veces Cognito necesita click en un botón (depende de configuración).

Mini-check para diagnosticar en 10 segundos

Justo antes de buscar el password, imprime la URL y un “snapshot” rápido:

```
System.out.println("URL actual: " + driver.getCurrentUrl());
System.out.println("HTML contiene password? " +
driver.getPageSource().contains("password"));
```

Si contains("password") da false, no estás en la pantalla correcta todavía.

Recomendación para que no se rompa

En Cognito Hosted UI, evita los ids formField:... y usa:

- By.name("username") / By.name("password") (si existen)
- o input[type=password]
- y siempre WebDriverWait

Si me pegas el **HTML del input de password** (inspeccionar elemento → copia la línea <input ...>), te doy el selector exacto más estable para tu caso.

```
package com.upm.library; import static org.hamcrest.CoreMatchers.is; import static
org.hamcrest.MatcherAssert.assertThat; import java.util.HashMap; import java.util.Map;
import org.junit.jupiter.api.AfterEach; import org.junit.jupiter.api.BeforeEach; import
org.junit.jupiter.api.Test; import org.openqa.selenium.By; import
org.openqa.selenium.Dimension; import org.openqa.selenium.JavascriptExecutor; import
org.openqa.selenium.Keys; import org.openqa.selenium.WebDriver; import
org.openqa.selenium.firefox.FirefoxDriver; public class ReservationTest { private WebDriver
driver; private Map<String, Object> vars; JavascriptExecutor js; @BeforeEach public void setUp()
{ driver = new FirefoxDriver(); js = (JavascriptExecutor) driver; vars = new HashMap<String,
Object>(); } @AfterEach public void tearDown() { driver.quit(); } @Test public void test1() {
driver.get("https://us-east-1snnleupih.auth.us-east-
1.amazoncognito.com/login?client_id=1h2dcs34d8ucv154k44joac1bc&code_challenge=GLP5b
SnjlkntR-
gRYCOm2bXxZdZsbRfO2JAEU_qcuc&code_challenge_method=S256&nonce=uOQBpTFKR--
1Gkg9BGewVGa5QKb5AANvDnPebUDiXEO&redirect_uri=http://localhost:8080/login/oauth2/c
ode/cognito&response_type=code&scope=phone+openid+email&state=k9LV339_KKImRvC11k
ePad79i-fEj8rqS0IneHuu-JQ%3D"); driver.manage().window().setSize(new Dimension(1936,
1056)); driver.findElement(By.id("formField:R6dpf55:")).sendKeys("prueba@prueba.com");
driver.findElement(By.id("formField:R6dpf55:")).sendKeys(Keys.ENTER);
driver.findElement(By.id("formField:r1:")).sendKeys("1234567aA!");
driver.findElement(By.id("formField:r1:")).sendKeys(Keys.ENTER);
driver.findElement(By.cssSelector(".two > .card:nth-child(1) > .p")).click();
driver.findElement(By.name("q")).click(); driver.findElement(By.name("q")).sendKeys("a");
driver.findElement(By.cssSelector(".primary")).click(); driver.findElement(By.linkText("Ver
ejemplares →")).click(); driver.findElement(By.linkText("Reservar ejemplar")).click();
driver.findElement(By.cssSelector(".primary")).click();
```

```
driver.findElement(By.cssSelector(".danger")).click();
assertThat(driver.switchTo().alert().getText(), is("¿Cancelar esta reserva?"));
driver.switchTo().alert().accept(); driver.findElement(By.cssSelector(".ghost")).click(); } } esto es
lo que tengo, donde lo tengo que cambiar
```

Tienes el test bien “montado”, lo que te está fallando es **solo esta parte**: estás usando IDs de Cognito (formField:...) que **cambian** y además **no esperas** a que aparezca el campo password.

Cambios que tienes que hacer (exactamente dónde)

1) Añade imports de WebDriverWait y ExpectedConditions

Arriba, con los imports, añade:

```
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import java.time.Duration;
```

2) Dentro de test1(), crea un wait

Justo después de setSize(...) pon:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));
```

3) Sustituye SOLO estas 4 líneas (email + password)

Ahora mismo tienes:

```
driver.findElement(By.id("formField:R6dpf55:")).sendKeys("prueba@prueba.com");
driver.findElement(By.id("formField:R6dpf55:")).sendKeys(Keys.ENTER);
driver.findElement(By.id("formField:r1:")).sendKeys("1234567aA!");
driver.findElement(By.id("formField:r1:")).sendKeys(Keys.ENTER);
```

Cámbialas por esto (más estable):

```
WebElement email =
wait.until(ExpectedConditions.elementToBeClickable(By.name("username")));
email.sendKeys("prueba@prueba.com");
email.sendKeys(Keys.ENTER);
```

```
WebElement password =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("password")));
password.sendKeys("1234567aA!");
password.sendKeys(Keys.ENTER);
```

 Con esto:

- dejas de depender de IDs tipo formField:r1: (frágiles)
 - esperas a que el password exista antes de escribir
-

Si tu Cognito NO usa name="username"

A veces el campo email/usuario se llama email o parecido. Alternativas rápidas:

Email por type=email

```
WebElement email = wait.until(
ExpectedConditions.elementToBeClickable(By.cssSelector("input[type='email']")))
);
```

Username por placeholder (si existe)

```
WebElement email = wait.until(
ExpectedConditions.elementToBeClickable(By.cssSelector("input[placeholder*='Email']")))
);
```

Password (casi siempre funciona)

```
WebElement password = wait.until(
ExpectedConditions.visibilityOfElementLocated(By.cssSelector("input[type='password']")))
);
```

4) (Opcional pero recomendado) Espera también tras el login

Después de hacer ENTER en password, antes de clicar .two > .card... añade una espera a algo de tu app (por ejemplo una URL o un elemento):

```
wait.until(ExpectedConditions.urlContains("localhost:8080"));
```

o a un elemento de tu home:

```
wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("q")));
```

Resumen: lo mínimo que debes cambiar

- Añadir imports de WebDriverWait/ExpectedConditions/Duration
- Crear wait en test1()
- Reemplazar las líneas de By.id("formField:...") por By.name("username") y By.name("password") con wait.until(...)

Si me pegas una captura del HTML del input de usuario (la línea <input ...> que sale en inspeccionar), te digo si es name="username" o qué selector exacto usar para tu Cognito.

```
Build info: version: '4.37.0', revision: '3c1da1e342' System info: os.name: 'Windows 10',
os.arch: 'amd64', os.version: '10.0', java.version: '25.0.1' Driver info:
org.openqa.selenium.firefox.FirefoxDriver Command: [99ba36c8-cde1-4990-b7e2-
394b583ae6c0, findElement {value=.two > .card:nth-child(1) > .p, using=css selector}]
Capabilities {acceptInsecureCerts: true, browserName: firefox, browserVersion: 147.0.4,
```



```
moz:accessibilityChecks: false, moz:buildID: 20260212191108, moz:geckodriverVersion: 0.36.0,
moz:headless: false, moz:platformVersion: 10.0, moz:processID: 12744, moz:profile:
C:\Users\Usuario\AppData\Lo..., moz:shutdownTimeout: 60000, moz:webdriverClick: true,
moz:windowless: false, pageLoadStrategy: normal, platformName: windows, proxy: Proxy(),
setWindowRect: true, strictFileInteractability: false, timeouts: {implicit: 0, pageLoad: 300000,
script: 30000}, unhandledPromptBehavior: dismiss and notify, userAgent: Mozilla/5.0 (Windows
NT 10....} Session ID: 99ba36c8-cde1-4990-b7e2-394b583ae6c0 at
java.base/jdk.internal.reflect.DirectConstructorHandleAccessor.newInstance(DirectConstructor
HandleAccessor.java:62) at
java.base/java.lang.reflect.Constructor.newInstanceWithCaller(Constructor.java:499) at
java.base/java.lang.reflect.Constructor.newInstance(Constructor.java:483)Build info: version:
'4.37.0', revision: '3c1da1e342' System info: os.name: 'Windows 10', os.arch: 'amd64',
os.version: '10.0', java.version: '25.0.1' Driver info: org.openqa.selenium.firefox.FirefoxDriver
Command: [99ba36c8-cde1-4990-b7e2-394b583ae6c0, findElement {value=.two > .card:nth-
child(1) > .p, using=css selector}] Capabilities {acceptInsecureCerts: true, browserName: firefox,
browserVersion: 147.0.4, moz:accessibilityChecks: false, moz:buildID: 20260212191108,
moz:geckodriverVersion: 0.36.0, moz:headless: false, moz:platformVersion: 10.0,
moz:processID: 12744, moz:profile: C:\Users\Usuario\AppData\Lo..., moz:shutdownTimeout:
60000, moz:webdriverClick: true, moz:windowless: false, pageLoadStrategy: normal,
platformName: windows, proxy: Proxy(), setWindowRect: true, strictFileInteractability: false,
timeouts: {implicit: 0, pageLoad: 300000, script: 30000}, unhandledPromptBehavior: dismiss and
notify, userAgent: Mozilla/5.0 (Windows NT 10....} Session ID: 99ba36c8-cde1-4990-b7e2-
394b583ae6c0 at
java.base/jdk.internal.reflect.DirectConstructorHandleAccessor.newInstance(DirectConstructor
HandleAccessor.java:62) at
java.base/java.lang.reflect.Constructor.newInstanceWithCaller(Constructor.java:499) at
java.base/java.lang.reflect.Constructor.newInstance(Constructor.java:483)
```

Ese log indica que ahora el fallo ya no es Cognito: **no encuentra el elemento** con selector CSS:

```
.two > .card:nth-child(1) > .p
```

Eso pasa por 1 de estas razones (casi siempre):

- **Todavía no has vuelto a tu app** después del login (sigues en Cognito o en una redirección).
- Tu app carga pero **tarda** y estás intentando clicar antes de que exista.
- El selector grabado es **demasiado frágil** (cambia con el HTML / el orden / responsive).

La solución es la misma estrategia: **espera explícita + selector más estable**.

1) Comprueba en qué página estás justo antes del click

Antes de esa línea, mete:

```
System.out.println("URL tras login: " + driver.getCurrentUrl());
```

Si NO ves `http://localhost:8080/...` entonces no estás en tu app todavía (o no logueó bien).

2) Cambia el click frágil por una espera a algo de TU app

En vez de clicar ese `.two > .card...`, espera a un elemento que tú sabes que aparece en tu home (por ejemplo el buscador `name="q"` que ya usas después).

Sustituye:

```
driver.findElement(By.cssSelector(".two > .card:nth-child(1) > .p")).click();
```

Por esto:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));

// espera a que estés en tu app (URL localhost)
wait.until(ExpectedConditions.urlContains("localhost:8080"));

// espera a que el buscador exista (tu app cargó)
wait.until(ExpectedConditions.elementToBeClickable(By.name("q")));
```

Y luego ya sigues con:

```
driver.findElement(By.name("q")).click();
driver.findElement(By.name("q")).sendKeys("a");
```

 Esto evita el click “intermedio” que no aporta nada si tu objetivo es buscar y reservar.

3) Si ese click es obligatorio, usa un selector estable

Ese selector `.two > .card:nth-child(1) > .p` es muy malo porque depende de estructura y orden.

Mejores opciones:

- `By.linkText("...")` si es un link con texto estable
- `By.cssSelector("[data-testid='algo']")` (lo mejor)
- `By.cssSelector("a[href='/ruta']")` si la ruta es fija

Ejemplo con `data-testid` (recomendado):

1. En tu HTML:

```
<a data-testid="card-libros" ...>...</a>
```

2. En Selenium:

```
wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-
testid='card-libros']"))).click();
```

4) Asegúrate de que usas wait en todo lo que sea “post-login”

Después del login, cada vez que hagas click en algo importante, mejor:

```
wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector(".primary"))).click();
```

Cambio mínimo recomendado en tu test

1. Asegúrate de tener estos imports (si no los tienes ya):

```
import java.time.Duration;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
```

2. Crea wait una vez al principio de test1():

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));
```

3. Sustituye el click frágil por esperar a name="q" (o a un elemento clave de tu home).

Si me dices **qué es exactamente ese elemento** que intentas clicar (¿una tarjeta “Libros”, “Reservas”, etc.), o me pegas el HTML de esa tarjeta, te doy el **selector exacto** para que sea estable y no dependa de nth-child.

```
"C:\Program Files\Java\jdk-25\bin\java.exe" -
javaagent:C:\Users\Usuario\AppData\Local\JetBrains\IntelliJ\Idea2025.3\captureAgent\debugg
er-
agent.jar=file:///C:/Users/Usuario/AppData/Local/Temp/capture5143522452902163226.props
-ea -Didea.test.cyclic.buffer.size=1048576 "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA
2025.3.1.1\lib\idea_rt.jar=57808" -Dkotlinx.coroutines.debug.enable.creation.stack.trace=false
-Ddebugger.agent.enable.coroutines=true -
Dkotlinx.coroutines.debug.enable.flows.stack.trace=true -
Dkotlinx.coroutines.debug.enable.mutable.state.flows.stack.trace=true -
Ddebugger.async.stack.trace.for.all.threads=true -Dfile.encoding=UTF-8 -
Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath
"C:\Users\Usuario\.m2\repository\org\junit\platform\junit-platform-launcher\6.0.1\junit-
platform-launcher-6.0.1.jar;C:\Program Files\JetBrains\IntelliJ IDEA
2025.3.1.1\lib\idea_rt.jar;C:\Program Files\JetBrains\IntelliJ IDEA
2025.3.1.1\plugins\junit\lib\junit6-rt.jar;C:\Program Files\JetBrains\IntelliJ IDEA
2025.3.1.1\plugins\junit\lib\junit5-rt.jar;C:\Program Files\JetBrains\IntelliJ IDEA
2025.3.1.1\plugins\junit\lib\junit-rt.jar;C:\Users\Usuario\tfm\Library\target\test-
classes;C:\Users\Usuario\tfm\Library\target\classes;C:\Users\Usuario\.m2\repository\org\spri
ngframework\boot\spring-boot-starter-data-jpa\4.0.1\spring-boot-starter-data-jpa-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-
starter\4.0.1\spring-boot-starter-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-
```

logging\4.0.1\spring-boot-starter-logging-
4.0.1.jar;C:\Users\Usuario\.m2\repository\ch\qos\logback\logback-classic\1.5.22\logback-
classic-1.5.22.jar;C:\Users\Usuario\.m2\repository\ch\qos\logback\logback-
core\1.5.22\logback-core-
1.5.22.jar;C:\Users\Usuario\.m2\repository\org\apache\logging\log4j\log4j-to-
slf4j\2.25.3\log4j-to-slf4j-
2.25.3.jar;C:\Users\Usuario\.m2\repository\org\apache\logging\log4j\log4j-api\2.25.3\log4j-
api-2.25.3.jar;C:\Users\Usuario\.m2\repository\org\slf4j\jul-to-slf4j\2.0.17\jul-to-slf4j-
2.0.17.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-
autoconfigure\4.0.1\spring-boot-autoconfigure-
4.0.1.jar;C:\Users\Usuario\.m2\repository\jakarta\annotation\jakarta.annotation-
api\3.0.0\jakarta.annotation-api-
3.0.0.jar;C:\Users\Usuario\.m2\repository\org\yaml\snakeyaml\2.5\snakeyaml-
2.5.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-
jdbc\4.0.1\spring-boot-starter-jdbc-
4.0.1.jar;C:\Users\Usuario\.m2\repository\com\zaxxer\HikariCP\7.0.2\HikariCP-
7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-data-
jpa\4.0.1\spring-boot-data-jpa-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-data-
commons\4.0.1\spring-boot-data-commons-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-
persistence\4.0.1\spring-boot-persistence-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\data\spring-data-
commons\4.0.1\spring-data-commons-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-
hibernate\4.0.1\spring-boot-hibernate-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-
jpa\4.0.1\spring-boot-jpa-
4.0.1.jar;C:\Users\Usuario\.m2\repository\jakarta\persistence\jakarta.persistence-
api\3.2.0\jakarta.persistence-api-
3.2.0.jar;C:\Users\Usuario\.m2\repository\org\hibernate\orm\hibernate-
core\7.2.0.Final\hibernate-core-
7.2.0.Final.jar;C:\Users\Usuario\.m2\repository\jakarta\transaction\jakarta.transaction-
api\2.0.1\jakarta.transaction-api-
2.0.1.jar;C:\Users\Usuario\.m2\repository\org\jboss\logging\jboss-logging\3.6.1.Final\jboss-
logging-3.6.1.Final.jar;C:\Users\Usuario\.m2\repository\org\hibernate\models\hibernate-
models\1.0.1\hibernate-models-
1.0.1.jar;C:\Users\Usuario\.m2\repository\com\fastxml\classmate\1.7.1\classmate-
1.7.1.jar;C:\Users\Usuario\.m2\repository\org\glassfish\jaxb\jaxb-runtime\4.0.6\jaxb-runtime-
4.0.6.jar;C:\Users\Usuario\.m2\repository\org\glassfish\jaxb\jaxb-core\4.0.6\jaxb-core-
4.0.6.jar;C:\Users\Usuario\.m2\repository\org\eclipse\angus\angus-activation\2.0.3\angus-
activation-2.0.3.jar;C:\Users\Usuario\.m2\repository\org\glassfish\jaxb\txw2\4.0.6\txw2-
4.0.6.jar;C:\Users\Usuario\.m2\repository\com\sun\istack\istack-commons-
runtime\4.1.2\istack-commons-runtime-
4.1.2.jar;C:\Users\Usuario\.m2\repository\jakarta\inject\jakarta.inject-api\2.0.1\jakarta.inject-
api-2.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\spring-orm\7.0.2\spring-
orm-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\data\spring-data-
jpa\4.0.1\spring-data-jpa-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\spring-context\7.0.2\spring-

context-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\spring-tx\7.0.2\spring-tx-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\spring-beans\7.0.2\spring-beans-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\antlr\antlr4-runtime\4.13.2\antlr4-runtime-4.13.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\spring-aspects\7.0.2\spring-aspects-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\aspectj\aspectjweaver\1.9.25.1\aspectjweaver-1.9.25.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-jdbc\4.0.1\spring-boot-jdbc-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot\4.0.1\spring-boot-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-sql\4.0.1\spring-boot-sql-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-transaction\4.0.1\spring-boot-transaction-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\spring-jdbc\7.0.2\spring-jdbc-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-security-oauth2-client\4.0.1\spring-boot-starter-security-oauth2-client-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-security\4.0.1\spring-boot-starter-security-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-security\4.0.1\spring-boot-security-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\security\spring-security-config\7.0.2\spring-security-config-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\security\spring-security-web\7.0.2\spring-security-web-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\spring-aop\7.0.2\spring-aop-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-security-oauth2-client\4.0.1\spring-boot-security-oauth2-client-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\security\spring-security-oauth2-client\7.0.2\spring-security-oauth2-client-7.0.2.jar;C:\Users\Usuario\.m2\repository\com\nimbusds\oauth2-oidc-sdk\11.26.1\oauth2-oidc-sdk-11.26.1.jar;C:\Users\Usuario\.m2\repository\com\github\stephenc\jcip\jcip-annotations\1.0-1\jcip-annotations-1.0-1.jar;C:\Users\Usuario\.m2\repository\com\nimbusds\content-type\2.3\content-type-2.3.jar;C:\Users\Usuario\.m2\repository\com\nimbusds\lang-tag\1.7\lang-tag-1.7.jar;C:\Users\Usuario\.m2\repository\org\springframework\security\spring-security-oauth2-jose\7.0.2\spring-security-oauth2-jose-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\security\spring-security-core\7.0.2\spring-security-core-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\security\spring-security-crypto\7.0.2\spring-security-crypto-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\spring-expression\7.0.2\spring-expression-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\security\spring-security-oauth2-core\7.0.2\spring-security-oauth2-core-7.0.2.jar;C:\Users\Usuario\.m2\repository\com\nimbusds\nimbus-jose-jwt\10.4\nimbus-jose-jwt-10.4.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-oauth2-resource-server\4.0.1\spring-boot-starter-oauth2-resource-server-

4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-security-oauth2-resource-server\4.0.1\spring-boot-security-oauth2-resource-server-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\security\spring-security-oauth2-resource-server\7.0.2\spring-security-oauth2-resource-server-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-webmvc\4.0.1\spring-boot-starter-webmvc-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-jackson\4.0.1\spring-boot-starter-jackson-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-jackson\4.0.1\spring-boot-jackson-4.0.1.jar;C:\Users\Usuario\.m2\repository\tools\jackson\core\jackson-databind\3.0.3\jackson-databind-3.0.3.jar;C:\Users\Usuario\.m2\repository\com\fasterxml\jackson\core\jackson-annotations\2.20\jackson-annotations-2.20.jar;C:\Users\Usuario\.m2\repository\tools\jackson\core\jackson-core\3.0.3\jackson-core-3.0.3.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-tomcat\4.0.1\spring-boot-starter-tomcat-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-tomcat-runtime\4.0.1\spring-boot-starter-tomcat-runtime-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\apache\tomcat\embed\tomcat-embed-core\11.0.15\tomcat-embed-core-11.0.15.jar;C:\Users\Usuario\.m2\repository\org\apache\tomcat\embed\tomcat-embed-el\11.0.15\tomcat-embed-el-11.0.15.jar;C:\Users\Usuario\.m2\repository\org\apache\tomcat\embed\tomcat-embed-websocket\11.0.15\tomcat-embed-websocket-11.0.15.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-tomcat\4.0.1\spring-boot-tomcat-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-http-converter\4.0.1\spring-boot-http-converter-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\spring-web\7.0.2\spring-web-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-webmvc\4.0.1\spring-boot-webmvc-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-servlet\4.0.1\spring-boot-servlet-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\spring-webmvc\7.0.2\spring-webmvc-7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-thymeleaf\4.0.1\spring-boot-starter-thymeleaf-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-thymeleaf\4.0.1\spring-boot-thymeleaf-4.0.1.jar;C:\Users\Usuario\.m2\repository\org\thymeleaf\extras\thymeleaf-extras-springsecurity6\3.1.3.RELEASE\thymeleaf-extras-springsecurity6-3.1.3.RELEASE.jar;C:\Users\Usuario\.m2\repository\org\thymeleaf\thymeleaf-spring6\3.1.3.RELEASE\thymeleaf-spring6-3.1.3.RELEASE.jar;C:\Users\Usuario\.m2\repository\org\thymeleaf\thymeleaf\3.1.3.RELEASE\thymeleaf-3.1.3.RELEASE.jar;C:\Users\Usuario\.m2\repository\org\attoparser\attoparser\2.0.7.RELEASE\attoparser-2.0.7.RELEASE.jar;C:\Users\Usuario\.m2\repository\org\unbescape\unbescape\1.1.6.RELEASE\unbescape-1.1.6.RELEASE.jar;C:\Users\Usuario\.m2\repository\org\slf4j\slf4j-api\2.0.17\slf4j-api-2.0.17.jar;C:\Users\Usuario\.m2\repository\com\h2database\h2\2.4.240\h2-

2.4.240.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-data-jpa-test\4.0.1\spring-boot-starter-data-jpa-test-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-jdbc-test\4.0.1\spring-boot-starter-jdbc-test-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-jdbc-test\4.0.1\spring-boot-jdbc-test-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-data-jpa-test\4.0.1\spring-boot-data-jpa-test-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-jpa-test\4.0.1\spring-boot-jpa-test-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-security-oauth2-client-test\4.0.1\spring-boot-starter-security-oauth2-client-test-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-security-test\4.0.1\spring-boot-starter-security-test-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-security-test\4.0.1\spring-boot-security-test-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\security\spring-security-test\7.0.2\spring-security-test-
7.0.2.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-webmvc-test\4.0.1\spring-boot-starter-webmvc-test-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-jackson-test\4.0.1\spring-boot-starter-jackson-test-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-webmvc-test\4.0.1\spring-boot-webmvc-test-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-web-server\4.0.1\spring-boot-web-server-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-resttestclient\4.0.1\spring-boot-resttestclient-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springdoc\springdoc-openapi-starter-webmvc-ui\3.0.1\springdoc-openapi-starter-webmvc-ui-
3.0.1.jar;C:\Users\Usuario\.m2\repository\org\springdoc\springdoc-openapi-starter-webmvc-api\3.0.1\springdoc-openapi-starter-webmvc-api-
3.0.1.jar;C:\Users\Usuario\.m2\repository\org\springdoc\springdoc-openapi-starter-common\3.0.1\springdoc-openapi-starter-common-
3.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-validation\4.0.1\spring-boot-validation-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\hibernate\validator\hibernate-validator\9.0.1.Final\hibernate-validator-
9.0.1.Final.jar;C:\Users\Usuario\.m2\repository\io\swagger\core\v3\swagger-core-jakarta\2.2.41\swagger-core-jakarta-
2.2.41.jar;C:\Users\Usuario\.m2\repository\io\swagger\core\v3\swagger-annotations-jakarta\2.2.41\swagger-annotations-jakarta-
2.2.41.jar;C:\Users\Usuario\.m2\repository\io\swagger\core\v3\swagger-models-jakarta\2.2.41\swagger-models-jakarta-
2.2.41.jar;C:\Users\Usuario\.m2\repository\jakarta\validation\jakarta.validation-api\3.1.1\jakarta.validation-api-
3.1.1.jar;C:\Users\Usuario\.m2\repository\com\fasterxml\jackson\dataformat\jackson-dataformat-yaml\2.20.1\jackson-dataformat-yaml-
2.20.1.jar;C:\Users\Usuario\.m2\repository\com\fasterxml\jackson\datatype\jackson-

datatype-jsr310\2.20.1\jackson-datatype-jsr310-
2.20.1.jar;C:\Users\Usuario\.m2\repository\org\webjars\swagger-ui\5.31.0\swagger-ui-
5.31.0.jar;C:\Users\Usuario\.m2\repository\org\webjars\webjars-locator-lite\1.1.2\webjars-
locator-lite-1.1.2.jar;C:\Users\Usuario\.m2\repository\org\jspecify\jspecify\1.0.0\jspecify-
1.0.0.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-
actuator\4.0.1\spring-boot-starter-actuator-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-
micrometer-metrics\4.0.1\spring-boot-starter-micrometer-metrics-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-
micrometer-metrics\4.0.1\spring-boot-micrometer-metrics-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-
micrometer-observation\4.0.1\spring-boot-micrometer-observation-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-actuator-
autoconfigure\4.0.1\spring-boot-actuator-autoconfigure-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-
actuator\4.0.1\spring-boot-actuator-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-
health\4.0.1\spring-boot-health-
4.0.1.jar;C:\Users\Usuario\.m2\repository\io\micrometer\micrometer-
observation\1.16.1\micrometer-observation-
1.16.1.jar;C:\Users\Usuario\.m2\repository\io\micrometer\micrometer-
commons\1.16.1\micrometer-commons-
1.16.1.jar;C:\Users\Usuario\.m2\repository\io\micrometer\micrometer-
jakarta9\1.16.1\micrometer-jakarta9-
1.16.1.jar;C:\Users\Usuario\.m2\repository\io\micrometer\micrometer-
core\1.16.1\micrometer-core-
1.16.1.jar;C:\Users\Usuario\.m2\repository\org\hdrhistogram\HdrHistogram\2.2.2\HdrHistogram-
am-
2.2.2.jar;C:\Users\Usuario\.m2\repository\org\latencyutils\LatencyUtils\2.0.3\LatencyUtils-
2.0.3.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-
java\4.18.1\selenium-java-
4.18.1.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-
api\4.37.0\selenium-api-
4.37.0.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-chrome-
driver\4.37.0\selenium-chrome-driver-
4.37.0.jar;C:\Users\Usuario\.m2\repository\com\google\auto\service\auto-service-
annotations\1.1.1\auto-service-annotations-
1.1.1.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-chromium-
driver\4.37.0\selenium-chromium-driver-
4.37.0.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-
json\4.37.0\selenium-json-
4.37.0.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-
manager\4.37.0\selenium-manager-
4.37.0.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-devtools-
v120\4.18.1\selenium-devtools-v120-
4.18.1.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-devtools-
v121\4.18.1\selenium-devtools-v121-
4.18.1.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-devtools-
v122\4.18.1\selenium-devtools-v122-

4.18.1.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-devtools-v85\4.18.1\selenium-devtools-v85-
4.18.1.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-edge-driver\4.37.0\selenium-edge-driver-
4.37.0.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-firefox-driver\4.37.0\selenium-firefox-driver-
4.37.0.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-http\4.37.0\selenium-http-
4.37.0.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-ie-driver\4.37.0\selenium-ie-driver-
4.37.0.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-remote-driver\4.37.0\selenium-remote-driver-
4.37.0.jar;C:\Users\Usuario\.m2\repository\com\google\guava\guava\33.5.0-jre\guava-33.5.0-jre.jar;C:\Users\Usuario\.m2\repository\com\google\guava\failureaccess\1.0.3\failureaccess-1.0.3.jar;C:\Users\Usuario\.m2\repository\com\google\guava\listenablefuture\9999.0-empty-to-avoid-conflict-with-guava\listenablefuture-9999.0-empty-to-avoid-conflict-with-guava.jar;C:\Users\Usuario\.m2\repository\com\google\j2objc\j2objc-annotations\3.1\j2objc-annotations-3.1.jar;C:\Users\Usuario\.m2\repository\io\opentelemetry\opentelemetry-api\1.55.0\opentelemetry-api-1.55.0.jar;C:\Users\Usuario\.m2\repository\io\opentelemetry\opentelemetry-context\1.55.0\opentelemetry-context-1.55.0.jar;C:\Users\Usuario\.m2\repository\io\opentelemetry\opentelemetry-common\1.55.0\opentelemetry-common-1.55.0.jar;C:\Users\Usuario\.m2\repository\io\opentelemetry\opentelemetry-exporter-logging\1.55.0\opentelemetry-exporter-logging-1.55.0.jar;C:\Users\Usuario\.m2\repository\io\opentelemetry\opentelemetry-sdk-common\1.55.0\opentelemetry-sdk-common-1.55.0.jar;C:\Users\Usuario\.m2\repository\io\opentelemetry\opentelemetry-sdk-extension-autoconfigure-spi\1.55.0\opentelemetry-sdk-extension-autoconfigure-spi-1.55.0.jar;C:\Users\Usuario\.m2\repository\io\opentelemetry\opentelemetry-sdk-extension-autoconfigure\1.55.0\opentelemetry-sdk-extension-autoconfigure-1.55.0.jar;C:\Users\Usuario\.m2\repository\io\opentelemetry\opentelemetry-sdk-trace\1.55.0\opentelemetry-sdk-trace-1.55.0.jar;C:\Users\Usuario\.m2\repository\io\opentelemetry\opentelemetry-sdk\1.55.0\opentelemetry-sdk-1.55.0.jar;C:\Users\Usuario\.m2\repository\io\opentelemetry\opentelemetry-sdk-metrics\1.55.0\opentelemetry-sdk-metrics-1.55.0.jar;C:\Users\Usuario\.m2\repository\io\opentelemetry\opentelemetry-sdk-logs\1.55.0\opentelemetry-sdk-logs-1.55.0.jar;C:\Users\Usuario\.m2\repository\net\bytebuddy\byte-buddy\1.17.8\byte-buddy-1.17.8.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-os\4.37.0\selenium-os-4.37.0.jar;C:\Users\Usuario\.m2\repository\org\apache\commons\commons-exec\1.5.0\commons-exec-1.5.0.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-safari-driver\4.37.0\selenium-safari-driver-4.37.0.jar;C:\Users\Usuario\.m2\repository\org\seleniumhq\selenium\selenium-support\4.37.0\selenium-support-4.37.0.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-starter-

test\4.0.3\spring-boot-starter-test-
4.0.3.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-
test\4.0.1\spring-boot-test-
4.0.1.jar;C:\Users\Usuario\.m2\repository\org\springframework\boot\spring-boot-test-
autoconfigure\4.0.1\spring-boot-test-autoconfigure-
4.0.1.jar;C:\Users\Usuario\.m2\repository\com\jayway\jsonpath\json-path\2.10.0\json-path-
2.10.0.jar;C:\Users\Usuario\.m2\repository\jakarta\xml\bind\jakarta.xml.bind-
api\4.0.4\jakarta.xml.bind-api-
4.0.4.jar;C:\Users\Usuario\.m2\repository\jakarta\activation\jakarta.activation-
api\2.1.4\jakarta.activation-api-2.1.4.jar;C:\Users\Usuario\.m2\repository\net\minidev\json-
smart\2.6.0\json-smart-2.6.0.jar;C:\Users\Usuario\.m2\repository\net\minidev\accessors-
smart\2.6.0\accessors-smart-
2.6.0.jar;C:\Users\Usuario\.m2\repository\org\ow2\asm\asm\9.7.1\asm-
9.7.1.jar;C:\Users\Usuario\.m2\repository\org\assertj\assertj-core\3.27.6\assertj-core-
3.27.6.jar;C:\Users\Usuario\.m2\repository\org\awaitility\awaitility\4.3.0\awaitility-
4.3.0.jar;C:\Users\Usuario\.m2\repository\org\hamcrest\hamcrest\3.0\hamcrest-
3.0.jar;C:\Users\Usuario\.m2\repository\org\junit\jupiter\junit-jupiter\6.0.1\junit-jupiter-
6.0.1.jar;C:\Users\Usuario\.m2\repository\org\junit\jupiter\junit-jupiter-api\6.0.1\junit-
jupiter-api-
6.0.1.jar;C:\Users\Usuario\.m2\repository\org\opentest4j\opentest4j\1.3.0\opentest4j-
1.3.0.jar;C:\Users\Usuario\.m2\repository\org\junit\platform\junit-platform-
commons\6.0.1\junit-platform-commons-
6.0.1.jar;C:\Users\Usuario\.m2\repository\org\apiguardian\apiguardian-api\1.1.2\apiguardian-
api-1.1.2.jar;C:\Users\Usuario\.m2\repository\org\junit\jupiter\junit-jupiter-
params\6.0.1\junit-jupiter-params-
6.0.1.jar;C:\Users\Usuario\.m2\repository\org\junit\jupiter\junit-jupiter-engine\6.0.1\junit-
jupiter-engine-6.0.1.jar;C:\Users\Usuario\.m2\repository\org\junit\platform\junit-platform-
engine\6.0.1\junit-platform-engine-
6.0.1.jar;C:\Users\Usuario\.m2\repository\org\mockito\mockito-core\5.20.0\mockito-core-
5.20.0.jar;C:\Users\Usuario\.m2\repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-
buddy-agent-
1.17.8.jar;C:\Users\Usuario\.m2\repository\org\objenesis\objenesis\3.3\objenesis-
3.3.jar;C:\Users\Usuario\.m2\repository\org\mockito\mockito-junit-jupiter\5.20.0\mockito-
junit-jupiter-
5.20.0.jar;C:\Users\Usuario\.m2\repository\org\skyscreamer\jsonassert\1.5.3\jsonassert-
1.5.3.jar;C:\Users\Usuario\.m2\repository\com\vaadin\external\google\android-
json\0.0.20131108.vaadin1\android-json-
0.0.20131108.vaadin1.jar;C:\Users\Usuario\.m2\repository\org\springframework\spring-
core\7.0.2\spring-core-7.0.2.jar;C:\Users\Usuario\.m2\repository\commons-
logging\commons-logging\1.3.5\commons-logging-
1.3.5.jar;C:\Users\Usuario\.m2\repository\org\springframework\spring-test\7.0.2\spring-test-
7.0.2.jar;C:\Users\Usuario\.m2\repository\org\xmlunit\xmlunit-core\2.10.4\xmlunit-core-
2.10.4.jar;C:\Users\Usuario\.m2\repository\io\github\bonigarcia\webdrivermanager\5.7.0\we-
bdrivermanager-
5.7.0.jar;C:\Users\Usuario\.m2\repository\com\google\code\gson\gson\2.13.2\gson-
2.13.2.jar;C:\Users\Usuario\.m2\repository\com\google\errorprone\error_prone_annotations
\2.41.0\error_prone_annotations-
2.41.0.jar;C:\Users\Usuario\.m2\repository\com\github\docker-java\docker-java\3.3.5\docker-
java-3.3.5.jar;C:\Users\Usuario\.m2\repository\com\github\docker-java\docker-java-

core\3.3.5\docker-java-core-3.3.5.jar;C:\Users\Usuario\.m2\repository\com\github\docker-
 java\docker-java-api\3.3.5\docker-java-api-
 3.3.5.jar;C:\Users\Usuario\.m2\repository\com\fastxml\jackson\core\jackson-
 databind\2.20.1\jackson-databind-
 2.20.1.jar;C:\Users\Usuario\.m2\repository\com\fastxml\jackson\core\jackson-
 core\2.20.1\jackson-core-
 2.20.1.jar;C:\Users\Usuario\.m2\repository\org\bouncycastle\bcprov-jdk18on\1.76\bcprov-
 jdk18on-1.76.jar;C:\Users\Usuario\.m2\repository\org\bouncycastle\bcprov-
 jdk18on\1.76\bcprov-jdk18on-
 1.76.jar;C:\Users\Usuario\.m2\repository\org\bouncycastle\bcutil-jdk18on\1.76\bcutil-
 jdk18on-1.76.jar;C:\Users\Usuario\.m2\repository\org\slf4j\jcl-over-slf4j\2.0.17\jcl-over-slf4j-
 2.0.17.jar;C:\Users\Usuario\.m2\repository\com\github\docker-java\docker-java-transport-
 httpclient5\3.3.5\docker-java-transport-httpclient5-
 3.3.5.jar;C:\Users\Usuario\.m2\repository\com\github\docker-java\docker-java-
 transport\3.3.5\docker-java-transport-
 3.3.5.jar;C:\Users\Usuario\.m2\repository\net\java\dev\jna\jna\5.13.0\jna-
 5.13.0.jar;C:\Users\Usuario\.m2\repository\org\broTLi\dec\0.1.2\dec-
 0.1.2.jar;C:\Users\Usuario\.m2\repository\org\apache\commons\commons-
 lang3\3.19.0\commons-lang3-
 3.19.0.jar;C:\Users\Usuario\.m2\repository\org\apache\commons\commons-
 compress\1.26.0\commons-compress-1.26.0.jar;C:\Users\Usuario\.m2\repository\commons-
 io\commons-io\2.15.1\commons-io-
 2.15.1.jar;C:\Users\Usuario\.m2\repository\org\apache\httpcomponents\client5\httpclient5\5.
 .5.1\httpclient5-
 5.5.1.jar;C:\Users\Usuario\.m2\repository\org\apache\httpcomponents\core5\httpcore5\5.3.
 6\httpcore5-
 5.3.6.jar;C:\Users\Usuario\.m2\repository\org\apache\httpcomponents\core5\httpcore5-
 h2\5.3.6\httpcore5-h2-
 5.3.6.jar;C:\Users\Usuario\.m2\repository\org\projectlombok\lombok\1.18.42\lombok-
 1.18.42.jar" com.intellij.rt.junit.JUnitStarter -ideVersion5 -junit6
 com.upm.library.ReservationTest.test1 org.openqa.selenium.TimeoutException: Expected
 condition failed: waiting for element to be clickable: By.name: q (tried for 15 second(s) with 500
 milliseconds interval) Build info: version: '4.37.0', revision: '3c1da1e342' System info: os.name:
 'Windows 10', os.arch: 'amd64', os.version: '10.0', java.version: '25.0.1' Driver info:
 org.openqa.selenium.firefox.FirefoxDriver Capabilities {acceptInsecureCerts: true,
 browserName: firefox, browserVersion: 147.0.4, moz:accessibilityChecks: false, moz:buildID:
 20260212191108, moz:geckodriverVersion: 0.36.0, moz:headless: false, moz:platformVersion:
 10.0, moz:processID: 26936, moz:profile: C:\Users\Usuario\AppData\Lo...,
 moz:shutdownTimeout: 60000, moz:webdriverClick: true, moz:windowless: false,
 pageLoadStrategy: normal, platformName: windows, proxy: Proxy(), setWindowRect: true,
 strictFileInteractability: false, timeouts: {implicit: 0, pageLoad: 300000, script: 30000},
 unhandledPromptBehavior: dismiss and notify, userAgent: Mozilla/5.0 (Windows NT 10....}
 Session ID: e1e61b35-c1ba-4d00-9a86-072875cd5bb50 at
 org.openqa.selenium.support.ui.WebDriverWait.timeoutException(WebDriverWait.java:84) at
 org.openqa.selenium.support.ui.FluentWait.until(FluentWait.java:228) at
 com.upm.library.ReservationTest.test1(ReservationTest.java:55)

Ese TimeoutException con By.name: q te está diciendo algo muy concreto:

✅ **El login NO está llegando a tu aplicación** (o llega pero no a la pantalla donde existe name="q").

Por eso, 15s después, Selenium sigue sin ver ese input.

En tu caso hay OAuth con Cognito → lo más típico es que ocurra una de estas 3 cosas:

1. **Sigue en Cognito** (login fallido / MFA / “new password required” / captcha / error)
2. **Se queda en una URL intermedia** (callback /login/oauth2/code/cognito o error de Spring Security)
3. **Tu app sí carga pero el buscador no está en esa vista** (selector incorrecto)

Vamos a arreglarlo sin adivinar.

1) Imprime la URL y el título justo después de enviar la contraseña

Justo después de:

```
password.sendKeys(Keys.ENTER);
```

añade:

```
System.out.println("URL tras password: " + driver.getCurrentUrl());  
System.out.println("TITLE: " + driver.getTitle());
```

👉 Si la URL sigue siendo https://...amazoncognito.com/... ⇒ no has salido de Cognito.

👉 Si es http://localhost:8080/login?error o similar ⇒ tu app rechazó el login.

👉 Si es http://localhost:8080/... pero no hay q ⇒ estás en otra página.

2) Espera explícitamente a volver a localhost:8080

En OAuth el redirect puede tardar más. Cambia tu espera a esto (más fiable):

```
wait.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.co  
m")));  
wait.until(ExpectedConditions.urlContains("localhost:8080"));
```

Y **solo después** intenta buscar elementos de tu app.

3) Sustituye la espera de name="q" por algo que seguro exista

Muchas veces name="q" no existe en el HTML final (o cambia).
Mejor espera a algo “global” de tu app:

- un elemento del navbar
- el <body> con una clase concreta
- un texto “Biblioteca”, “Inicio”, etc.
- o simplemente que el DOM esté listo

Ejemplo esperando el body:

```
wait.until(ExpectedConditions.presenceOfElementLocated(By.tagName("body")));
```

Y luego **log** para ver qué hay:

```
System.out.println(driver.getPageSource().substring(0, 5000));
```

(temporal para depurar)

4) Aumenta el timeout del post-login (OAuth suele tardar)

Pon 30s tras el login:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(30));
```

5) Muy probable: el login requiere CLICK en botón, no ENTER

En Cognito a veces ENTER no dispara el submit como esperas.

Cambia el submit a click:

```
WebElement signIn = wait.until(ExpectedConditions.elementToBeClickable(  
By.cssSelector("button[type='submit']")  
));  
signIn.click();
```

Hazlo tanto tras email como tras password si hay pantallas separadas.

6) Solución “pro” para E2E: evita Cognito en los tests

Esto es importante: los E2E con IdP externo suelen ser frágiles (cambios de UI, MFA, rate limits).

Lo habitual en proyectos es tener un **perfil e2e** que:

- habilite un “login de test” (solo en test)
- o un usuario mockeado
- o un bypass de OAuth para entorno local/CI

Si te interesa, te digo cómo hacerlo con Spring Security sin tocar prod.

Te dejo el bloque recomendado (cópialo tal cual)

En tu test1(), justo después de enviar password:

```
password.sendKeys(Keys.ENTER);
```

```
System.out.println("URL      tras      password:  "      +      driver.getCurrentUrl());  
System.out.println("TITLE:    "      +      driver.getTitle());
```

```
// Espera a salir de Cognito y volver a tu app
wait.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
wait.until(ExpectedConditions.urlContains("localhost:8080"));
```

```
System.out.println("URL en app: " + driver.getCurrentUrl());
```

```
// Ahora sí: busca el elemento de tu app (si existe)
wait.until(ExpectedConditions.presenceOfElementLocated(By.tagName("body")));
```

Luego, en vez de `By.name("q")`, busca algo real de tu home (si me dices qué es, te doy selector exacto).

Lo mínimo que necesito de ti para afinarlo al 100%

Pégame la salida de:

- URL tras password: ...
- URL en app: ... (si llega)

Con esa URL exacta te digo qué condición esperar (y si estás en `/login/oauth2/code/...`, `/login?error`, etc.).

URL tras password: `https://us-east-1snnleupih.auth.us-east-1.amazoncognito.com/verifyPassword?client_id=1h2dcs34d8ucv154k44joac1bc&code_challenge=GLP5bSnpjlkntR-gRYCOm2bXxZdZsbRfO2JAEU_qcuc&code_challenge_method=S256&nonce=uOQBpTFKR--1Gkg9BGewVGa5QKb5AANvDnPebUDiXEo&redirect_uri=http%3A%2F%2Flocalhost%3A8080%2Flogin%2Foauth2%2Fcode%2Fcognito&response_type=code&scope=phone+openid+email&state=k9LV339_KKlMvC11kePad79i-fEj8rqS0IneHuu-JQ%3D` TITLE: Enter your password

Perfecto: con eso ya está clarísimo 

Estás **todavía en Cognito** en la página `verifyPassword` (título “Enter your password”). O sea: **no estás enviando el formulario de login** (o Cognito no lo acepta), por eso nunca vuelves a `localhost:8080` y luego no aparece `name="q"`.

La solución: **no uses ENTER** para “enviar”, haz **click en el botón de Sign in/Continue** y espera a que cambie la URL. Además, usa un selector más robusto para el botón.

Cambios exactos que tienes que hacer

1) En la parte del password: reemplaza el ENTER por click al botón submit

Sustituye:

```
driver.findElement(By.id("formField:r1:")).sendKeys("1234567aA!");
driver.findElement(By.id("formField:r1:")).sendKeys(Keys.ENTER);
```

por algo así (con wait):

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(30));

// password (mejor por name)
WebElement password =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("password")));
password.clear();
password.sendKeys("1234567aA!");

// click en el botón de submit (en Cognito suele ser type=submit)
WebElement submit =
wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("button[type='submit']")));
submit.click();
```

2) Luego espera a salir de Cognito y volver a tu app

Justo después del click:

```
wait.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.co
m")));
wait.until(ExpectedConditions.urlContains("localhost:8080"));
```

Importante: a veces Cognito tiene 2 botones / texto distinto

Si button[type='submit'] no lo pilla, alternativas típicas:

```
// por texto visible (si es un <button>)
WebElement submit = wait.until(ExpectedConditions.elementToBeClickable(
    By.xpath("//button[contains(., 'Sign in') or contains(., 'Iniciar sesión') or contains(., 'Continue') or
contains(., 'Continuar')]")
));
submit.click();
```

(Es feo, pero funciona si Cognito cambia el DOM.)

Bonus: el email también mejor con click en submit

En algunas configuraciones, Cognito tiene “Next” para pasar a password. Si tu flujo es “email → next → password”, haz:

```
WebElement email =
wait.until(ExpectedConditions.elementToBeClickable(By.name("username")));
email.clear();
email.sendKeys("prueba@prueba.com");

WebElement submit1 =
wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("button[type='submit']")));
submit1.click();
```

y luego esperas al password.

Por qué ENTER no te vale aquí

En Cognito Hosted UI, el campo puede estar dentro de componentes que **no envían el form con ENTER** (o el foco está en otro sitio), así que te quedas en `verifyPassword` aunque hayas escrito bien.

Siguiente paso si tras click sigue en Cognito

Si tras hacer click sigues en `amazoncognito.com`, entonces hay un **error de login** o un paso extra (MFA, “new password required”, captcha). En ese caso, añade esto para ver el mensaje de error:

```
System.out.println("URL tras submit: " + driver.getCurrentUrl());
System.out.println("Page contains error? " +
driver.getPageSource().toLowerCase().contains("error"));
```

o espera a un div de error (si existe) y lo imprimes.

Si quieres, pégame el HTML del botón (inspeccionar → copiar <button ...>) de esa pantalla “Enter your password” y te doy el selector **perfecto** para Cognito en tu caso.

```
org.openqa.selenium.TimeoutException: Expected condition failed: waiting for element
to be clickable: By.cssSelector: button[type='submit'] (tried for 15 second(s) with 500
milliseconds interval) Build info: version: '4.37.0', revision: '3c1da1e342' System info: os.name:
'Windows 10', os.arch: 'amd64', os.version: '10.0', java.version: '25.0.1' Driver info:
org.openqa.selenium.firefox.FirefoxDriver Capabilities {acceptInsecureCerts: true,
browserName: firefox, browserVersion: 147.0.4, moz:accessibilityChecks: false, moz:buildID:
20260212191108, moz:geckodriverVersion: 0.36.0, moz:headless: false, moz:platformVersion:
10.0, moz:processID: 27340, moz:profile: C:\Users\Usuario\AppData\Lo...,
moz:shutdownTimeout: 60000, moz:webdriverClick: true, moz:windowless: false,
pageLoadStrategy: normal, platformName: windows, proxy: Proxy(), setWindowRect: true,
strictFileInteractability: false, timeouts: {implicit: 0, pageLoad: 300000, script: 30000},
unhandledPromptBehavior: dismiss and notify, userAgent: Mozilla/5.0 (Windows NT 10....}
Session ID: 3b1e2db7-ea43-41f3-8891-e28aa710519b
```

Eso significa que **en esa pantalla de Cognito no existe (o no es “clickable”)** un `button[type="submit"]`. Cognito a veces usa:

- `<input type="submit">`
- un `<button>` sin type
- o un “Sign in” que es un `<a>/div` con `role=button`

Así que hay que buscar el botón de forma más flexible.

Cambia el selector del botón por uno “multi-opción”

1) Importa ByAll (opcional) o usa try/catch (más simple)

Vamos con lo simple: intenta varios selectores hasta encontrar uno.

Añade estos imports si no los tienes:

```
import org.openqa.selenium.WebElement;
import org.openqa.selenium.TimeoutException;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import java.time.Duration;
```

2) Sustituye la búsqueda del submit por este bloque

Justo después de escribir el password:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(30));
```

```
// password
WebElement password =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("input[type='password']")));
password.clear();
password.sendKeys("1234567aA!");
```

```
// intenta encontrar un botón de envío de varias formas
WebElement submit;
try {
    submit =
wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("button[type='submit']")));
} catch (TimeoutException e1) {
    try {
        submit =
wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("input[type='submit']")));
    } catch (TimeoutException e2) {
        // último recurso: botón por texto
        submit = wait.until(ExpectedConditions.elementToBeClickable(
            By.xpath("//*[self::button or self::a or self::div][contains(., 'Sign in') or contains(., 'Iniciar sesión') or contains(., 'Continue') or contains(., 'Continuar')]")));
    }
}
```

```
submit.click();
```

Esto cubre la mayoría de variantes de Cognito Hosted UI.

Si sigue sin encontrarlo: casi seguro está dentro de un iframe (o el texto es diferente)

Para salir de dudas, mete un “debug” justo antes de buscar el submit:

```
System.out.println("URL actual: " + driver.getCurrentUrl());
System.out.println("TITLE: " + driver.getTitle());
System.out.println("Hay iframes: " + driver.findElements(By.tagName("iframe")).size());
```

Si hay iframes (>0)

Cognito no suele, pero si en tu caso sí, tendrás que hacer:

```
driver.switchTo().frame(0); // o el iframe correcto
```

y luego buscar el botón.

Recomendación importante (te va a ahorrar horas)

No uses Cognito UI para E2E si puedes evitarlo. Es normal que:

- cambie el DOM,
- meta pasos extra (MFA),
- y rompa tests.

Lo típico en proyectos E2E es:

- tener un “modo e2e” en Spring que **simule login** (solo en test),
- o usar un token prehecho / endpoint de test.

Pero para que lo termines ya, con lo de arriba debería avanzar.

Si me pegas **una captura del HTML del botón** en la página “Enter your password” (inspeccionar → copia el elemento), te doy el selector exacto (sin “último recurso” con xpath).

```
@Test public void test1() { driver.get("https://us-east-1snnleupih.auth.us-east-1.amazoncognito.com/login?client_id=1h2dcs34d8ucv154k44joac1bc&code_challenge=GLP5bSnpjlkntR-gRYCOm2bXxZdZsbRfO2JAEU_qcuc&code_challenge_method=S256&nonce=uOQBpTFKR--1Gkg9BGewVGa5QKb5AANvDnPebUDiXEo&redirect_uri=http://localhost:8080/login/oauth2/code/cognito&response_type=code&scope=phone+openid+email&state=k9LV339_KKImRvC11kePad79i-fEj8rqS0lneHuu-JQ%3D"); driver.manage().window().setSize(new Dimension(1936, 1056)); WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15)); WebElement email = wait.until(ExpectedConditions.elementToBeClickable(By.name("username"))); email.sendKeys("prueba@prueba.com"); //email.sendKeys(Keys.ENTER); WebElement submit = wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("button[type='submit']"))); submit.click(); WebElement password = wait.until(ExpectedConditions.visibilityOfElementLocated(By.name("password"))); password.sendKeys("1234567aA!"); //password.sendKeys(Keys.ENTER); WebElement submit1 = wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("button[type='submit']"))); submit1.click(); System.out.println("URL tras password: " + driver.getCurrentUrl());
```

```

System.out.println("TITLE:  " + driver.getTitle()); WebDriverWait wait2 = new
WebDriverWait(driver, Duration.ofSeconds(15)); // espera a que estés en tu app (URL localhost)
wait2.until(ExpectedConditions.urlContains("localhost:8080")); // espera a que el buscador
exista (tu app cargó) wait2.until(ExpectedConditions.elementToBeClickable(By.name("q")));
driver.findElement(By.cssSelector(".two > .card:nth-child(1) > .p")).click();
driver.findElement(By.name("q")).click(); driver.findElement(By.name("q")).sendKeys("a");
driver.findElement(By.cssSelector(".primary")).click(); driver.findElement(By.linkText("Ver
ejemplares →")).click(); driver.findElement(By.linkText("Reservar ejemplar")).click();
driver.findElement(By.cssSelector(".primary")).click();
driver.findElement(By.cssSelector(".danger")).click();
assertThat(driver.switchTo().alert().getText(), is("¿Cancelar esta reserva?"));
driver.switchTo().alert().accept(); driver.findElement(By.cssSelector(".ghost")).click(); } tengo ese
codigo y este es el error: URL tras password: https://us-east-1snnleupih.auth.us-east-
1.amazoncognito.com/verifyPassword?client_id=1h2dcs34d8ucv154k44joac1bc&code_challen
ge=GLP5bSnpiJkntR-
gRYCOm2bXxZdZsbRfO2JAEU_qcuc&code_challenge_method=S256&nonce=uOQBpTFKR--
1Gkg9BGewVGa5QKb5AANvDnPebUDiXeo&redirect_uri=http%3A%2F%2Flocalhost%3A8080%
2Flogin%2Foauth2%2Fcode%2Fcognito&response_type=code&scope=phone+openid+email&st
ate=k9LV339_KKImRvC11kePad79i-fEj8rqS0IneHuu-JQ%3D TITLE: Enter your password
org.openqa.selenium.TimeoutException: Expected condition failed: waiting for element to be
clickable: By.name: q (tried for 15 second(s) with 500 milliseconds interval) Build info: version:
'4.37.0', revision: '3c1da1e342' System info: os.name: 'Windows 10', os.arch: 'amd64',
os.version: '10.0', java.version: '25.0.1' Driver info: org.openqa.selenium.firefox.FirefoxDriver
Capabilities {acceptInsecureCerts: true, browserName: firefox, browserVersion: 147.0.4,
moz:accessibilityChecks: false, moz:buildID: 20260212191108, moz:geckodriverVersion: 0.36.0,
moz:headless: false, moz:platformVersion: 10.0, moz:processID: 28320, moz:profile:
C:\Users\Usuario\AppData\Lo..., moz:shutdownTimeout: 60000, moz:webdriverClick: true,
moz:windowless: false, pageLoadStrategy: normal, platformName: windows, proxy: Proxy(),
setWindowRect: true, strictFileInteractability: false, timeouts: {implicit: 0, pageLoad: 300000,
script: 30000}, unhandledPromptBehavior: dismiss and notify, userAgent: Mozilla/5.0 (Windows
NT 10....) Session ID: 24cef6e0-56c9-4c16-a410-b70160377261 Cuando abre la ventana de
firefox para el test, la pagina se queda como congelado al meter el pasword y luego da timeout

```

Ya está clarísimo lo que pasa: **el click al “submit” NO está disparando el login real en Cognito**, así que te quedas en verifyPassword (título “Enter your password”). Por eso nunca vuelves a localhost:8080 y acaba en timeout esperando q.

Y lo de “se queda congelado” suele ser una de estas 3 cosas en Cognito Hosted UI + Selenium:

1. **Hay un overlay/spinner** que tapa el botón o la página (el click no llega).
2. **El botón está deshabilitado** hasta que el formulario valida (y Selenium cree que hizo click pero no).
3. **Cognito muestra un error** (credenciales inválidas / password policy / captcha / MFA) pero tu test no lo está leyendo.

Vamos a atacarlo con un bloque robusto que:

- hace click “normal” y si no, click por JS (para overlays)
- espera a salir de Cognito
- si NO sale, imprime el error visible de Cognito

Cambios concretos (copia/pega)

1) NO crees wait2. Usa un único wait más largo

Pon 30s:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(30));
```

2) Reemplaza TODO tu login por este bloque

(Sustituye desde WebElement email = ... hasta el System.out.println)

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(30));
```

```
WebElement email =
wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("input[name='username'],
input[type='email']"))));
email.clear();
email.sendKeys("prueba@prueba.com");
```

```
// botón "next/sign in" (puede ser button o input)
WebElement submitEmail = wait.until(ExpectedConditions.presenceOfElementLocated(
    By.cssSelector("button[type='submit'], input[type='submit']")
));
wait.until(ExpectedConditions.elementToBeClickable(submitEmail)).click();
```

```
// espera a que el password exista
WebElement password =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.cssSelector("input[name='password'],
input[type='password']"))));
password.clear();
password.sendKeys("1234567aA!");
```

```
// click robusto (normal -> si no funciona, JS click)
WebElement submitPwd = wait.until(ExpectedConditions.presenceOfElementLocated(
    By.cssSelector("button[type='submit'], input[type='submit']")
));
try {
    wait.until(ExpectedConditions.elementToBeClickable(submitPwd)).click();
} catch (Exception ex) {
    ((JavascriptExecutor) driver).executeScript("arguments[0].click();", submitPwd);
}
```

```
// DEBUG
System.out.println("URL tras submit password: " + driver.getCurrentUrl());
System.out.println("TITLE tras submit password: " + driver.getTitle());
```

3) Cambia tu espera a localhost para que no “pase” si sigue en Cognito

Ahora mismo esperas solo `urlContains("localhost:8080")`. Si Cognito se queda “pensando” te comes un timeout sin saber por qué.

Pon esto justo después del debug:

```
// Espera a salir de Cognito. Si NO sales, vamos a capturar el error.
try {
    wait.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
    wait.until(ExpectedConditions.urlContains("localhost:8080"));
} catch (TimeoutException te) {
    System.out.println("SIGUE EN COGNITO. URL: " + driver.getCurrentUrl());
    // intenta imprimir un mensaje de error visible (muy típico en Cognito)
    var errors = driver.findElements(By.cssSelector("[data-testid='form-error'], .error, .text-error, p[role='alert']"));
    if (!errors.isEmpty()) {
        System.out.println("ERROR VISIBLE: " + errors.get(0).getText());
    } else {
        System.out.println("No se encontró selector de error típico. Mira la pantalla: sigue en Cognito.");
    }
    throw te;
}
```

Con esto, si se queda en Cognito, **verás por consola el motivo** (muchas veces “Incorrect username or password”, “Password attempts exceeded”, MFA, etc.).

4) El motivo más frecuente en tu caso

Si “se congela” justo al enviar password y no redirige, suele ser:

- **Credenciales incorrectas**
- **MFA/Challenge** activado (SMS code, TOTP)
- **Captcha/bot protection**
- **Cognito Hosted UI bloqueando automatización** (raro, pero pasa)

Tu test actual **no maneja MFA**. Si tu pool tiene MFA, **jamás** llegará a localhost:8080 sin meter el código.

Cómo saber si es MFA

En cuanto falle, mira si la URL cambia a algo tipo:

- `/mfa /verify /challenge` o aparece un input de “code”.

Puedes detectarlo así:

```
System.out.println("¿Hay campo code? " +
!driver.findElements(By.cssSelector("input[name='code'], input[autocomplete='one-time-code']")).isEmpty());
```

5) Y lo más importante para E2E (recomendación realista)

Probar E2E **pasando por Cognito UI** es de lo más frágil que existe.

Lo normal en E2E es:

- **Bypass de login en perfil e2e** (Spring Security) o
- **obtener token** vía API (sin UI) y setear sesión/cookies

Si quieres, te doy el patrón “limpio” para Spring Boot:

- un perfil e2e con un SecurityFilterChain que autentica automáticamente a un usuario de test en /e2e-login
- y así tus tests prueban tu app, no el IdP.

Dime esto (y no hace falta más)

Cuando falle con el bloque de arriba, pégame la línea:

- SIGUE EN COGNITO. URL: ...
y si sale:
- ERROR VISIBLE: ...

Con eso te digo si es **credenciales, MFA, captcha o botón mal detectado**, y te lo dejo funcionando.

```
RL      tras      submit      password:      https://us-east-1snnleupih.auth.us-east-1.amazoncognito.com/verifyPassword?client_id=1h2dcs34d8ucv154k44joac1bc&code_challenge=GLP5bSnpjlkntR-gRYCOm2bXxZdZsbRfO2JAEU_qcuc&code_challenge_method=S256&nonce=uOQBpTFKR--1Gkg9BGewVGa5QKb5AANvDnPebUDiXeo&redirect_uri=http%3A%2F%2Flocalhost%3A8080%2Flogin%2Foauth2%2Fcode%2Fcognito&response_type=code&scope=phone+openid+email&state=k9LV339_KKImRvC11kePad79i-fEj8rqS0IneHuu-JQ%3D TITLE tras submit password: Enter your password org.openqa.selenium.NoSuchElementException: Unable to locate element: .two > .card:nth-child(1) > .p For documentation on this error, please visit: https://www.selenium.dev/documentation/webdriver/troubleshooting/errors#nosuchelementexception Build info: version: '4.37.0', revision: '3c1da1e342' System info: os.name: 'Windows 10', os.arch: 'amd64', os.version: '10.0', java.version: '25.0.1' Driver info: org.openqa.selenium.firefox.FirefoxDriver
```

Vas bien: ese output confirma 100% que **sigues en Cognito** después de “enviar” el password:

- URL sigue en amazoncognito.com/verifyPassword

- TITLE sigue "Enter your password"

Así que **el login no se está completando** (no hay redirect a localhost:8080). Por eso luego falla buscando .two > ... (eso solo existe en tu app, no en Cognito).

Ahora mismo tu test está avanzando aunque no hayas salido de Cognito. Hay que hacer dos cosas:

1. **Parar el test si no sales de Cognito** (para no buscar elementos de tu app).
2. **Capturar el mensaje de error** que Cognito está mostrando (o detectar MFA/captcha).

1) Obliga a que el test NO continúe si sigues en Cognito

Justo después del submitPwd.click() (y tus prints), mete esto:

```
// Espera a que cambie a tu app
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(30));
try {
    wait.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
    wait.until(ExpectedConditions.urlContains("localhost:8080"));
} catch (TimeoutException e) {
    // Si no sales de Cognito, imprime info útil y falla el test aquí mismo
    System.out.println("NO SALIÓ DE COGNITO. URL: " + driver.getCurrentUrl());
    System.out.println("TITLE: " + driver.getTitle());

    // Intenta sacar mensaje de error típico de Cognito
    var alerts = driver.findElements(By.cssSelector("[role='alert'], .text-error, .error, .alert, .banner"));
    for (var a : alerts) {
        String t = a.getText().trim();
        if (!t.isEmpty()) System.out.println("ALERT: " + t);
    }

    // Detecta MFA / código
    boolean hayCode = !driver.findElements(By.cssSelector("input[autocomplete='one-time-code'], input[name*='code'], input[id*='code']")).isEmpty();
    System.out.println("¿Pide código/MFA?: " + hayCode);

    throw e;
}
```

 Con esto ya no te saldrá el .two... porque el test se corta antes.

2) Por qué Cognito no avanza (causas típicas)

Si tras enviar password sigues en verifyPassword, casi siempre es:

A) Credenciales incorrectas

Cognito se queda en la misma pantalla y muestra un error (a veces en un role="alert").

B) MFA / Challenge requerido

Te pedirá código SMS/TOTP. En ese caso aparecerá un input de “code”.

C) Captcha / bot protection

A veces hay un widget o bloqueo silencioso (menos común, pero posible).

D) Estás clicando un “submit” que no es el correcto

En Cognito a veces hay varios botones o el botón real está “tapado” por overlay y el click no llega.

3) Haz el click todavía más fiable (scroll + click)

Antes del click al submit de password:

```
((JavascriptExecutor)
driver).executeScript("arguments[0].scrollIntoView({block:'center'});", submitPwd);
Thread.sleep(200); // solo para estabilizar Cognito UI
try {
    submitPwd.click();
} catch (Exception ex) {
    ((JavascriptExecutor) driver).executeScript("arguments[0].click();", submitPwd);
}
```

(Sí: Thread.sleep no es elegante, pero con IdPs externos a veces es la diferencia entre “funciona” y “flaky”).

4) Si sale MFA, tu test tiene que introducir el código

Si el log te imprime ¿Pide código/MFA?: true, entonces:

- **no podrás completar el login** sin meter ese OTP
- para E2E, lo ideal es **desactivar MFA en el usuario de test** o usar un modo e2e en tu app.

5) Recomendación para que puedas hacer E2E de verdad

Si tu objetivo es probar *reservas en tu app*, lo más sensato es **no automatizar Cognito UI**.

En Spring Boot, para profile=e2e puedes:

- habilitar un endpoint /e2e-login que haga login automático como usuario de test
- o permitir una “mock auth” solo en local/CI

Así tus E2E prueban tu aplicación, no el proveedor OAuth (que cambia y mete MFA/captcha).

Qué necesito para decirte la causa exacta (y solucionarlo)

Cuando pongas el bloque de “falla si no sales de Cognito”, pégame lo que imprima como:

- ALERT: ... (si aparece)
- ¿Pide código/MFA?: true/false

Con eso te digo si es:

- password incorrecta,
 - MFA,
 - o selector/click incorrecto,
- y el siguiente cambio exacto.

porque no pasa del login? puede que amazon cognito lo bloquee de alguna forma? igual se puede hacer el login con un secret para la aplicacion?

No pasa del login porque **Cognito no está completando el flujo** (no emite el redirect de vuelta a <http://localhost:8080/login/oauth2/code/cognito>). Eso puede ocurrir por varias causas, y sí: **puede “bloquear” automatizaciones**, pero lo más habitual es algo más simple.

Motivos más comunes (en orden de probabilidad)

1) Credenciales no válidas o usuario en estado “raro”

Cognito se queda en verifyPassword si:

- password incorrecta
- usuario no confirmado
- usuario bloqueado por intentos
- “new password required”
- necesita aceptar términos, etc.

Normalmente muestra un mensaje, pero a veces está en un div que tu test no imprime.

2) MFA / desafío adicional

Si el user pool tiene MFA (SMS/TOTP) o “challenge”:

- tras enviar password **no redirige**
- pasa a pantalla de código (o se queda en verifyPassword si no avanza bien)

3) Bot / anti-automation / captcha / rate limit

Puede pasar, sobre todo si:

- haces muchos intentos seguidos

- el navegador parece automatizado
- hay WAF delante (CloudFront/WAF)

Pero Cognito “puro” no suele meter captcha; lo verías si tienes algo extra delante (WAF, custom domain con protección, etc.).

4) El click “submit” no está haciendo submit de verdad

Por overlays/spinners, botón deshabilitado, DOM dinámico, etc. (por eso insistí en click robusto y en imprimir errores).

¿Se puede hacer login “con un secret” para la app?

No de la forma que imaginas para un test E2E de navegador.

- El **client secret** (si tu app es “confidential”) se usa en el **backend**, en el intercambio del code por tokens, **no** para loguear desde el navegador.
- En el flujo normal “Authorization Code + PKCE” (muy típico en apps web), el navegador no debería usar secrets.

Lo que *sí* puedes hacer (y es lo recomendable para E2E):

Opción A (recomendada): evitar la UI de Cognito en E2E

1) “Bypass” de login solo en profile=e2e

En Spring Security creas un perfil e2e que:

- permite un endpoint tipo /e2e-login
- al visitarlo te autentica como usuario de test
- y redirige a /

✅ Ventajas: estable, rápido, sin depender de Cognito, sin MFA/captcha.

Opción B: login por API (ROPC) — solo si lo aceptas

Cognito tiene un flujo **USER_PASSWORD_AUTH** usando el endpoint de tokens (InitiateAuth), que en la práctica es “username+password y te devuelve tokens”.

⚠️ Pero ojo:

- No siempre está habilitado (hay que permitir ese flow en el App Client).
- No es recomendable para aplicaciones públicas.
- Para E2E puede valer si solo es entorno de test.

Luego con esos tokens tendrías que:

- o simular sesión en tu app (depende de cómo Spring mantenga la sesión),
- o usar esos tokens directamente si tu app acepta JWT en vez de sesión.

Opción C: test “real” del OAuth pero sin navegador

Eso sería más “integration test” que E2E:

- pruebas el intercambio code -> token y endpoints protegidos
- sin UI

Qué haría yo en tu caso (para que avances ya)

Como tú quieres probar **reservar ejemplar**, mi consejo práctico:

 **Haz E2E contra tu app, no contra Cognito UI.**
Implementa un modo e2e de autenticación en Spring Boot (solo para local/CI).

Ejemplo de idea (sin mucho lío)

- Perfil e2e
- Ruta /e2e-login?user=prueba@prueba.com
- Entra ahí → Spring crea un Authentication y te deja logueado
- Selenium ya empieza en http://localhost:8080/e2e-login y listo

Antes de decidir: confirma si Cognito está mostrando un error

En tu test, justo después de intentar login, imprime cualquier alert:

```
driver.findElements(By.cssSelector("[role='alert'], .text-error, .error, .alert"))  
.forEach(e -> System.out.println("ALERT: " + e.getText()));
```

Si sale algo como “Incorrect username or password” o “User does not exist”, ya sabemos que no es “bloqueo”, sino login fallido.

Si me dices **cómo tu app guarda la autenticación** después del login:

- ¿usa **sesión** (JSESSIONID) con OAuth2 client?
- ¿o usa **JWT** en el frontend?

te digo la opción más fácil entre:

1. bypass en Spring (la más simple y estable)
2. login por API (ROPC) + inyectar sesión/tokens para Selenium

he iniciado sesion en mi app con el mismo usuario y no me ha dado ningun problema

Vale: si tú puedes iniciar sesión manualmente con ese usuario, entonces **no es credenciales**. En automatización suele ser una de estas dos:

1. **Cognito detecta “automation” / hay protección** (WAF/anti-bot, o el propio navegador automatizado) y el submit no llega a completarse.
2. **El submit no se está ejecutando igual que cuando lo haces tú** (overlay, foco, validación JS, click en el botón equivocado, etc.).

Como estás en verifyPassword incluso después de “click”, lo más útil ahora es **forzar el submit de forma equivalente a un usuario** y, si falla, **capturar el error que Cognito muestra**.

1) Haz submit del formulario (no solo click)

En Cognito Hosted UI suele haber un <form>. En vez de click, prueba a enviar el formulario:

Sustituye el bloque del password por esto:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(30));

WebElement password = wait.until(ExpectedConditions.visibilityOfElementLocated(
    By.cssSelector("input[name='password'], input[type='password']")
));
password.clear();
password.sendKeys("1234567aA!");

// Enviar el formulario "de verdad"
password.submit();
```

Luego espera el redirect:

```
wait.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
wait.until(ExpectedConditions.urlContains("localhost:8080"));
```

Esto muchas veces funciona cuando click() no.

2) Si submit() no funciona: dispara el submit con JS

Justo después de escribir la contraseña:

```
((JavascriptExecutor) driver).executeScript(
    "document.querySelector('input[type='password']").closest('form').submit();"
);
```

Y la misma espera al redirect.

3) Asegura que no hay un “overlay” tapando el botón

Antes de hacer click al submit, añade:

```
WebElement submitBtn = driver.findElement(By.cssSelector("button, input[type='submit']"));
```

```
((JavascriptExecutor) driver).executeScript("arguments[0].scrollIntoView({block:'center'});", submitBtn);
Thread.sleep(300);
((JavascriptExecutor) driver).executeScript("arguments[0].click();", submitBtn);
```

Sí, es tosco, pero con UIs externas reduce muchísimo “click fantasma”.

4) Captura el mensaje de error real (aunque no lo veas)

Justo después de intentar enviar, imprime cualquier alerta:

```
driver.findElements(By.cssSelector("[role='alert'], .error, .text-error, .alert, .banner"))
.forEach(e -> {
    String t = e.getText().trim();
    if (!t.isEmpty()) System.out.println("ALERT: " + t);
});
```

Si hay rate-limit, challenge, o “suspicious activity”, saldrá aquí.

5) Si es bloqueo anti-bot: la forma práctica de E2E

Aunque se pueda “pelear” con Cognito, en la práctica lo mejor para E2E es:

Opción recomendada: bypass de login solo en perfil e2e

Tu E2E debería probar **reservas**, no el IdP.

En Spring Boot creas un perfil e2e que habilita un endpoint como:

- /e2e-login → autentica un usuario de test y redirige a /

Selenium empieza por ahí y se olvida de Cognito.

Si quieres, te paso el SecurityFilterChain exacto para Spring Security (con OAuth2 client) para que funcione en 10–15 líneas.

Qué haría yo ahora mismo (orden)

1. Cambiar submit1.click() por password.submit() (es lo más “humano”).
2. Si no, JS submit del form.
3. Si sigue igual, asumimos bloqueo/flow extra y pasamos a bypass e2e en Spring.

Si pegas el HTML del formulario del password (la línea del <form ...> o <button ...> de esa pantalla), te digo cuál de las 3 variantes es la más segura para tu DOM.

con que perfil se ejecutan los test?

Buena pregunta 👍 — y además **muy importante** para lo que estás intentando hacer.

Respuesta corta

👉 Los tests **NO** se ejecutan con ningún perfil de Spring por defecto. Si no dices nada, se ejecutan con el perfil “default”.

Eso explica muchas cosas de lo que te está pasando.

Qué significa eso en la práctica

- Cuando ejecutas:

`./mvnw test`

o desde IntelliJ → Run Test

👉 Spring **NO** usa `application-e2e.yml`, ni `application-test.yml`

👉 Usa `application.yml` / `application.properties` (perfil por defecto)

Y si tu test **ni siquiera levanta Spring** (como tu test Selenium):

- Spring **no está involucrado**
 - el perfil depende solo de cómo arranques la app **en otra terminal**
-

En tu caso concreto (muy importante)

Tu test Selenium:

- **NO levanta Spring Boot**
- Solo abre un navegador y accede a URLs

Por tanto:

- El **perfil del backend** es el que tengas activo cuando arrancas Spring Boot
- El **test JUnit no tiene perfil** salvo que tú lo fuerces

Ejemplo típico:

`./mvnw spring-boot:run`

👉 perfil **default**

Si quieres e2e:

`./mvnw spring-boot:run -Dspring-boot.run.profiles=e2e`

Cómo forzar un perfil en tests JUnit (cuando SÍ usas Spring)

Esto es para tests con `@SpringBootTest` (no Selenium puro):

```
        @ActiveProfiles("e2e")
@SpringBootTest
class                               MiTest                               {
}
```

O con propiedad:

```
@SpringBootTest(properties = "spring.profiles.active=e2e")
```

⚠ **Esto NO afecta** a tu test Selenium actual, porque:

- no tiene @SpringBootTest
 - no arranca Spring dentro del test
-

Entonces, ¿qué perfil estás usando ahora mismo?

Muy probablemente:

- Tu app está arrancada con **perfil default**
 - Tu test Selenium intenta pasar por **Cognito real**
 - No hay bypass / modo e2e
 - Cognito se comporta distinto con automatización → te bloquea o mete challenge
-

Qué te recomiendo (para que esto deje de doler)

Opción correcta para E2E

1. Arranca tu app con perfil e2e
2. En ese perfil:
 - desactiva OAuth real
 - o habilita un **login automático de test**
3. Selenium entra **directamente en tu app**, no en Cognito

Ejemplo:

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=e2e
```

Y en Selenium:

```
driver.get("http://localhost:8080/e2e-login");
```

Si quieres, siguiente paso

Si me dices:

- ¿usas **Spring Security con OAuth2 client** (sí)?
- ¿quieres un **login automático SOLO para e2e**?

Te paso **el código exacto**:

- SecurityFilterChain para profile=e2e
- endpoint /e2e-login
- y cómo arrancar todo para que Selenium funcione a la primera

Esto es exactamente lo que hacen equipos cuando el IdP rompe E2E 🤯

```
URL          tras          submit          password:
http://localhost:8080/login/oauth2/code/cognito?code=5937ae57-b5c8-40df-a7f9-
96b0e8fb8c59&state=k9LV339_KKImRvC11kePad79i-fEj8rqS0lneHuu-JQ%3D TITLE tras submit
password: Problema al cargar la página org.openqa.selenium.NoSuchElementException: Unable
to locate element: .two > .card:nth-child(1) > .p For documentation on this error, please visit:
https://www.selenium.dev/documentation/webdriver/troubleshooting/errors#nosuchelement
exception Build info: version: '4.37.0', revision: '3c1da1e342' System info: os.name: 'Windows
10', os.arch: 'amd64', os.version: '10.0', java.version: '25.0.1' Driver info:
org.openqa.selenium.firefox.FirefoxDriver
```

Buen avance: **ya saliste de Cognito** y volviste a tu app con el code:

`http://localhost:8080/login/oauth2/code/cognito?...`

Pero Firefox te muestra **“Problema al cargar la página”** ⇒ esa URL de callback **no está cargando** (tu backend no responde, se corta la conexión, o hay un error SSL/redirect). Por eso luego tu test intenta buscar `.two > ...` y falla: **nunca llegó a tu home**.

Lo que tienes que hacer ahora

1. **No sigas con el test** hasta que completes el callback.
2. Diagnostica por qué Firefox dice **“Problema al cargar la página”**.

1) Asegúrate de que tu Spring Boot está levantado y accesible

Antes de ejecutar el test:

- tu app debe estar corriendo en localhost:8080
- y ese endpoint `/login/oauth2/code/cognito` lo maneja Spring Security durante el login

Comprueba en el momento del fallo:

- ¿la app sigue viva o se paró?
- mira la consola donde corre Spring Boot: **¿hay stacktrace justo cuando llega el callback?**

Ese stacktrace suele decir exactamente qué pasa (mismatch de state, invalid redirect, error token exchange, etc.).

2) En el test: espera a que llegue a una URL “final” (no al callback)

Ahora mismo estás imprimiendo la URL justo tras submit y luego sigues.

Tienes que hacer algo así:

```
// después de enviar password y hacer submit
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(30));

// Espera a entrar al callback
wait.until(ExpectedConditions.urlContains("/login/oauth2/code/cognito"));

// Y ahora espera a que Spring redirija a tu home (o dashboard)
wait.until(ExpectedConditions.not(ExpectedConditions.urlContains("/login/oauth2/code/cognito")));
wait.until(ExpectedConditions.urlContains("localhost:8080"));
```

Luego ya buscas elementos de tu app.

3) Tu error “Problema al cargar la página” suele ser una de estas

A) Tu backend no está corriendo / se cae

- Firefox muestra “problema al cargar” cuando no hay respuesta.
- En la consola de Spring Boot verás que se ha parado o una excepción fatal.

B) Spring Security falla en el callback (y Firefox muestra error genérico)

Posibles causas típicas:

- **redirect_uri** no coincide exactamente con lo configurado en Cognito
- problema intercambiando el code por tokens (client id/secret mal, PKCE, etc.)
- “state” mismatch porque el navegador no guarda la cookie de sesión (SameSite/cookies)

➡ Aquí el **log del backend** es lo más importante.

4) Arregla tu selector .two > .card:nth-child(1) > .p

Aunque arregles el login, ese selector es frágil.

Pero **ahora mismo no importa**, porque ni llegas a esa pantalla. Primero arreglamos el callback.

Lo más útil que puedes hacer ya (sin adivinar)

Pégame el error del backend

En la consola donde corre Spring Boot, en el momento en que Selenium llega a:

/login/oauth2/code/cognito?...

debería aparecer algo. Pégame esas líneas.

Con eso te digo exactamente si es:

- redirect mismatch
- fallo token exchange
- cookie/session (SameSite)
- etc.

Si quieres avanzar aunque OAuth falle: “modo e2e”

Como te dije antes: para E2E de negocio, lo normal es evitar el IdP.

Si quieres, te paso el patrón **perfil e2e**:

- /e2e-login te autentica como usuario de test
- Selenium entra ahí y ya puede reservar sin Cognito

Pero como ahora ya estás muy cerca, yo miraría primero el **log del callback**.

🔗 Pásame el stacktrace/log de Spring Boot justo cuando falla el callback y lo cerramos en un paso.

```
ahora mismo el test que tengo es este. package com.upm.library; import static
org.hamcrest.CoreMatchers.is; import static org.hamcrest.MatcherAssert.assertThat; import
java.time.Duration; import java.util.HashMap; import java.util.Map; import
org.junit.jupiter.api.AfterEach; import org.junit.jupiter.api.BeforeEach; import
org.junit.jupiter.api.Test; import org.openqa.selenium.*; import
org.openqa.selenium.firefox.FirefoxDriver; import
org.openqa.selenium.support.ui.ExpectedConditions; import
org.openqa.selenium.support.ui.WebDriverWait; import
org.springframework.boot.test.context.SpringBootTest; import
org.springframework.test.context.ActiveProfiles; @ActiveProfiles("aws") @SpringBootTest
public class ReservationTest { private WebDriver driver; private Map<String, Object> vars;
JavascriptExecutor js; @BeforeEach public void setUp() { driver = new FirefoxDriver(); js =
(JavascriptExecutor) driver; vars = new HashMap<String, Object>(); } @AfterEach public void
tearDown() { driver.quit(); } @Test public void test1() { driver.get("http://localhost:8080");
driver.manage().window().setSize(new Dimension(1936, 1056)); WebDriverWait wait = new
WebDriverWait(driver, Duration.ofSeconds(30)); WebElement email =
wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("input[name='username'],
input[type='email']"))); email.clear(); email.sendKeys("prueba@prueba.com"); // botón
"next/sign in" (puede ser button o input) WebElement submitEmail =
wait.until(ExpectedConditions.presenceOfElementLocated(
By.cssSelector("button[type='submit'], input[type='submit']")));
wait.until(ExpectedConditions.elementToBeClickable(submitEmail)).click(); WebDriverWait
```

```

wait2 = new WebDriverWait(driver, Duration.ofSeconds(30)); WebElement password =
wait.until(ExpectedConditions.visibilityOfElementLocated(
By.cssSelector("input[name='password'], input[type='password']" )); password.clear();
password.sendKeys("1234567aA!"); // Enviar el formulario "de verdad" password.submit();
wait2.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
wait2.until(ExpectedConditions.urlContains("localhost:8080")); // DEBUG
System.out.println("URL tras submit password: " + driver.getCurrentUrl());
System.out.println("TITLE tras submit password: " + driver.getTitle()); // Espera a que cambie a
tu app WebDriverWait wait1 = new WebDriverWait(driver, Duration.ofSeconds(30)); try {
wait1.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
wait1.until(ExpectedConditions.urlContains("localhost:8080")); } catch (TimeoutException e) { //
Si no sales de Cognito, imprime info útil y falla el test aquí mismo System.out.println("NO SALIÓ
DE COGNITO. URL: " + driver.getCurrentUrl()); System.out.println("TITLE: " + driver.getTitle()); //
Intenta sacar mensaje de error típico de Cognito var alerts =
driver.findElements(By.cssSelector("[role='alert'], .text-error, .error, .alert, .banner")); for (var a :
alerts) { String t = a.getText().trim(); if (!t.isEmpty()) System.out.println("ALERT: " + t); } // Detecta
MFA / código boolean hayCode =
!driver.findElements(By.cssSelector("input[autocomplete='one-time-code'],
input[name*='code'], input[id*='code']")).isEmpty(); System.out.println("¿Pide código/MFA?: "
+ hayCode); throw e; } // espera a que estés en tu app (URL localhost)
driver.findElement(By.cssSelector(".two > .card:nth-child(1) > .p")).click();
driver.findElement(By.name("q")).click(); driver.findElement(By.name("q")).sendKeys("a");
driver.findElement(By.cssSelector(".primary")).click(); driver.findElement(By.linkText("Ver
ejemplares →")).click(); driver.findElement(By.linkText("Reservar ejemplar")).click();
driver.findElement(By.cssSelector(".primary")).click();
driver.findElement(By.cssSelector(".danger")).click();
assertThat(driver.switchTo().alert().getText(), is("¿Cancelar esta reserva?"));
driver.switchTo().alert().accept(); driver.findElement(By.cssSelector(".ghost")).click(); } }
org.openqa.selenium.WebDriverException: Reached error page:
about:neterror?e=connectionFailure&u=http%3A//localhost%3A8080/&c=UTF-
8&d=Firefox%20no%20puede%20establecer%20una%20conexi%C3%B3n%20con%20el%20ser
vidor%20en%20localhost%3A8080.&a= Build info: version: '4.37.0', revision: '3c1da1e342'
System info: os.name: 'Windows 10', os.arch: 'amd64', os.version: '10.0', java.version: '25.0.1'
Driver info: org.openqa.selenium.firefox.FirefoxDriver Command: [7053d906-8ba1-4c28-b463-
bd39eaefa5e6, get {url=http://localhost:8080}] Capabilities {acceptInsecureCerts: true,
browserName: firefox, browserVersion: 147.0.4, moz:accessibilityChecks: false, moz:buildID:
20260212191108, moz:geckodriverVersion: 0.36.0, moz:headless: false, moz:platformVersion:
10.0, moz:processID: 17472, moz:profile: C:\Users\Usuario\AppData\Lo...,
moz:shutdownTimeout: 60000, moz:webdriverClick: true, moz:windowless: false,
pageLoadStrategy: normal, platformName: windows, proxy: Proxy(), setWindowRect: true,
strictFileInteractability: false, timeouts: {implicit: 0, pageLoad: 300000, script: 30000},
unhandledPromptBehavior: dismiss and notify, userAgent: Mozilla/5.0 (Windows NT 10....}
Session ID: 7053d906-8ba1-4c28-b463-bd39eaefa5e6 at
java.base/jdk.internal.reflect.DirectConstructorHandleAccessor.newInstance(DirectConstructor
HandleAccessor.java:62) at
java.base/java.lang.reflect.Constructor.newInstanceWithCaller(Constructor.java:499) at
java.base/java.lang.reflect.Constructor.newInstance(Constructor.java:483) at
org.openqa.selenium.remote.ErrorCodec.decode(ErrorCodec.java:167) at
org.openqa.selenium.remote.codec.w3c.W3CHttpResponseCodec.decode(W3CHttpResponseC

```

```
odec.java:138)                                     at
org.openqa.selenium.remote.codec.w3c.W3CHttpResponseCodec.decode(W3CHttpResponseC
odec.java:50)                                     at
org.openqa.selenium.remote.HttpCommandExecutor.execute(HttpCommandExecutor.java:215)
at
org.openqa.selenium.remote.service.DriverCommandExecutor.invokeExecute(DriverCommand
Executor.java:
```

Ese error ya no es Cognito: es **Firefox no puede conectar con** http://localhost:8080.

about:nerror?e=connectionFailure ... Firefox no puede establecer una conexión con el servidor en localhost:8080

Eso significa **simplemente**: cuando arranca el test, **tu Spring Boot NO está escuchando en 8080** (o se cae al arrancar, o está en otro puerto, o no se levanta a tiempo).

Además, en tu caso has puesto `@SpringBootTest`, así que **Spring debería arrancar dentro del test**, pero hay dos detalles clave:

1. `@SpringBootTest` por defecto arranca en **puerto aleatorio** (no 8080) si usas `webEnvironment=RANDOM_PORT`.
2. Si no indicas `webEnvironment`, a veces arranca en modo "mock" (sin servidor real) y **no abre ningún puerto**.

Arreglo: arranca Spring Boot en un puerto real y úsalo en Selenium

Opción A (la más directa): arrancar con puerto definido `DEFINED_PORT`

Cambia tu anotación a:

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.DefinedPort)
@ActiveProfiles("aws")
public class ReservationTest {
```

Y asegúrate de que en tu config (perfil aws o default) tengas:

```
server.port=8080
```

o en `application-aws.properties/yml`.

✅ Con esto, cuando el test empiece, **Spring Boot se levanta en 8080** y Firefox ya puede entrar.

Importante: espera a que el servidor esté listo antes de abrir el navegador

Aunque `@SpringBootTest` suele esperar, con Selenium a veces conviene un "ping" simple.

Puedes meter al principio del test:

```
new WebDriverWait(driver, Duration.ofSeconds(30))
.until(d -> ((JavascriptExecutor)d).executeScript("return
document.readyState").equals("complete"));
```

Pero lo más fiable es que Spring esté de verdad arriba (con DEFINED_PORT suele bastar).

Opción B (más limpia): puerto aleatorio + inyectar el puerto al test

Esto evita conflictos si el 8080 está ocupado.

```
@SpringBootTest(webEnvironment =  
SpringBootTest.WebEnvironment.RANDOM_PORT)  
@ActiveProfiles("aws")  
class ReservationTest {  
  
    @LocalServerPort  
    int port;  
  
    @Test  
    void test1() {  
        driver.get("http://localhost:" + port);  
        ...  
    }  
}
```

Necesitas importar:

```
import org.springframework.boot.test.web.server.LocalServerPort;
```

✅ Esta es la opción que más recomiendo a largo plazo.

Cosas que debes revisar (muy típicas)

- ¿Tienes otra app usando 8080? (Postman, otro Spring, Docker...)
 - ¿En el perfil aws estás cambiando server.port a otro valor?
 - ¿El contexto Spring falla al arrancar y por eso no llega a abrir puerto? (mira la consola del test: si hay stacktrace antes, ese es el motivo real)
-

Un detalle: tu @ActiveProfiles("aws") solo afecta a Spring, no a Selenium

Está bien, pero asegúrate de que el perfil aws **no requiere servicios externos** para arrancar (DB, AWS, etc.), porque si falla el arranque, nunca habrá servidor.

Qué te recomiendo hacer ya

1. Cambia a **RANDOM_PORT** + @LocalServerPort (te evitas el 8080 ocupado).
2. Abre con driver.get("http://localhost:" + port);

Si me pegas el encabezado del log del test (las primeras líneas cuando arranca Spring) te digo si está arrancando bien o si se está cayendo por configuración del perfil aws.

con ese cambio ha ejecutado bien

Genial — entonces el problema era exactamente ese: **tu app no estaba escuchando en localhost:8080 durante el test**, y al usar RANDOM_PORT + @LocalServerPort ya apuntas al puerto real y Selenium puede entrar.

Para dejarlo “redondo” y evitar que vuelva a romperse, te recomiendo estos 3 ajustes rápidos:

1) Deja el test con RANDOM_PORT + @LocalServerPort

Ejemplo mínimo:

```
@SpringBootTest(webEnvironment =  
SpringBootTest.WebEnvironment.RANDOM_PORT)  
@ActiveProfiles("aws")  
class ReservationTest {  
  
    @LocalServerPort  
    int port;  
  
    @BeforeEach  
    void setUp() { driver = new FirefoxDriver(); }  
  
    @Test  
    void test1() {  
        driver.get("http://localhost:" + port);  
        ...  
    }  
}
```

Import:

```
import org.springframework.boot.test.web.server.LocalServerPort;
```

2) Quita selectores frágiles (nth-child)

Tu `.two > .card:nth-child(1) > .p` se romperá fácil. Si puedes, pon en tu HTML:

```
<div data-testid="home-card-1">...</div>
```

Y en Selenium:

```
wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("[data-  
testid='home-card-1']"))).click();
```

3) Evita Cognito UI en E2E (si algún día te vuelve a fallar)

Si lo tienes estable ahora, perfecto. Pero si en CI te mete MFA/captcha o se vuelve flaky, lo típico es añadir un **perfil e2e** con login “mock” solo para pruebas.

Si quieres, pégame la parte donde ahora construyes la URL con el puerto y te digo cómo dejar el test limpio (1 wait, sin waits duplicados, y con esperas correctas tras el OAuth).

```
package com.upm.library; import static org.hamcrest.CoreMatchers.is; import static
org.hamcrest.MatcherAssert.assertThat; import java.time.Duration; import java.util.HashMap;
import java.util.Map; import org.junit.jupiter.api.AfterEach; import
org.junit.jupiter.api.BeforeEach; import org.junit.jupiter.api.Test; import org.openqa.selenium.*;
import org.openqa.selenium.firefox.FirefoxDriver; import
org.openqa.selenium.support.ui.ExpectedConditions; import
org.openqa.selenium.support.ui.WebDriverWait; import
org.springframework.boot.test.context.SpringBootTest; import
org.springframework.test.context.ActiveProfiles; @ActiveProfiles("aws")
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.DEFINED_PORT) public
class ReservationTest { private WebDriver driver; private Map<String, Object> vars;
JavascriptExecutor js; @BeforeEach public void setUp() { driver = new FirefoxDriver(); js =
(JavascriptExecutor) driver; vars = new HashMap<String, Object>(); } @AfterEach public void
tearDown() { driver.quit(); } @Test public void test1() { driver.get("http://localhost:8080");
driver.manage().window().setSize(new Dimension(1936, 1056)); WebDriverWait wait = new
WebDriverWait(driver, Duration.ofSeconds(30)); WebElement email =
wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("input[name='username'],
input[type='email']"))); email.clear(); email.sendKeys("prueba@prueba.com"); // botón
"next/sign in" (puede ser button o input) WebElement submitEmail =
wait.until(ExpectedConditions.presenceOfElementLocated(
By.cssSelector("button[type='submit'], input[type='submit']")));
wait.until(ExpectedConditions.elementToBeClickable(submitEmail)).click(); WebDriverWait
wait2 = new WebDriverWait(driver, Duration.ofSeconds(30)); WebElement password =
wait.until(ExpectedConditions.visibilityOfElementLocated(
By.cssSelector("input[name='password'], input[type='password']"))); password.clear();
password.sendKeys("1234567aA!"); // Enviar el formulario "de verdad" password.submit();
wait2.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
wait2.until(ExpectedConditions.urlContains("localhost:8080")); // DEBUG
System.out.println("URL tras submit password: " + driver.getCurrentUrl());
System.out.println("TITLE tras submit password: " + driver.getTitle()); // Espera a que cambie a
tu app WebDriverWait wait1 = new WebDriverWait(driver, Duration.ofSeconds(30)); try {
wait1.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
wait1.until(ExpectedConditions.urlContains("localhost:8080")); } catch (TimeoutException e) { //
Si no sales de Cognito, imprime info útil y falla el test aquí mismo System.out.println("NO SALIÓ
DE COGNITO. URL: " + driver.getCurrentUrl()); System.out.println("TITLE: " + driver.getTitle()); //
Intenta sacar mensaje de error típico de Cognito var alerts =
driver.findElements(By.cssSelector("[role='alert'], .text-error, .error, .alert, .banner")); for (var a :
alerts) { String t = a.getText().trim(); if (!t.isEmpty()) System.out.println("ALERT: " + t); } // Detecta
MFA / código boolean hayCode =
!driver.findElements(By.cssSelector("input[autocomplete='one-time-code'],
input[name*='code'], input[id*='code']")).isEmpty(); System.out.println("¿Pide código/MFA?: "
+ hayCode); throw e; } // espera a que estés en tu app (URL localhost)
driver.findElement(By.cssSelector(".two > .card:nth-child(1) > .p")).click();
```

```

driver.findElement(By.name("q")).click();      driver.findElement(By.name("q")).sendKeys("a");
driver.findElement(By.cssSelector(".primary")).click();      driver.findElement(By.linkText("Ver
ejemplares →")).click();      driver.findElement(By.linkText("Reservar ejemplar")).click();
driver.findElement(By.cssSelector(".primary")).click();
driver.findElement(By.cssSelector(".danger")).click();
assertThat(driver.switchTo().alert().getText(),      is("¿Cancelar esta reserva?"));
driver.switchTo().alert().accept(); driver.findElement(By.cssSelector(".ghost")).click(); } } este es
el test, optimízalo

```

Aquí tienes una versión **más limpia, estable y corta** del mismo test. He hecho estas mejoras:

- **Un solo WebDriverWait** (quitando wait1/wait2 duplicados).
- **Constantes** para URL/timeout/credenciales.
- **Métodos auxiliares** (waitClickable, waitVisible, click, type).
- **Login Cognito** más robusto (email → submit → password → submit()).
- **No continúa** si no ha salido de Cognito (falla con info útil).
- Evita algunos findElement directos (mejor wait antes de click).

Nota: mantengo tu DEFINED_PORT + localhost:8080 porque así lo estás usando. Si quieres que sea más robusto (sin conflicto de puertos), te lo paso también con RANDOM_PORT.

Test optimizado (cópialo tal cual)

```

package                                com.upm.library;

import                                org.hamcrest.CoreMatchers.is;
import                                static org.hamcrest.MatcherAssert.assertThat;

import                                java.time.Duration;

import                                org.junit.jupiter.api.*;
import                                org.openqa.selenium.*;
import                                org.openqa.selenium.firefox.FirefoxDriver;
import                                org.openqa.selenium.support.ui.ExpectedConditions;
import                                org.openqa.selenium.support.ui.WebDriverWait;
import                                org.springframework.boot.test.context.SpringBootTest;
import                                org.springframework.test.context.ActiveProfiles;

@ActiveProfiles("aws")
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.DEFINED_PORT)
public class ReservationTest {

    private static final String BASE_URL = "http://localhost:8080";
    private static final Duration TIMEOUT = Duration.ofSeconds(30);

```



```

// Idealmente, no hardcodees credenciales: pásalas por env vars en CI
private static final String USER = "prueba@prueba.com";
private static final String PASS = "1234567aA!";

private WebDriver driver;
private WebDriverWait wait;

@BeforeEach
void setUp() {
    driver = new FirefoxDriver();
    wait = new WebDriverWait(driver, TIMEOUT);
    driver.manage().window().setSize(new Dimension(1936, 1056));
}

@AfterEach
void tearDown() {
    if (driver != null) driver.quit();
}

@Test
void reservarYCancelar() {

    // 1) Login Cognito
    loginCognito(USER, PASS);

    // 2) Ya estás en tu app (aquí conviene esperar a algo "de tu app")
    // Si 'q' existe en tu home, es un buen ancla:
    waitClickable(By.name("q"));

    // Si este click es obligatorio, mantenlo; si no lo es, bórralo.
    click(By.cssSelector(".two > .card:nth-child(1) > .p"));

    // 3) Flujo: buscar -> ver ejemplares -> reservar -> cancelar
    click(By.name("q"));
    type(By.name("q"), "a");

    click(By.cssSelector(".primary"));
    click(By.linkText("Ver ejemplares →"));
    click(By.linkText("Reservar ejemplar"));
    click(By.cssSelector(".primary"));

    click(By.cssSelector(".danger"));

    Alert alert = wait.until(ExpectedConditions.alertIsPresent());
    assertThat(alert.getText(), is("¿Cancelar esta reserva?"));
    alert.accept();

    click(By.cssSelector(".ghost"));

```

```

}

private void loginCognito(String username, String password) {
    // Espera a que aparezca el campo username/email
    WebElement email = waitVisible(By.cssSelector("input[name='username'],
input[type='email']"));
    email.clear();
    email.sendKeys(username);

    // Botón siguiente/sign in (varía entre button e input)
    click(By.cssSelector("button[type='submit'],
input[type='submit']"));

    WebElement pwd = waitVisible(By.cssSelector("input[name='password'],
input[type='password']"));
    pwd.clear();
    pwd.sendKeys(password);

    // Submit "real" (suele funcionar mejor que click en Cognito)
    pwd.submit();

    // Espera a salir del dominio de Cognito y volver a tu app
    try

wait.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
    wait.until(ExpectedConditions.urlContains("localhost:8080"));
    // Opcional: espera a que ya no estés en el callback

wait.until(ExpectedConditions.not(ExpectedConditions.urlContains("/login/oauth2/code/")));
} catch (TimeoutException e) {
    // Debug útil si Cognito mete challenge/MFA/error
    System.out.println("NO SALIÓ DE COGNITO / NO VOLVIÓ A LA APP");
    System.out.println("URL: " + driver.getCurrentUrl());
    System.out.println("TITLE: " + driver.getTitle());

    driver.findElements(By.cssSelector("[role='alert'], .text-error, .error, .alert, .banner"))
        .stream()
        .map(WebElement::getText)
        .map(String::trim)
        .filter(t -> !t.isEmpty())
        .forEach(t -> System.out.println("ALERT: " + t));

    boolean hayCode = !driver.findElements(
        By.cssSelector("input[autocomplete='one-time-code'],
input[id*='code']")
    ).isEmpty();
    System.out.println("¿Pide código/MFA?: " + hayCode);

    throw e;
}

```

```

    }

    private WebElement waitVisible(By locator) {
        return wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
    }

    private WebElement waitClickable(By locator) {
        return wait.until(ExpectedConditions.elementToBeClickable(locator));
    }

    private void click(By locator) {
        waitClickable(locator).click();
    }

    private void type(By locator, String text) {
        WebElement el = waitVisible(locator);
        el.clear();
        el.sendKeys(text);
    }
}

```

Dos mejoras opcionales (muy recomendadas)

1. **Evitar** DEFINED_PORT: usa RANDOM_PORT + @LocalServerPort para que no falle si el 8080 está ocupado.
2. Cambiar .two > .card:nth-child(1) > .p por un selector estable (data-testid). Ese selector se romperá con cualquier cambio de maquetación.

Si me dices si puedes tocar el HTML para meter data-testid, te digo exactamente qué atributo añadir y cómo quedaría el selector en Selenium.

al exportar un nuevo test me sale este error en selenium ide Test 'http://localhost:8080' has a problem: Incomplete command 'open'. Missing expected target argument.

Ese error de **Selenium IDE** es sencillo y muy típico 
No es de tu app ni de Spring, es **del test grabado**.

Qué significa exactamente el error

Incomplete command 'open'. Missing expected target argument

En Selenium IDE, el comando open **siempre necesita una URL como target**.

Ejemplo válido:

Command: open
Target: http://localhost:8080

Ejemplo inválido (lo que tienes ahora):

Command: open
Target: ← vacío

Por eso Selenium IDE se queja.

Por qué pasa al grabar

Suele ocurrir cuando:

- Empiezas a grabar **con la página ya abierta**
- O borras/modificas la primera línea del test
- O Selenium IDE falla al capturar la URL inicial

Entonces crea el comando open pero **sin URL**.

Cómo arreglarlo (2 formas)

Opción 1 (la más rápida): rellenar el target a mano

1. Abre el test en Selenium IDE
2. Selecciona la línea:
open
3. En la columna **Target**, escribe:
http://localhost:8080
4. Guarda el test
5. Ejecuta otra vez

 Solucionado.

Opción 2 (mejor práctica): borrar y volver a grabar bien

1. Borra la línea open que está mal
2. Pulsa **Record**
3. **Empieza desde una página en blanco**
4. Escribe manualmente en el navegador:
http://localhost:8080
5. Selenium IDE grabará correctamente:

open | http://localhost:8080

Consejo importante para tu proyecto (muy útil)

Como tu backend usa **puerto dinámico en tests (RANDOM_PORT)**, **NO confíes ciegamente en el open grabado**.

Cuando exportes a JUnit:

- cambia siempre el open por una variable o constante en Java:

```
driver.get(BASE_URL);
```

y define:

```
String BASE_URL = "http://localhost:8080";
```

(o con @LocalServerPort si lo usas)

Resumen rápido

- ❌ Error NO es de Spring ni Selenium WebDriver
- ✅ Es un open sin URL en Selenium IDE
- ✂ Solución: pon http://localhost:8080 en el **Target**
- 💡 Siempre revisa la primera línea del test grabado

Si quieres, dime:

- qué versión de Selenium IDE usas
- si grabas en Chrome o Firefox

y te digo la **configuración más estable** para grabar tests que luego exportes a JUnit sin sorpresas.

```
He exportado este test: package com.upm.library; import org.junit.Test; import org.junit.Before; import org.junit.After; import static org.junit.Assert.*; import static org.hamcrest.CoreMatchers.is; import static org.hamcrest.core.IsNot.not; import org.openqa.selenium.By; import org.openqa.selenium.WebDriver; import org.openqa.selenium.firefox.FirefoxDriver; import org.openqa.selenium.chrome.ChromeDriver; import org.openqa.selenium.remote.RemoteWebDriver; import org.openqa.selenium.remote.DesiredCapabilities; import org.openqa.selenium.Dimension; import org.openqa.selenium.WebElement; import org.openqa.selenium.interactions.Actions; import org.openqa.selenium.support.ui.ExpectedConditions; import org.openqa.selenium.support.ui.WebDriverWait; import org.openqa.selenium.JavascriptExecutor; import org.openqa.selenium.Alert; import org.openqa.selenium.Keys; import java.util.*; import java.net.MalformedURLException; import java.net.URL; public class LoanTest { private WebDriver driver; private Map<String, Object> vars;
```

```

JavascriptExecutor js; @Before public void setUp() { driver = new FirefoxDriver(); js =
(JavascriptExecutor) driver; vars = new HashMap<String, Object>(); } @After public void
tearDown() { driver.quit(); } @Test public void loanTest() { driver.get("http://localhost:8080");
driver.findElement(By.id("formField:R6dpf55:")).click();
driver.findElement(By.id("formField:R6dpf55:")).sendKeys("prueba@prueba.com");
driver.findElement(By.cssSelector(".awsui_button_vjswe_14izv_157")).click(); { WebElement
element = driver.findElement(By.cssSelector(".awsui_button_vjswe_14izv_157")); Actions
builder = new Actions(driver); builder.moveToElement(element).perform(); }
driver.findElement(By.id("formField:r1:")).sendKeys("1234567aA!");
driver.findElement(By.cssSelector(".awsui_variant-primary_vjswe_14izv_230")).click(); {
WebElement element = driver.findElement(By.cssSelector(".awsui_variant-
primary_vjswe_14izv_230")); Actions builder = new Actions(driver);
builder.moveToElement(element).perform(); } { WebElement element =
driver.findElement(By.tagName("body")); Actions builder = new Actions(driver);
builder.moveToElement(element, 0, 0).perform(); } driver.findElement(By.cssSelector(".three >
.card:nth-child(1) > .p")).click(); driver.findElement(By.name("userQ")).click();
driver.findElement(By.name("userQ")).sendKeys("a");
driver.findElement(By.cssSelector(".primary")).click();
driver.findElement(By.cssSelector(".item:nth-child(2) > .right span")).click();
driver.findElement(By.name("copyCode")).click();
driver.findElement(By.name("copyCode")).sendKeys("1");
driver.findElement(By.cssSelector(".btn:nth-child(5)")).click();
driver.findElement(By.cssSelector(".btn:nth-child(6)")).click();
driver.findElement(By.linkText("BIBLIOTECA UNIVERSIDAD POLITÉCNICA DE MADRID")).click();
driver.findElement(By.cssSelector(".two > .card:nth-child(2) > .p")).click();
driver.findElement(By.cssSelector(".ghost")).click(); } } Esto no funciona, quiero que lo modifiques
teniendo en cuenta el siguiente test que si que funciona. package com.upm.library; import static
org.hamcrest.CoreMatchers.is; import static org.hamcrest.MatcherAssert.assertThat; import
java.time.Duration; import java.util.HashMap; import java.util.Map; import
org.junit.jupiter.api.AfterEach; import org.junit.jupiter.api.BeforeEach; import
org.junit.jupiter.api.Test; import org.openqa.selenium.*; import
org.openqa.selenium.firefox.FirefoxDriver; import
org.openqa.selenium.support.ui.ExpectedConditions; import
org.openqa.selenium.support.ui.WebDriverWait; import
org.springframework.boot.test.context.SpringBootTest; import
org.springframework.test.context.ActiveProfiles; @ActiveProfiles("aws")
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.DEFINED_PORT) public
class ReservationTest { private WebDriver driver; private Map<String, Object> vars;
JavascriptExecutor js; @BeforeEach public void setUp() { driver = new FirefoxDriver(); js =
(JavascriptExecutor) driver; vars = new HashMap<String, Object>(); } @AfterEach public void
tearDown() { driver.quit(); } @Test public void test1() { driver.get("http://localhost:8080");
driver.manage().window().setSize(new Dimension(1936, 1056)); WebDriverWait wait = new
WebDriverWait(driver, Duration.ofSeconds(30)); WebElement email =
wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("input[name='username'],
input[type='email']"))); email.clear(); email.sendKeys("prueba@prueba.com"); // botón
"next/sign in" (puede ser button o input) WebElement submitEmail =
wait.until(ExpectedConditions.presenceOfElementLocated(
By.cssSelector("button[type='submit'], input[type='submit']")));
wait.until(ExpectedConditions.elementToBeClickable(submitEmail)).click(); WebDriverWait

```

```

wait2 = new WebDriverWait(driver, Duration.ofSeconds(30)); WebElement password =
wait.until(ExpectedConditions.visibilityOfElementLocated(
By.cssSelector("input[name='password'], input[type='password']" )); password.clear();
password.sendKeys("1234567aA!"); // Enviar el formulario "de verdad" password.submit();
wait2.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
wait2.until(ExpectedConditions.urlContains("localhost:8080")); // DEBUG
System.out.println("URL tras submit password: " + driver.getCurrentUrl());
System.out.println("TITLE tras submit password: " + driver.getTitle()); // Espera a que cambie a
tu app WebDriverWait wait1 = new WebDriverWait(driver, Duration.ofSeconds(30)); try {
wait1.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
wait1.until(ExpectedConditions.urlContains("localhost:8080")); } catch (TimeoutException e) { //
Si no sales de Cognito, imprime info útil y falla el test aquí mismo System.out.println("NO SALIÓ
DE COGNITO. URL: " + driver.getCurrentUrl()); System.out.println("TITLE: " + driver.getTitle()); //
Intenta sacar mensaje de error típico de Cognito var alerts =
driver.findElements(By.cssSelector("[role='alert'], .text-error, .error, .alert, .banner")); for (var a :
alerts) { String t = a.getText().trim(); if (!t.isEmpty()) System.out.println("ALERT: " + t); } // Detecta
MFA / código boolean hayCode =
!driver.findElements(By.cssSelector("input[autocomplete='one-time-code'],
input[name*='code'], input[id*='code']")).isEmpty(); System.out.println("¿Pide código/MFA?: "
+ hayCode); throw e; } // espera a que estés en tu app (URL localhost)
driver.findElement(By.cssSelector(".two > .card:nth-child(1) > .p")).click();
driver.findElement(By.name("q")).click(); driver.findElement(By.name("q")).sendKeys("a");
driver.findElement(By.cssSelector(".primary")).click(); driver.findElement(By.linkText("Ver
ejemplares →")).click(); driver.findElement(By.linkText("Reservar ejemplar")).click();
driver.findElement(By.cssSelector(".primary")).click();
driver.findElement(By.cssSelector(".danger")).click();
assertThat(driver.switchTo().alert().getText(), is("¿Cancelar esta reserva?"));
driver.switchTo().alert().accept(); driver.findElement(By.cssSelector(".ghost")).click(); } } package
com.upm.library; import static org.hamcrest.CoreMatchers.is; import static
org.hamcrest.MatcherAssert.assertThat; import java.time.Duration; import java.util.HashMap;
import java.util.Map; import org.junit.jupiter.api.AfterEach; import
org.junit.jupiter.api.BeforeEach; import org.junit.jupiter.api.Test; import org.openqa.selenium.*;
import org.openqa.selenium.firefox.FirefoxDriver; import
org.openqa.selenium.support.ui.ExpectedConditions; import
org.openqa.selenium.support.ui.WebDriverWait; import
org.springframework.boot.test.context.SpringBootTest; import
org.springframework.test.context.ActiveProfiles; @ActiveProfiles("aws")
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.DEFINED_PORT) public
class ReservationTest { private WebDriver driver; private Map<String, Object> vars;
JavascriptExecutor js; @BeforeEach public void setUp() { driver = new FirefoxDriver(); js =
(JavascriptExecutor) driver; vars = new HashMap<String, Object>(); } @AfterEach public void
tearDown() { driver.quit(); } @Test public void test1() { driver.get("http://localhost:8080");
driver.manage().window().setSize(new Dimension(1936, 1056)); WebDriverWait wait = new
WebDriverWait(driver, Duration.ofSeconds(30)); WebElement email =
wait.until(ExpectedConditions.elementToBeClickable(By.cssSelector("input[name='username'],
input[type='email']" )); email.clear(); email.sendKeys("prueba@prueba.com"); // botón
"next/sign in" (puede ser button o input) WebElement submitEmail =
wait.until(ExpectedConditions.presenceOfElementLocated(
By.cssSelector("button[type='submit'], input[type='submit']" ));

```

```

wait.until(ExpectedConditions.elementToBeClickable(submitEmail)).click();    WebDriverWait
wait2 = new WebDriverWait(driver, Duration.ofSeconds(30)); WebElement password =
wait.until(ExpectedConditions.visibilityOfElementLocated(
By.cssSelector("input[name='password'], input[type='password']")); password.clear();
password.sendKeys("1234567aA!"); // Enviar el formulario "de verdad" password.submit();
wait2.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
wait2.until(ExpectedConditions.urlContains("localhost:8080")); //          DEBUG
System.out.println("URL tras submit password: " + driver.getCurrentUrl());
System.out.println("TITLE tras submit password: " + driver.getTitle()); // Espera a que cambie a
tu app WebDriverWait wait1 = new WebDriverWait(driver, Duration.ofSeconds(30)); try {
wait1.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
wait1.until(ExpectedConditions.urlContains("localhost:8080")); } catch (TimeoutException e) { //
Si no sales de Cognito, imprime info útil y falla el test aquí mismo System.out.println("NO SALIÓ
DE COGNITO. URL: " + driver.getCurrentUrl()); System.out.println("TITLE: " + driver.getTitle()); //
Intenta sacar mensaje de error típico de Cognito var alerts =
driver.findElements(By.cssSelector("[role='alert'], .text-error, .error, .alert, .banner")); for (var a :
alerts) { String t = a.getText().trim(); if (!t.isEmpty()) System.out.println("ALERT: " + t); } // Detecta
MFA / código boolean hayCode =
!driver.findElements(By.cssSelector("input[autocomplete='one-time-code'],
input[name*='code'], input[id*='code']")).isEmpty(); System.out.println("¿Pide código/MFA?: "
+ hayCode); throw e; } // espera a que estés en tu app (URL localhost)
driver.findElement(By.cssSelector(".two > .card:nth-child(1) > .p")).click();
driver.findElement(By.name("q")).click(); driver.findElement(By.name("q")).sendKeys("a");
driver.findElement(By.cssSelector(".primary")).click(); driver.findElement(By.linkText("Ver
ejemplares →")).click(); driver.findElement(By.linkText("Reservar ejemplar")).click();
driver.findElement(By.cssSelector(".primary")).click();
driver.findElement(By.cssSelector(".danger")).click();
assertThat(driver.switchTo().alert().getText(), is("¿Cancelar esta reserva?"));
driver.switchTo().alert().accept(); driver.findElement(By.cssSelector(".ghost")).click(); } }

```

Claro. El LoanTest exportado “no funciona” por lo mismo que te pasó antes: trae **selectores frágiles** (formField:..., clases awsui_* que cambian) y **no espera** a que Cognito redirija correctamente antes de seguir.

Abajo te dejo un LoanTest **reescrito** siguiendo exactamente el patrón del ReservationTest que sí te funciona:

- JUnit 5 + SpringBootTest (igual que tu test bueno)
- Login Cognito robusto (username/password + submit() + waits)
- Warnings/Actions innecesarios eliminados
- WAIts antes de cada paso importante
- Mantengo tus pasos del flujo “préstamo”, pero evitando dependencias raras de CSS.

OJO: tu LoanTest usa name="userQ" y name="copyCode" (diferentes a q). He respetado eso.

LoanTest modificado (basado en tu ReservationTest)

```
package com.upm.library;

import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.CoreMatchers.is;

import java.time.Duration;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import org.openqa.selenium.*;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;

@ActiveProfiles("aws")
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.DEFINED_PORT)
public class LoanTest {

    private WebDriver driver;
    private WebDriverWait wait;

    @BeforeEach
    void setUp() {
        driver = new FirefoxDriver();
        driver.manage().window().setSize(new Dimension(1936, 1056));
        wait = new WebDriverWait(driver, Duration.ofSeconds(30));
    }

    @AfterEach
    void tearDown() {
        if (driver != null) driver.quit();
    }

    @Test
    void loanTest() {
        driver.get("http://localhost:8080");

        // 1) Login Cognito (igual que tu test que funciona)
        loginCognito("prueba@prueba.com", "1234567aA!");

        // 2) Espera a estar ya en tu app (mejor ancla: algo que exista sí o sí)
        // Si en esta pantalla existe userQ, perfecto:
        wait.until(ExpectedConditions.presenceOfElementLocated(By.tagName("body")));
    }
}
```

```

// Si este click es necesario en tu flujo, mantenlo.
// (Es frágil, pero lo dejamos de momento porque es lo que grabaste)
click(By.cssSelector(".three > .card:nth-child(1) > .p"));

// 3) Buscar usuario / item (según lo grabado)
type(By.name("userQ"), "a");
click(By.cssSelector(".primary"));

// 4) Seleccionar item (esto es frágil: nth-child)
// Ideal: cambiar por data-testid o link/texto estable
click(By.cssSelector(".item:nth-child(2) > .right span"));

// 5) Introducir código de ejemplar y prestar
type(By.name("copyCode"), "1");
click(By.cssSelector(".btn:nth-child(5)"));
click(By.cssSelector(".btn:nth-child(6)"));

// 6) Volver a home y salir/volver (según grabación)
click(By.linkText("BIBLIOTECA UNIVERSIDAD POLITÉCNICA DE MADRID"));
click(By.cssSelector(".two > .card:nth-child(2) > .p"));
click(By.cssSelector(".ghost"));
}

private void loginCognito(String username, String password) {
    WebElement email = wait.until(ExpectedConditions.elementToBeClickable(
        By.cssSelector("input[name='username']", input[type='email']")
    ));
    email.clear();
    email.sendKeys(username);

    WebElement submitEmail = wait.until(ExpectedConditions.elementToBeClickable(
        By.cssSelector("button[type='submit']", input[type='submit']")
    ));
    submitEmail.click();

    WebElement pwd = wait.until(ExpectedConditions.visibilityOfElementLocated(
        By.cssSelector("input[name='password']", input[type='password']")
    ));
    pwd.clear();
    pwd.sendKeys(password);

    // submit "real"
    pwd.submit();

    // espera redirect fuera de Cognito y dentro de localhost
    wait.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
    wait.until(ExpectedConditions.urlContains("localhost:8080"));
}

```

```

//          opcional:          espera          a          salir          del          callback
wait.until(ExpectedConditions.not(ExpectedConditions.urlContains("/login/oauth2/code/")));
}

private          void          click(By          locator)          {
    wait.until(ExpectedConditions.elementToBeClickable(locator)).click();
}

private          void          type(By          locator,          String          text)          {
    WebElement    el    =    wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
    el.clear();
    el.sendKeys(text);
}
}

```

Por qué tu export fallaba (y esto lo arregla)

- By.id("formField:R6dpf55:") y By.id("formField:r1:") **cambian** ⇒ ahora usamos name='username' / name='password'.
- .awsui_* son clases generadas (cambian con builds) ⇒ ahora usamos button[type='submit'].
- No esperabas al redirect OAuth ⇒ ahora esperamos salir de amazoncognito.com y salir de /login/oauth2/code/....

Siguiente mejora (la que más te conviene)

Los selectores como:

- .item:nth-child(2) > .right span
- .btn:nth-child(5)
- .three > .card:nth-child(1) > .p

son *frágiles*. Si puedes tocar el HTML, añade data-testid y lo dejamos “a prueba de cambios”.

Si me dices qué acción exacta es cada botón (por ejemplo “Prestar”, “Confirmar”, etc.), te cambio esos 3 selectores por otros mucho más estables (por texto o por atributo).

como puedo elegir el orden de los test?

Buena

pregunta



La respuesta corta es: **en JUnit NO deberías depender del orden de los tests**, pero sí se puede **controlar** cuando lo necesitas (por ejemplo en E2E como los tuyos).

Te explico **qué opciones tienes**, de más recomendada a menos.



Opción recomendada para E2E: un solo test que encadena el flujo

Para pruebas E2E reales (login → reservar → prestar → devolver...), **lo correcto es un único @Test**.



Ventajas:

- Evitas dependencia entre tests
- Evitas problemas de estado
- Mucho más estable en CI

Ejemplo:

```
@Test
void flujoCompleto() {
    login();
    reservar();
    prestar();
    devolver();
}
```

Esto es lo que ya estás haciendo, y está 👍.



Opción válida si quieres varios tests: orden explícito con JUnit 5

Como usas **JUnit 5**, esta es la forma correcta.

1) Añade el ordenador de tests

En la clase:

```
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
class ReservationTest {
```

2) Marca cada test con @Order

```
    @Test
    @Order(1)
    void login() { ... }

    @Test
    @Order(2)
    void reservar() { ... }

    @Test
```

```
@Order(3)
void cancelarReserva() { ... }
```

📌 Esto SOLO funciona dentro de la misma clase.

🏆 Ordenar clases de test (menos recomendable)

JUnit **no garantiza el orden entre clases**, pero Maven Surefire sí puede “forzar” algo.

En pom.xml (solo si te lo piden)

```
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>3.2.5</version>
<configuration>
  <runOrder>alphabetical</runOrder>
</configuration>
</plugin>
```

Entonces:

- LoanTest se ejecutará antes que ReservationTest (orden alfabético).

⚠️ **No recomendado** si un test depende de otro.

❌ Lo que NO debes hacer

- Confiar en el orden “natural”
 - Hacer que un test dependa del estado creado por otro test
 - Compartir sesión/navegador entre tests sin control
-

Recomendación final (en tu caso concreto)

Como estás haciendo **E2E reales con Selenium + OAuth**:

✓	✓	Un	flujo	=	un @Test
	Si	separas,	usa @Order dentro	de	la misma clase
✗	No dependas del orden entre clases				

Ejemplo ideal para ti

```
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
class E2ETest {

    @Test
    void reservarEjemplar() { ... }

    @Order(1)
```

```

@Test                                                                                                     @Order(2)
void                                prestarEjemplar()                                {                                ...                                }

@Test                                                                                                     @Order(3)
void                                devolverEjemplar()                                {                                ...                                }
}

```

Si quieres, dime:

- si necesitas **compartir login** entre tests
- o si quieres que **cada test sea independiente**

y te digo **la estructura exacta** para tu proyecto (la que menos te va a dar problemas en CI).

```

// Generated by Selenium IDE import org.junit.Test; import org.junit.Before; import
org.junit.After; import static org.junit.Assert.*; import static org.hamcrest.CoreMatchers.is;
import static org.hamcrest.CoreMatchers.isNot; import org.openqa.selenium.By; import
org.openqa.selenium.WebDriver; import org.openqa.selenium.firefox.FirefoxDriver; import
org.openqa.selenium.WebElement; import org.openqa.selenium.interactions.Actions; import
org.openqa.selenium.JavascriptExecutor; import org.openqa.selenium.Keys; import java.util.*;
public class ReturnTest { private WebDriver driver; private Map<String, Object> vars;
JavascriptExecutor js; @Before public void setUp() { driver = new FirefoxDriver(); js =
(JavascriptExecutor) driver; vars = new HashMap<String, Object>(); } @After public void
tearDown() { driver.quit(); } @Test public void httplocalhost8080() {
driver.findElement(By.id("formField:R6dpf55:")).sendKeys("prueba@prueba.com");
driver.findElement(By.cssSelector(".awsui_button_vjswe_14izv_157")).click(); { WebElement
element = driver.findElement(By.cssSelector(".awsui_button_vjswe_14izv_157")); Actions
builder = new Actions(driver); builder.moveToElement(element).perform(); }
driver.findElement(By.id("formField:r1:")).click();
driver.findElement(By.id("formField:r1:")).sendKeys("123456aA!");
driver.findElement(By.id("formField:r1:")).sendKeys(Keys.ENTER);
driver.findElement(By.id(":r2:")).click(); driver.findElement(By.id("formField:r1:")).click();
driver.findElement(By.id("formField:r1:")).sendKeys("1234567aA!");
driver.findElement(By.cssSelector(".awsui_variant-primary_vjswe_14izv_230")).click(); {
WebElement element = driver.findElement(By.cssSelector(".awsui_variant-
primary_vjswe_14izv_230")); Actions builder = new Actions(driver);
builder.moveToElement(element).perform(); } { WebElement element =
driver.findElement(By.tagName("body")); Actions builder = new Actions(driver);
builder.moveToElement(element, 0, 0).perform(); } driver.findElement(By.cssSelector(".three >
.card:nth-child(2)")).click(); driver.findElement(By.name("copyId")).click();
driver.findElement(By.name("copyId")).sendKeys("2");
driver.findElement(By.cssSelector(".primary")).click();
driver.findElement(By.cssSelector(".btn:nth-child(5)")).click();
assertThat(driver.switchTo().alert().getText(), is("¿Confirmar devolución del ejemplar?"));
driver.switchTo().alert().accept(); driver.findElement(By.linkText("← Volver")).click();

```

```

driver.findElement(By.cssSelector(".two > .card:nth-child(2) > .p")).click();
driver.findElement(By.cssSelector(".ghost")).click(); driver.close();
driver.get("http://localhost:8080"); driver.findElement(By.id("formField:R6dpf55:")).click();
driver.findElement(By.id("formField:R6dpf55:")).sendKeys("prueba@prueba.com");
driver.findElement(By.cssSelector(".awsui_button_vjswe_14izv_157")).click(); { WebElement
element = driver.findElement(By.cssSelector(".awsui_button_vjswe_14izv_157")); Actions
builder = new Actions(driver); builder.moveToElement(element).perform(); }
driver.findElement(By.id("formField:r1:")).sendKeys("1234567aA!");
driver.findElement(By.cssSelector(".awsui_variant-primary_vjswe_14izv_230")).click(); {
WebElement element = driver.findElement(By.cssSelector(".awsui_variant-
primary_vjswe_14izv_230")); Actions builder = new Actions(driver);
builder.moveToElement(element).perform(); } { WebElement element =
driver.findElement(By.tagName("body")); Actions builder = new Actions(driver);
builder.moveToElement(element, 0, 0).perform(); } driver.findElement(By.cssSelector(".three >
.card:nth-child(1) > .p")).click(); driver.findElement(By.name("userQ")).click();
driver.findElement(By.name("userQ")).sendKeys("a");
driver.findElement(By.cssSelector(".primary")).click();
driver.findElement(By.cssSelector(".item:nth-child(2) > .right span")).click();
driver.findElement(By.name("copyCode")).click();
driver.findElement(By.name("copyCode")).sendKeys("1");
driver.findElement(By.cssSelector(".btn:nth-child(5)")).click();
driver.findElement(By.cssSelector(".btn:nth-child(6)")).click();
driver.findElement(By.linkText("BIBLIOTECA UNIVERSIDAD POLITÉCNICA DE MADRID")).click();
driver.findElement(By.cssSelector(".two > .card:nth-child(2) > .p")).click();
driver.findElement(By.cssSelector(".ghost")).click(); } } ahora con esto, pero manten el login del
test anterior

```

Perfecto: te dejo el **ReturnTest reescrito** para que:

- Mantenga **exactamente el login robusto** del test anterior (Cognito: username/password + submit() + waits)
- Quite los IDs/clases frágiles (formField:*, awsui_*) y los Actions innecesarios
- Use **JUnit 5 + SpringBootTest** como tus tests que funcionan
- No haga driver.close() a mitad (eso te rompe la sesión y a veces el WebDriver)

He mantenido el flujo que grabaste: **devolver ejemplar (copyId=2)** y luego el flujo de **préstamo (copyCode=1)**.

Nota: tus selectores .three > .card:nth-child(...), .btn:nth-child(...), .item:nth-child(2)... siguen siendo frágiles, pero los dejo porque no tengo data-testid tuyos. Si luego me dices qué texto tienen esos botones ("Devolver", "Confirmar", etc.), los cambio por selectores por texto o testid.

ReturnTest (con login del test anterior)

```

package com.upm.library;

import static org.hamcrest.CoreMatchers.is;
import static org.hamcrest.MatcherAssert.assertThat;

import java.time.Duration;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import org.openqa.selenium.*;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;

@ActiveProfiles("aws")
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.DEFINED_PORT)
public class ReturnTest {

    private WebDriver driver;
    private WebDriverWait wait;

    private static final String BASE_URL = "http://localhost:8080";
    private static final String USER = "prueba@prueba.com";
    private static final String PASS = "1234567aA!";

    @BeforeEach
    void setUp() {
        driver = new FirefoxDriver();
        driver.manage().window().setSize(new Dimension(1936, 1056));
        wait = new WebDriverWait(driver, Duration.ofSeconds(30));
    }

    @AfterEach
    void tearDown() {
        if (driver != null) driver.quit();
    }

    @Test
    void devolverYPrestar() {
        // --- 1) Ir a la app + login (robusto como tu test anterior) ---
        driver.get(BASE_URL);
        loginCognito(USER, PASS);

        // --- 2) DEVOLUCIÓN (según lo grabado) ---
    }

```



```

// Card "devolución" (grabaste: .three > .card:nth-child(2))
click(By.cssSelector(".three > .card:nth-child(2)"));

type(By.name("copyId"), "2");
click(By.cssSelector(".primary"));

// botón acción (grabaste: .btn:nth-child(5))
click(By.cssSelector(".btn:nth-child(5)"));

Alert alert = wait.until(ExpectedConditions.alertIsPresent());
assertThat(alert.getText(), is("¿Confirmar devolución del ejemplar?"));
alert.accept();

click(By.linkText("← Volver"));

// Volver a home / perfil
click(By.cssSelector(".two > .card:nth-child(2) > .p"));
click(By.cssSelector(".ghost"));

// --- 3) PRÉSTAMO (lo que grabaste después) ---
// En tu grabación volviste a abrir / cerrar navegador. Aquí no hace falta:
// simplemente volvemos al home y seguimos.
driver.get(BASE_URL);
loginCognito(USER, PASS);

click(By.cssSelector(".three > .card:nth-child(1) > .p"));
type(By.name("userQ"), "a");
click(By.cssSelector(".primary"));

click(By.cssSelector(".item:nth-child(2) > .right span"));

type(By.name("copyCode"), "1");
click(By.cssSelector(".btn:nth-child(5)"));
click(By.cssSelector(".btn:nth-child(6)"));

click(By.linkText("BIBLIOTECA UNIVERSIDAD POLITÉCNICA DE MADRID"));
click(By.cssSelector(".two > .card:nth-child(2) > .p"));
click(By.cssSelector(".ghost"));
}

// ==== Login robusto (copiado del patrón que te funciona) ====
private void loginCognito(String username, String password) {
// Si ya estás logueado (sesión viva), puede que no aparezca Cognito.
// En ese caso, simplemente no hacemos nada.
if (driver.getCurrentUrl().contains("amazoncognito.com")) {
// estamos en Cognito, seguimos
} else {
// si el home ya cargó y no hay inputs de Cognito, salimos
if (driver.findElements(By.cssSelector("input[name='username']",

```

```

input[type='email']").isEmpty()
    && driver.findElements(By.cssSelector("input[name='password'],
input[type='password']").isEmpty())
    return;
}
}

WebElement email = wait.until(ExpectedConditions.elementToBeClickable(
    By.cssSelector("input[name='username'],
input[type='email']")
));
email.clear();
email.sendKeys(username);

WebElement submitEmail = wait.until(ExpectedConditions.elementToBeClickable(
    By.cssSelector("button[type='submit'],
input[type='submit']")
));
submitEmail.click();

WebElement pwd = wait.until(ExpectedConditions.visibilityOfElementLocated(
    By.cssSelector("input[name='password'],
input[type='password']")
));
pwd.clear();
pwd.sendKeys(password);

// Submit "real" (Cognito suele ir mejor así)
pwd.submit();

// Espera a salir de Cognito y entrar en tu app
wait.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
wait.until(ExpectedConditions.urlContains("localhost:8080"));
wait.until(ExpectedConditions.not(ExpectedConditions.urlContains("/login/oauth2/code/")));
}

// ===== Helpers =====
private void click(By locator) {
    wait.until(ExpectedConditions.elementToBeClickable(locator)).click();
}

private void type(By locator, String text) {
    WebElement el = wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
    el.clear();
    el.sendKeys(text);
}
}

```

Dos mejoras rápidas (te lo digo porque te va a ahorrar tiempo)

1. **Evita repetir login:** si tu app mantiene sesión, puedes no hacer `driver.get(BASE_URL)` de nuevo y solo navegar.

2. Los selectores `nth-child` y `.btn:nth-child(5)` son los que más se rompen. Si puedes, añade `data-testid` y los cambiamos a `[data-testid="return-submit"]`, etc.

Si me dices **qué pantalla es** `.three > .card:nth-child(2)` (por ejemplo “Devoluciones”), y qué hace `.btn:nth-child(5)/.btn:nth-child(6)` (texto del botón), te lo cambio a selectores por texto/atributo y quedará muchísimo más estable.

Como puedo validar que después de un test E2E que devuelve un libro con días de retraso, se haya generado por ejemplo una sanción y que los días de sanción son correctos

Para validar eso en un E2E con Selenium/Spring Boot, tienes 3 caminos (de más fiable a más “solo UI”). La clave es **no confiar solo en lo visual** si quieres verificar “se creó sanción” y “días correctos”.

1) Validación más fiable: comprobar en la API o en la base de datos al final del E2E

Después de hacer la devolución con retraso en el navegador:

A) Comprobar por API (recomendado)

Si tu app tiene endpoint tipo:

- `GET /api/users/{id}/sanctions`
- `GET /api/sanctions?user=...`
- `GET /api/loans/{id}` (y ahí viene la sanción)

En el test, tras la devolución, haces una llamada HTTP desde Java y verificas:

- existe sanción
- `days == esperado`
- fechas de inicio/fin correctas

Ventaja: estable, rápido, no depende de selectores.

Cómo hacerlo rápido en el test (sin meterte en `WebClient`):
usa `RestTemplate/WebClient` o `TestRestTemplate` si estás en `@SpringBootTest(RANDOM_PORT)`.

Ejemplo (si usas `RANDOM_PORT`):

```
@Autowired                                TestRestTemplate                                rest;

var sanctions = rest.getForObject("/api/sanctions?userId=123", SanctionDto[].class);
assertThat(sanctions.length,                greaterThan(0));
assertThat(sanctions[0].days(), is(esperado));
```

B) Comprobar en BD (igual de fiable)

Si estás en `@SpringBootTest`, puedes inyectar un repositorio:

@Autowired

SanctionRepository

sanctionRepo;

```
var s = sanctionRepo.findTopByUserIdOrderByCreatedAtDesc(userId).orElseThrow();  
assertThat(s.getDays(), is(esperado));
```

Ventaja: máxima

precisión.

Ojo: si el E2E corre contra una BD real compartida, necesitas limpiar datos o usar datos únicos.

2) Validación UI-only (válida, pero más frágil)

Si tu app muestra:

- un banner “Sanción creada: X días”
- una sección “Mis sanciones”
- o en perfil “Sancionado hasta...”

Entonces al final del flujo:

1. navegas a la pantalla de sanciones
2. localizas el texto / tabla
3. asserts sobre número de días

Ejemplo:

```
click(By.linkText("Mis sanciones"));  
WebElement days = wait.until(visibilityOfElementLocated(By.cssSelector("[data-  
testid='sanction-days']"))));  
assertThat(Integer.parseInt(days.getText()), is(esperado));
```

Para que esto sea estable, pon data-testid en los campos clave.

3) Controlar el “retraso” sin esperar días

No vas a esperar 7 días reales 😊. Tienes que “simular” el tiempo.

Opciones típicas:

A) Clock **inyectable** en Spring (mejor práctica)

En tu app, en vez de `LocalDate.now()` usas `Clock`:

```
LocalDate.now(clock)
```

En tests, puedes usar un `@TestConfiguration` que inyecte un `Clock.fixed(...)` con una fecha “avanzada”.

Entonces:

- prestas hoy
- “mueves” el clock a +N días

- devuelves
- verificas sanción

B) Endpoint/admin “advance time” solo para perfil e2e

En profile=e2e, expones algo tipo:

- POST /e2e/time/advance?days=7
- y tu servicio usa ese “offset” (solo en e2e).

C) Modificar directamente el préstamo en BD antes de devolver

Si no quieres tocar el código, en el test puedes:

- crear el préstamo
- actualizar dueDate en BD a una fecha pasada
- hacer la devolución en UI
- validar sanción

Esto es rápido y práctico para E2E.

Mi recomendación para tu caso

Para validar “se genera sanción” y “días correctos” de forma robusta:

- ☒ UI para ejecutar el flujo (devolver con retraso)
- ☒ DB o API para verificar el resultado (sanción y días)

Así el E2E es estable aunque cambie el HTML.

Ejemplo de estructura (ideal)

1. UI: devolver ejemplar con dueDate ya vencida
2. Assert: leer sanción por repositorio / API
3. Assert: days == expectedDays (y si aplica, startDate/endDate)

Si me dices:

- cómo calculáis la sanción (p.ej. “1 día de sanción por cada día de retraso”, o “mínimo 3 días”...)
- dónde se guarda (tabla/entidad Sanction, Penalty, etc.)
- y si tienes endpoint o repositorio

te pongo el código exacto en tu estilo (@SpringBootTest, Selenium + repositorio) para verificarlo al final del test.

LoanTest.java

File

ReservationTest.java

File

ReturnTest.java

File

Tengo estos test, quiero que los optimices quitando código duplicado

Thought for a few seconds

Vale — en tus 3 tests hay muchísimo código duplicado: **setup/teardown de WebDriver, WebDriverWait, helpers click/type, login Cognito**, y constantes (BASE_URL, usuario/pass, timeout, tamaño ventana). Eso aparece en *LoanTest*, *ReservationTest* y *ReturnTest*.

La forma más limpia es sacar todo eso a una **clase base** y que los tests hereden.

1) Crea una clase base común: BaseE2ETest.java

Guárdala en `src/test/java/com/upm/library/BaseE2ETest.java`

```
package com.upm.library;

import java.time.Duration;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;

import org.openqa.selenium.*;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public abstract class BaseE2ETest {

    protected static final String BASE_URL = "http://localhost:8080";
    protected static final Duration TIMEOUT = Duration.ofSeconds(30);

    // Mejor por variables de entorno, pero lo dejo como lo tienes
    protected static final String USER = "prueba@prueba.com";
    protected static final String PASS = "1234567aA!";
```

```

protected WebDriver driver;
protected WebDriverWait wait;

@BeforeEach
void baseSetUp() {
    driver = new FirefoxDriver();
    driver.manage().window().setSize(new Dimension(1936, 1056));
    wait = new WebDriverWait(driver, TIMEOUT);
}

@AfterEach
void baseTearDown() {
    if (driver != null) driver.quit();
}

protected void openHome() {
    driver.get(BASE_URL);
}

protected void loginCognito() {
    // Si ya estás en la app y no hay inputs de Cognito, no hagas nada
    boolean hayUsername = !driver.findElements(By.cssSelector("input[name='username'],
input[type='email']")).isEmpty();
    boolean hayPassword = !driver.findElements(By.cssSelector("input[name='password'],
input[type='password']")).isEmpty();
    if (!driver.getCurrentUrl().contains("amazoncognito.com") && !hayUsername &&
!hayPassword) {
        return;
    }

    WebElement email = wait.until(ExpectedConditions.elementToBeClickable(
        By.cssSelector("input[name='username'],
input[type='email']")
    ));
    email.clear();
    email.sendKeys(USER);

    wait.until(ExpectedConditions.elementToBeClickable(
        By.cssSelector("button[type='submit'],
input[type='submit']")
    )).click();

    WebElement pwd = wait.until(ExpectedConditions.visibilityOfElementLocated(
        By.cssSelector("input[name='password'],
input[type='password']")
    ));
    pwd.clear();
    pwd.sendKeys(PASS);

    // Cognito suele ir mejor con submit()
    pwd.submit();
}

```

```
// vuelve a tu app y sale del callback
wait.until(ExpectedConditions.not(ExpectedConditions.urlContains("amazoncognito.com")));
wait.until(ExpectedConditions.urlContains("localhost:8080"));
wait.until(ExpectedConditions.not(ExpectedConditions.urlContains("/login/oauth2/code/")));
}

protected void click(By locator) {
    wait.until(ExpectedConditions.elementToBeClickable(locator)).click();
}

protected void type(By locator, String text) {
    WebElement el = wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
    el.clear();
    el.sendKeys(text);
}

protected Alert waitAlert() {
    return wait.until(ExpectedConditions.alertIsPresent());
}
}
```

2) Refactor de tus tests para heredar y quitar duplicado

LoanTest.java (limpio)

```
package com.upm.library;

import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;

@ActiveProfiles("aws")
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.DEFINED_PORT)
public class LoanTest extends BaseE2ETest {

    @Test
    void loanTest() {
        openHome();
        loginCognito();

        click(By.cssSelector(".three > .card:nth-child(1) > .p"));
        type(By.name("userQ"), "a");
        click(By.cssSelector(".primary"));

        click(By.cssSelector(".item:nth-child(2) > .right span"));

        type(By.name("copyCode"), "3");
        click(By.cssSelector(".btn:nth-child(5)"));
    }
}
```



```

        click(By.cssSelector(".btn:nth-child(6)"));

        click(By.linkText("BIBLIOTECA UNIVERSIDAD POLITÉCNICA DE MADRID"));
        click(By.cssSelector(".two > .card:nth-child(2) > .p"));
        click(By.cssSelector(".ghost"));
    }
}

```

ReservationTest.java (limpio)

```

package com.upm.library;

import static org.hamcrest.CoreMatchers.is;
import static org.hamcrest.MatcherAssert.assertThat;

import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;

@ActiveProfiles("aws")
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.DEFINED_PORT)
public class ReservationTest extends BaseE2ETest {

    @Test
    void reservarYCancelar() {
        openHome();
        loginCognito();

        click(By.cssSelector(".two > .card:nth-child(1) > .p"));
        click(By.name("q"));
        type(By.name("q"), "a");
        click(By.cssSelector(".primary"));
        click(By.linkText("Ver ejemplares →"));
        click(By.linkText("Reservar ejemplar"));
        click(By.cssSelector(".primary"));

        click(By.cssSelector(".danger"));
        assertThat(waitAlert().getText(), is("¿Cancelar esta reserva?"));
        waitAlert().accept();

        click(By.cssSelector(".ghost"));
    }
}

```

ReturnTest.java (limpio)

```

package com.upm.library;

import static org.hamcrest.CoreMatchers.is;
import static org.hamcrest.MatcherAssert.assertThat;

```

```

import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;

@ActiveProfiles("aws")
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.DEFINED_PORT)
public class ReturnTest extends BaseE2ETest {

    @Test
    void devolverYPrestar() {
        openHome();
        loginCognito();

        // Devolver
        click(By.cssSelector(".three > .card:nth-child(2)"));
        type(By.name("copyId"), "2");
        click(By.cssSelector(".primary"));
        click(By.cssSelector(".btn:nth-child(5)"));

        assertThat(waitAlert().getText(), is("¿Confirmar devolución del ejemplar?"));
        waitAlert().accept();

        click(By.linkText("← Volver"));
        click(By.cssSelector(".two > .card:nth-child(2) > .p"));
        click(By.cssSelector(".ghost"));

        // Prestar (según tu flujo)
        openHome();
        loginCognito();

        click(By.cssSelector(".three > .card:nth-child(1) > .p"));
        type(By.name("userQ"), "a");
        click(By.cssSelector(".primary"));
        click(By.cssSelector(".item:nth-child(2) > .right span"));

        type(By.name("copyCode"), "1");
        click(By.cssSelector(".btn:nth-child(5)"));
        click(By.cssSelector(".btn:nth-child(6)"));

        click(By.linkText("BIBLIOTECA UNIVERSIDAD POLITÉCNICA DE MADRID"));
        click(By.cssSelector(".two > .card:nth-child(2) > .p"));
        click(By.cssSelector(".ghost"));
    }
}

```

3) Qué duplicado te quita esto

- new FirefoxDriver(), setSize, new WebDriverWait(...) repetido en cada clase
 - login Cognito repetido (y en ReservationTest estaba incluso duplicado con wait1/wait2 y prints)
 - helpers click/type repetidos
-

Si quieres, el siguiente paso para dejarlo aún más estable es cambiar los selectores frágiles (nth-child, .btn:nth-child(5)) por data-testid, pero con esta refactorización ya eliminas lo gordo del código duplicado.

estos en la clase normal, no en test, estan bien asi o estarían mejor con autowired

Sí, **están bien así** — y, de hecho, **mejor que con @Autowired en campos**.

Lo que tienes en la captura es **inyección por constructor** con dependencias final. Es la opción recomendada en Spring hoy en día porque:

- **Inmutabilidad:** final evita que la dependencia cambie y te protege de null.
- **Más testeable:** en tests unitarios puedes instanciar el controller pasando mocks sin necesidad de Spring.
- **Más claro:** las dependencias obligatorias quedan explícitas en el constructor.
- **Evita problemas** de inyección por reflexión en campos.

¿Hace falta @Autowired en el constructor?

- Si tu clase tiene **un solo constructor**, no hace falta `@Autowired` (Spring lo usa igual).
- Solo lo necesitarías si hubiera **más de un constructor** y quisieras indicar cuál usar.

¿Cuándo usar `@Autowired`?

- Si tienes una dependencia **opcional** (poco común en controllers/services).
- Si haces **setter injection** para algo que puede cambiar (también poco común).
- Para inyección “rápida” en prototipos, pero no es lo ideal.

Cómo dejarlo aún más limpio

Si usas Lombok, puedes quitar el constructor “a mano”:

```
@RequiredArgsConstructor
@RestController
public class CatalogController {
    private final CatalogService catalogService;
    private final CopyRepository copyRepository;
    private final BookRepository bookRepository;
    private final UserRepository userRepository;
    private final LoanRepository loanRepository;
    private final ReservationRepository reservationRepository;
}
```

Conclusión: tu enfoque actual (constructor + final) es el que elegiría. `@Autowired` en campos no te aportaría nada y suele ser peor estilo.

como puedo decidir el orden de los test

Depende de si hablas del **orden de métodos dentro de una clase** o del **orden entre clases**. Te dejo las opciones prácticas (con JUnit 5, que es lo que estás usando en tus tests buenos).

1) Ordenar métodos `@Test` dentro de una clase (JUnit 5)

Pon esto encima de la clase:

```
import org.junit.jupiter.api.MethodOrderer;
import org.junit.jupiter.api.TestMethodOrder;
import org.junit.jupiter.api.Order;

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
class MiE2ETest {
    @Test @Order(1)
    void primero() {}

    @Test @Order(2)
```

```

void                                segundo()                                {}
}

```

Eso fuerza el orden **solo dentro de esa clase**.

2) Ordenar tests por nombre (sin poner @Order)

Si prefieres ordenar por nombre:

```

@TestMethodOrder(MethodOrderer.MethodName.class)
class                               MiE2ETest                               {
    @Test                            void          test_01_login()           {}
    @Test                            void          test_02_reservar()         {}
}

```

3) Orden entre *clases* de test (LoanTest antes que ReturnTest, etc.)

JUnit **no garantiza** el orden entre clases. Si lo necesitas en Maven, se controla con **Surefire**.

En pom.xml:

```

<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>3.2.5</version>
<configuration>
    <runOrder>alphabetical</runOrder>
</configuration>
</plugin>

```

Con eso, Maven ejecuta clases en orden alfabético.

En IntelliJ el orden puede diferir si ejecutas “toda la carpeta”; ahí manda el runner del IDE.

Lo que te recomiendo para E2E con Selenium

- **Mejor:** un solo @Test por flujo completo (login → acción → asserts), porque el orden entre tests es frágil.
- Si necesitas separar: usa @Order dentro de una sola clase.

Si me dices si quieres ordenar **métodos** o **clases** y si lo ejecutas con **Maven** o desde **IntelliJ**, te digo la opción exacta para tu caso.

ponme los asserts para verificar que en el primero no hay sancion y en el segundo si

Perfecto, aquí ya estás en el **sitio correcto** para hacerlo bien 👍
Como estás usando `penaltyRepository` **dentro del E2E**, lo ideal es validar **antes y después** de la devolución.

Voy a asumir que tu repositorio devuelve algo tipo:

```
Optional<Penalty>  
List<Penalty>
```

(adapta ligeramente si el tipo cambia).

1 Assert **ANTES**: no hay sanción activa

Justo **antes de devolver**, comprueba que **no existe sanción**:

```
assertThat(  
    penaltyRepository.findByIdAndActiveIsTrue(2L),  
    is(Optional.empty())  
);
```

Si devuelve una lista:

```

        assertThat(
            penaltyRepository.findByIdAndActiveIsTrue(2L),
            is(Collections.emptyList())
        );

```

📌 Esto garantiza que el test empieza limpio.

2 Ejecutas la devolución (tu código actual)

```

        click(By.cssSelector(".three"), By.cssSelector(".card:nth-child(2)"));
        type(By.name("copyId"), "2");
        click(By.cssSelector(".primary"));
        click(By.cssSelector(".btn:nth-child(5)"));

        assertThat(
            waitAlert().getText(),
            is("¿Confirmar devolución del ejemplar?")
        );
        waitAlert().accept();

```

3 Assert DESPUÉS: sí hay sanción activa

Ahora validas que **se ha creado** la sanción:

Si devuelve Optional<Penalty>

```

        var penaltyOpt = penaltyRepository.findByIdAndActiveIsTrue(2L);

        assertThat("Debe existir una sanción activa", penaltyOpt.isPresent(), is(true));

        var penalty = penaltyOpt.get();

```

Si devuelve List<Penalty>

```

        var penalties = penaltyRepository.findByIdAndActiveIsTrue(2L);

        assertThat("Debe existir una sanción activa", penalties.isEmpty(), is(false));

        var penalty = penalties.get(0);

```

4 Assert de los días de sanción

Ahora valida la lógica de negocio:

```

        assertThat(penalty.getDays(), is(ESPERADOS));

```

Si depende del retraso:

```

        long diasRetraso = ChronoUnit.DAYS.between(
            penalty.getLoan().getDueDate(),

```

```

    penalty.getCreatedAt().toLocalDate()
);

assertThat(penalty.getDays(), is((int) diasRetraso));

```

5 Test completo (compacto y claro)

```

@Test
void devolverConSancion() {
    openHome();
    loginCognito();

    // ANTES: no hay sanción
    assertThat(
        penaltyRepository.findByUserIdAndActiveIsTrue(2L),
        is(Optional.empty())
    );

    // DEVOLVER
    click(By.cssSelector(".three > .card:nth-child(2)"));
    type(By.name("copyId"), "2");
    click(By.cssSelector(".primary"));
    click(By.cssSelector(".btn:nth-child(5)"));

    assertThat(waitAlert().getText(), is("¿Confirmar devolución del ejemplar?"));
    waitAlert().accept();

    // DESPUÉS: sí hay sanción
    var penalty = penaltyRepository.findByUserIdAndActiveIsTrue(2L)
        .orElseThrow(() -> new AssertionError("No se creó la sanción"));

    assertThat(penalty.getDays(), is(ESPERADOS));

    click(By.linkText("← Volver"));
    click(By.cssSelector(".two > .card:nth-child(2) > .p"));
    click(By.cssSelector(".ghost"));
}

```

Nota importante (muy buena práctica)

Si este test se ejecuta muchas veces:

- limpia sanciones del usuario en @BeforeEach, o
 - usa un **usuario de test exclusivo** para E2E
-

Si quieres, dime:

- cómo se calcula exactamente la sanción (regla exacta)
 - qué devuelve findByIdAndActiveIsTrue
- y te ajusto los asserts **exactos a tu modelo real**.

incluye un assert de que el start date es hoy

Perfecto



Añadimos ese assert **bien hecho** (sin falsos negativos por horas/zonas).

La idea clave es: **comparar fechas, no timestamps**.

Voy a asumir que tu Penalty tiene algo como:

- LocalDate startDate o
- LocalDateTime startDate

Te pongo **las dos variantes**.

✅ **Assert: start date = hoy**

1 Si startDate es LocalDate

```
assertThat(
    penalty.getStartDate(),
    is(LocalDate.now())
);
```

2 Si startDate es LocalDateTime (más habitual)

👉 **Nunca compares con now() directamente**, compara el toLocalDate():

```
assertThat(
    penalty.getStartDate().toLocalDate(),
    is(LocalDate.now())
);
```

🔧 **Test completo (fragmento final)**

Así queda integrado en tu E2E:

```
var penalty = penaltyRepository.findByIdAndActiveIsTrue(2L)
    .orElseThrow(() -> new AssertionError("No se creó la sanción"));

assertThat(penalty.getDays(), is(ESPERADOS));
```

```
assertThat(
    penalty.getStartDate().toLocalDate(),
    is(LocalDate.now())
);
```

Consejo PRO (para evitar flakes en CI)

Si algún día:

- el test cruza medianoche
- hay servidores en otra zona horaria

La versión **más robusta** es usar Clock inyectado en tu app y reutilizarlo en el test.
Pero para E2E locales/CI normal, este assert es totalmente válido 👍

Si quieres, en el siguiente mensaje puedo:

- ayudarte a **validar también el endDate**
- o calcular dinámicamente los **días esperados según el retraso real**

creame un local date con fecha 2026-01-14 y sacame los días hasta hoy

Claro



En Java (con java.time) sería así:

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

LocalDate start = LocalDate.of(2026, 1, 14);
LocalDate today = LocalDate.now();

long days = ChronoUnit.DAYS.between(start, today);

System.out.println(days);
```

Resultado a día de hoy

Si **hoy es 2026-03-03**, el resultado es:

48



Han pasado **48 días** desde **2026-01-14** hasta hoy.

Tip para tests

Si lo usas en asserts E2E (para sanciones, retrasos, etc.), es buena idea fijar la fecha “hoy” con un Clock o comentarlo claramente para evitar confusión cuando el test se ejecute en otro día.

Si quieres, te preparo el **assert exacto** usando este cálculo en tu test de sanciones.

```

===== CONDITIONS EVALUATION REPORT
===== Positive matches: ----- AopAutoConfiguration
matched: - @ConditionalOnBooleanProperty (spring.aop.auto=true) matched
(OnPropertyCondition) AopAutoConfiguration.AspectJAutoProxyingConfiguration matched: -
@ConditionalOnClass found required class 'org.aspectj.weaver.Advice' (OnClassCondition)
AopAutoConfiguration.AspectJAutoProxyingConfiguration.CglibAutoProxyConfiguration
matched: - @ConditionalOnBooleanProperty (spring.aop.proxy-target-class=true) matched
(OnPropertyCondition) ApplicationAvailabilityAutoConfiguration#applicationAvailability
matched: - @ConditionalOnMissingBean (types:
org.springframework.boot.availability.ApplicationAvailability; SearchStrategy: all) did not find
any beans (OnBeanCondition) AvailabilityHealthContributorAutoConfiguration matched: -
@ConditionalOnClass found required class
'org.springframework.boot.health.contributor.Health' (OnClassCondition)
AvailabilityProbesAutoConfiguration matched: - @ConditionalOnClass found required class
'org.springframework.boot.actuate.endpoint.annotation.Endpoint' (OnClassCondition) -
@ConditionalOnBooleanProperty (management.endpoint.health.probes.enabled=true)
matched (OnPropertyCondition)
AvailabilityProbesAutoConfiguration#livenessStateHealthIndicator matched: -
@ConditionalOnMissingBean (names: livenessStateHealthIndicator; SearchStrategy: all) did not
find any beans (OnBeanCondition)
AvailabilityProbesAutoConfiguration#readinessStateHealthIndicator matched: -
@ConditionalOnMissingBean (names: readinessStateHealthIndicator; SearchStrategy: all) did
not find any beans (OnBeanCondition) CompositeMeterRegistryAutoConfiguration matched: -
@ConditionalOnClass found required class
'io.micrometer.core.instrument.composite.CompositeMeterRegistry' (OnClassCondition)
DataJpaRepositoriesAutoConfiguration matched: - @ConditionalOnClass found required class
'org.springframework.data.jpa.repository.JpaRepository' (OnClassCondition) -
@ConditionalOnBooleanProperty (spring.data.jpa.repositories.enabled=true) matched
(OnPropertyCondition) - @ConditionalOnBean (types: javax.sql.DataSource; SearchStrategy: all)
found bean 'dataSource'; @ConditionalOnMissingBean (types:
org.springframework.data.jpa.repository.support.JpaRepositoryFactoryBean,org.springframew
ork.data.jpa.repository.config.JpaRepositoryConfigExtension; SearchStrategy: all) did not find
any beans (OnBeanCondition)

```

Eso que has pegado es el **Conditions Evaluation Report** de Spring Boot (el “autoconfiguration report”). No es un error en sí: es el listado de *qué autoconfiguraciones se activaron* y por qué.

- **“Positive matches”** = configuraciones que **sí** se aplicaron (por ejemplo AOP, Actuator probes, JPA repos...).

- Suele salir cuando tienes habilitado el **debug** de Spring Boot.

¿Por qué te aparece?

Normalmente por alguna de estas:

- has puesto debug=true en application.properties/yml
- arrancas con --debug
- tienes el logger de org.springframework.boot.autoconfigure en DEBUG

¿Cómo quitarlo?

Opción 1: application.properties

Asegúrate de no tener:

debug=true

Ponlo en false o bórralo.

Opción 2: si arrancas con parámetros

No uses:

--debug

Opción 3: logging

Baja el nivel si lo tienes en DEBUG:

logging.level.org.springframework.boot.autoconfigure=INFO

¿Es relevante para tus E2E?

Solo indica que, por ejemplo:

- AOP está activo (spring.aop.auto=true)
- JPA repos están activos y encontró un DataSource

Si me dices **qué problema estabas viendo** (fallo al arrancar, test que no encuentra algo, etc.), te digo qué parte del report mirar.

```
@Test void devolverConSancion() { openHome(); loginCognito(); assertThat(
penaltyRepository.findByUserIdAndActiveIsTrue(2L), is(Collections.emptyList())) ; // Devolver
click(By.cssSelector(".three > .card:nth-child(2)")); type(By.name("copyId"), "2");
click(By.cssSelector(".primary")); click(By.cssSelector(".btn:nth-child(5)"));
assertThat(waitAlert().getText(), is("¿Confirmar devolución del ejemplar?"));
waitAlert().accept(); var penalties = penaltyRepository.findByUserIdAndActiveIsTrue(2L);
assertThat("Debe existir una sanción activa", penalties.isEmpty(), is(false)); var penalty =
penalties.get(0); assertThat( penalty.startDate(), is(LocalDate.now())) ; long days =
ChronoUnit.DAYS.between(LocalDate.of(2026, 1, 24), LocalDate.now());
assertThat(penalty.endDate(), is(systemConfigService.getPenaltyDays() * days)); } Tengo este
```

test, el assert que comprueba si la sanción se ha guardado o no, falla. Al debuggear he visto que primero hace el assert y luego se llama al controlador para guardarlo. Arregla eso

Te falla porque **la sanción se crea en un proceso asíncrono/transactional** y tu test consulta el repositorio **antes de que la transacción que guarda la sanción haya terminado/commit**. En UI parece “devuelto”, pero en BD aún no está.

La solución correcta en E2E es **esperar (polling) hasta que exista la sanción**, en vez de leer el repositorio una sola vez.

✅ **Arreglo: Awaitility (ya lo tienes en el classpath por Spring Boot starter test)**

Sustituye tu bloque “después de devolver” por un `await().untilAsserted(...)`.

Añade imports:

```
import static org.awaitility.Awaitility.await;
import java.util.concurrent.TimeUnit;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
```

Y cambia la parte de verificación así:

```
// Asegura estado inicial limpio
assertThat(penaltyRepository.findByIdAndActiveIsTrue(2L), is(Collections.emptyList()));

// Devolver (UI)
click(By.cssSelector(".three > .card:nth-child(2)"));
type(By.name("copyId"), "2");
click(By.cssSelector(".primary"));
click(By.cssSelector(".btn:nth-child(5)"));
assertThat(waitAlert().getText(), is("¿Confirmar devolución del ejemplar?"));
waitAlert().accept();

// Espera a que la sanción se persista (polling hasta 10s)
await()
    .atMost(10, TimeUnit.SECONDS)
    .pollInterval(200, TimeUnit.MILLISECONDS)
    .untilAsserted(() -> {
        var penalties = penaltyRepository.findByIdAndActiveIsTrue(2L);
        assertThat("Debe existir una sanción activa", penalties.isEmpty(), is(false));

        var penalty = penalties.get(0);
        assertThat(penalty.getStartDate(), is(LocalDate.now()));

        long daysLate = ChronoUnit.DAYS.between(LocalDate.of(2026, 1, 24), LocalDate.now());
        long expectedEndDays = systemConfigService.getPenaltyDays() * daysLate;

        // OJO: ajusta esto según el tipo real de endDate (ver nota abajo)
```

```
assertThat(penalty.getEndDate(), is(expectedEndDays));
});
```

Con esto, aunque el controlador/servicio guarde la sanción “un poco después”, el test **no falla**: espera a que aparezca.

Nota importante sobre tu assert de endDate

Ahora mismo tienes:

```
assertThat(penalty.getEndDate(), is(systemConfigService.getPenaltyDays() * days));
```

Eso solo es correcto si endDate es un **número** (por ejemplo “días de sanción”). Pero normalmente endDate es **una fecha** (LocalDate).

Si endDate es LocalDate, lo correcto sería:

```
LocalDate expectedEnd =
LocalDate.now().plusDays(systemConfigService.getPenaltyDays() * daysLate);
assertThat(penalty.getEndDate(), is(expectedEnd));
```

Si me confirmas el tipo de penalty.getEndDate() (¿LocalDate o long/int?), te dejo el assert exacto.

Alternativa sin Awaitility (solo con WebDriverWait)

También se puede hacer con un wait.until(driver -> ...), pero Awaitility es más limpio para BD.

Si quieres, te ajusto también el cálculo de daysLate para que salga del **dueDate real del préstamo** en BD (mucho más fiable que hardcodear 2026-01-24).

ahora quiero validar que no hay sancion y que la devolucion se ha completado

Perfecto. Para validar **(1) que NO hay sanción** y **(2) que la devolución se completó**, lo más fiable es:

1. **BD/Repositorio** → no existe sanción activa
2. **UI o BD** → la devolución quedó registrada (por ejemplo, el préstamo está “returned”, o ya no aparece como prestado, o aparece un mensaje de éxito)

Como no tengo tu modelo exacto, te dejo 2 variantes para (2): por **UI** y por **repositorio**. Tú eliges la que encaje con tu app.

1) Assert: NO hay sanción (con espera corta para evitar flakiness)

Aunque esperes “no hay”, conviene esperar un poquito para evitar que se cree tarde.

```
import static org.awaitility.Awaitility.await;
import java.util.concurrent.TimeUnit;

// ...

await()
    .atMost(2, TimeUnit.SECONDS)
    .untilAsserted(() ->
        assertThat(penaltyRepository.findByIdAndActiveIsTrue(2L), is(Collections.emptyList()))
    );
```

2A) Validar devolución completada por UI (recomendado si tienes feedback visible)

Después de aceptar el alert:

```
assertThat(waitAlert().getText(), is("¿Confirmar devolución del ejemplar?"));
waitAlert().accept();
```

```
// ejemplo 1: aparece un mensaje toast/alerta (ajusta selector/texto)
WebElement msg = wait.until(ExpectedConditions.visibilityOfElementLocated(
    By.cssSelector(".toast, .alert-success, [role='status']")
));
assertThat(msg.getText(), is("Devolución realizada correctamente"));
```

Si no tienes toast, otra opción UI típica:

- que el ejemplar ya no salga en “Préstamos activos”
- o que el estado cambie a “Disponibile”

Ejemplo:

```
// ejemplo 2: el ejemplar 2 ya no aparece como prestado (ajusta)
assertThat(driver.getPageSource().contains("Ejemplar 2 - Prestado"), is(false));
```

2B) Validar devolución completada por BD (más fiable)

Si tienes loanRepository o similar, lo ideal es verificar que el préstamo pasó a devuelto.

Ejemplos típicos (elige lo que exista en tu modelo):

Si existe returnedAt:

```
var loan = loanRepository.findActiveLoanByCopyId(2L).orElse(null);
assertThat("No debería existir préstamo activo para ese ejemplar", loan, is((Loan)null));
```

```
// o si consultas el último préstamo:
var lastLoan = loanRepository.findTopByCopyIdOrderByIdDesc(2L).orElseThrow();
assertThat("Debe tener returnedAt", lastLoan.getReturnedAt() != null, is(true));
```

Si existe status:

```
var lastLoan = loanRepository.findTopByCopyIdOrderByIdDesc(2L).orElseThrow();
assertThat(lastLoan.getStatus(), is(LoanStatus.RETURNED));
```

Y si es asíncrono, lo haces con await():

```
await().atMost(5, TimeUnit.SECONDS).untilAsserted(() -> {
var lastLoan = loanRepository.findTopByCopyIdOrderByIdDesc(2L).orElseThrow();
assertThat(lastLoan.getReturnedAt() != null, is(true));
});
```

Ejemplo completo (NO sanción + devolución OK)

Te lo dejo con la opción BD para devolución (la más robusta), y con await en ambas:

```
@Test
void devolverSinSancion() {
    openHome();
    loginCognito();

    // Devolver (UI)
    click(By.cssSelector(".three > .card:nth-child(2)"));
    type(By.name("copyId"), "2");
    click(By.cssSelector(".primary"));
    click(By.cssSelector(".btn:nth-child(5)"));
    assertThat(waitAlert().getText(), is("¿Confirmar devolución del ejemplar?"));
    waitAlert().accept();

    // 1) NO hay sanción (espera corta)
    await().atMost(2, TimeUnit.SECONDS).untilAsserted(() ->
        assertThat(penaltyRepository.findByUserIdAndActiveIsTrue(2L), is(Collections.emptyList()))
    );

    // 2) devolución completada (ajusta a tu repo/modelo)
    await().atMost(5, TimeUnit.SECONDS).untilAsserted(() -> {
        var lastLoan = loanRepository.findTopByCopyIdOrderByIdDesc(2L).orElseThrow();
        assertThat(lastLoan.getReturnedAt() != null, is(true));
    });
}
```

Dime esto y te lo dejo exacto sin “ajusta selector”

1. ¿Tienes loanRepository? ¿Qué métodos tiene?
2. ¿Cómo se representa la devolución en tu entidad? (returnedAt, status, active=false, etc.)
3. ¿Tu Penalty es por userId o por loanId?

Con eso te pongo los asserts exactos a tu código real.


```
assertThat(                penaltyRepository.findByUserIdAndActiveIsTrue(2L),
is(Collections.emptyList())) ); tengo esto en el código, como puedo verificar que no está vacío?
```

Ahora mismo estás comprobando que **está vacío**:

```
assertThat(
penaltyRepository.findByUserIdAndActiveIsTrue(2L),
is(Collections.emptyList()))
);
```

Si quieres comprobar que **NO está vacío**, tienes varias formas.

✅ Opción 1 (la más clara)

Comprueba directamente isEmpty():

```
var      penalties      =      penaltyRepository.findByUserIdAndActiveIsTrue(2L);

assertThat(penalties.isEmpty(), is(false));
```

✅ Opción 2 (más legible)

Usando not() de Hamcrest:

```
assertThat(
penaltyRepository.findByUserIdAndActiveIsTrue(2L),
is(not(Collections.emptyList())))
);
```

Necesitas este import:

```
import static org.hamcrest.core.IsNot.not;
```

✅ Opción 3 (la que recomiendo)

Comprueba el tamaño:

```
assertThat(
penaltyRepository.findByUserIdAndActiveIsTrue(2L).size(),
is(1)
);
```

o si puede haber más de una:

```
assertThat(
penaltyRepository.findByUserIdAndActiveIsTrue(2L).size()      >      0,
```

```
is(true)
);
```

★ La mejor para tests

Yo usaría esto porque es muy claro:

```
var    penalties    =    penaltyRepository.findByUserIdAndActiveIsTrue(2L);

assertThat("Debe existir una sanción activa", penalties.isEmpty(), is(false));
```

💡 Si quieres, también puedo enseñarte una forma **más limpia con AssertJ** que usan muchos proyectos Spring y que queda mucho mejor para listas.

ReturnTest.java

File

ReservationTest.java

File

LoanTest.java

File

BaseE2ETest.java

File

tengo estos test, si los ejecuto individualmente funcionan bien, pero si los ejecuto con mvn clean install fallan

Thought for 7s

Eso suele pasar por **dependencia entre tests**. Individualmente pasan porque empiezan “limpios”, pero con mvn clean install se ejecutan **todos seguidos** y un test le deja estado al siguiente.

En tus E2E, las causas más probables son estas:

1) Compartís datos y os pisáis

Veo que usas siempre cosas como:

- el mismo usuario (prueba@prueba.com)
- los mismos ejemplares (copyId=2, copyCode=1, copyCode=3)
- reservas, préstamos y devoluciones que cambian el estado real

Entonces pasa esto:

- un test presta un ejemplar
- otro espera que esté disponible
- uno crea una sanción
- otro espera que no exista

Ejecutados solos pasan. En conjunto, no.

2) Estás usando DEFINED_PORT

Si varios tests arrancan Spring en 8080, puede haber:

- conflictos de puerto
- contexto reutilizado raro
- navegador entrando antes de que el contexto correcto esté listo

Para E2E suele ir mejor RANDOM_PORT + @LocalServerPort.

3) Orden no garantizado

Con Maven no puedes asumir que:

- ReservationTest vaya antes que LoanTest
- o que el estado quede como tú esperas

4) Posible ejecución paralela

A veces Maven/Surefire/Failsafe puede ejecutar cosas en paralelo o reutilizar forks de forma que los Selenium E2E se molestan entre sí.

Qué haría yo para arreglarlo

Opción A — la más importante: limpiar estado antes de cada test

Antes de cada test, deja la BD en un estado conocido.

Por ejemplo en @BeforeEach:

- borrar sanciones del usuario de test
- borrar reservas activas
- devolver/préstamos activos
- restaurar disponibilidad de ejemplares

Algo así, conceptualmente:

```
@BeforeEach
void cleanState() {
    penaltyRepository.deleteAllByUserId(2L);
    reservationRepository.deleteAllByUserId(2L);
}
```

```
loanRepository.deleteActiveLoansByUserId(2L);
copyRepository.resetAvailabilityForTestData();
}
```

Si no tienes esos métodos, créalos o usa SQL de limpieza.

Opción B — usar datos distintos por test

En vez de reutilizar siempre los mismos:

- usuario A para reservas
- usuario B para préstamos
- usuario C para devoluciones

y ejemplares distintos para cada flujo.

Eso reduce muchísimo los choques.

Opción C — cambiar a RANDOM_PORT

En vez de:

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.DefinedPort)
```

usa:

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RandomPort)
```

y:

```
@LocalServerPort
```

```
int port;
```

luego:

```
driver.get("http://localhost:" + port);
```

Opción D — desactivar paralelismo para E2E

En pom.xml, para tests E2E con Selenium, mejor sin paralelo.

Ejemplo con Surefire:

```
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>3.2.5</version>
<configuration>
<parallel>none</parallel>
<reuseForks>>false</reuseForks>
```

```
</configuration>
</plugin>
```

Opción E — no depender del orden

Cada test debe poder ejecutarse:

- solo
- después de cualquier otro
- en cualquier orden

Si necesitas un flujo completo, mejor un solo @Test grande que varios tests acoplados.

Lo más probable en tu caso

Por lo que has ido enseñando, apostarí a que el problema principal es:

estado **compartido** **en** **BD**
más concretamente:

- préstamos/reservas/sanciones que deja un test y rompen el siguiente

Solución práctica mínima

Haz estas 3 cosas:

1. Cambia a RANDOM_PORT
2. Desactiva paralelismo en Maven
3. En @BeforeEach, limpia sanciones, préstamos y reservas del usuario/copies de test

Cómo comprobar que es eso

Ejecuta:

```
mvn -Dtest=ReservationTest,LoanTest,ReturnTest test
```

y mira cuál falla cuando va después de otro. Casi seguro verás que el estado esperado ya no coincide.

Si me pegas tus entidades o repositorios de Penalty, Loan, Reservation y Copy, te escribo un cleanState() exacto para tus tests.

```
no       tengo       esa       librería       de       maven,       tengo       <plugin>
<groupId>org.springframework.boot</groupId>       <artifactId>spring-boot-maven-
plugin</artifactId> </plugin>
```

No pasa nada  **No necesitas añadir nada especial** para poder configurar cómo Maven ejecuta los tests. Aunque tengas solo:

```
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
```

los tests los ejecuta realmente **Surefire**, que Maven usa por defecto aunque no esté declarado.

Si quieres controlar cómo se ejecutan tus E2E (muy recomendable con Selenium), añade **este plugin** al pom.xml.

Añade esto a <plugins>

```
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>3.2.5</version>
<configuration>
  <parallel>none</parallel>
  <reuseForks>false</reuseForks>
</configuration>
</plugin>
```

Quedaría algo así:

```
<build>
<plugins>

<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
</plugin>

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.2.5</version>
  <configuration>
    <parallel>none</parallel>
    <reuseForks>false</reuseForks>
  </configuration>
</plugin>

</plugins>
</build>
```

Qué hace esto

- parallel=none → evita que Maven ejecute tests en paralelo (muy importante con Selenium).
- reuseForks=false → cada test arranca en un fork limpio, evitando que el contexto de Spring o el WebDriver se mezclen.

💡 **Pero ojo:** aunque arregla muchos problemas, **tu problema seguramente es el estado compartido en la BD** (préstamos, reservas, sanciones). Los E2E con Selenium **no deberían depender del orden de ejecución**.

Si quieres, puedo enseñarte **la forma profesional de aislar E2E en Spring Boot** (lo que usan muchos proyectos grandes) para que:

- mvn clean install
- IntelliJ
- CI/CD

ejecuten siempre los tests **sin fallos aleatorios**.

identifica en los test que te he pasado, cosas que se guardan pero no se eliminan

Thought for a couple of seconds

Sí. Por lo que has ido enseñando de ReservationTest, LoanTest y ReturnTest, las cosas que **cambian estado y se guardan**, pero **no siempre se revierten o limpian**, son estas:

1) Reservas

En ReservationTest:

- haces una reserva
- luego la cancelas

Eso en principio **sí se limpia** dentro del propio test, **siempre que el test llegue al final**. Problema: si falla antes del .danger o antes de aceptar el alert, **la reserva se queda creada**.

2) Préstamos

En LoanTest:

- haces un préstamo (copyCode, botones de confirmar, etc.)
- **no vi una devolución dentro de ese mismo test**

Eso significa que:

- el préstamo queda activo en BD

- el ejemplar queda prestado/no disponible
- otro test que espere ese ejemplar disponible puede fallar

Este es uno de los más claros.

3) Devoluciones

En ReturnTest:

- haces una devolución
- luego haces también un préstamo otra vez

O sea:

- primero alteras el estado del préstamo existente
- y después vuelves a crear otro préstamo

Por tanto, al final del test **vuelves a dejar estado persistido**.
No queda "limpio".

4) Sanciones

En el test devolverConSancion() que enseñaste:

- verificas que se genera una sanción
- pero **no la eliminas al final**

Entonces:

- esa sanción queda activa para el usuario
- cualquier test posterior que espere Collections.emptyList() en penaltyRepository.findByIdAndActiveIsTrue(2L) puede fallar

Este es otro foco clarísimo.

5) Estado de ejemplares/copias

En varios tests usas:

- copyId = 2
- copyCode = 1
- copyCode = 3

Esos ejemplares cambian de estado:

- disponible
- reservado
- prestado
- devuelto

Aunque no “guardes” explícitamente una entidad en el test, **tu aplicación sí lo persiste**. Y no estás restaurando ese estado al final de cada test.

6) Posible sesión/login

Esto normalmente no es el problema principal porque el navegador se cierra en @AfterEach, pero si hay lógica de sesión persistida server-side, también puede dejar rastro. Aun así, comparado con préstamos/reservas/sanciones, esto suele importar menos.

Resumen claro: qué se guarda y no se elimina

Lo más problemático que veo es:

- **Préstamos creados** en LoanTest
- **Sanciones creadas** en devolverConSancion()
- **Préstamo re-creado** al final de ReturnTest
- **Reservas** si ReservationTest falla antes de cancelarlas
- **Estado de copias/ejemplares** modificado en todos esos flujos

Qué revisaría test por test

ReservationTest

Riesgo:

- deja reserva si falla antes de cancelarla

LoanTest

Riesgo:

- deja préstamo activo
- deja ejemplar no disponible

ReturnTest

Riesgo:

- modifica un préstamo existente
- puede crear sanción
- luego vuelve a crear préstamo

devolverConSancion()

Riesgo:

- deja sanción activa
- deja devolución realizada
- deja el préstamo en estado devuelto

Lo que deberías limpiar en @BeforeEach o @AfterEach

Para ese usuario y esas copias de test, deberías limpiar al menos:

- sanciones activas
- reservas activas
- préstamos activos
- estado de las copias usadas (1, 2, 3)
- cualquier relación “copy assigned to loan/reservation”

Conclusión

Los dos grandes culpables de que fallen al ejecutar todos juntos son casi seguro:

1. **préstamos que se crean y no se revierten**
2. **sanciones que se crean y no se borran**

Si me pegas otra vez el código actual de esos tests o de tus repositorios (PenaltyRepository, LoanRepository, ReservationRepository, CopyRepository), te digo exactamente qué método de limpieza te falta en cada uno.

`penalties.stream( p -> p.deactivate);` como puedo corregir este stream. Penalties es una `list<Penalty>`

Tu stream tiene varios problemas:

`penalties.stream( p -> p.deactivate);`

Errores:

1. `stream()` **no recibe parámetros**.
2. Falta una operación terminal (`forEach`, `map`, etc.).
3. `deactivate` parece un **método**, por lo que necesita `()`.

Forma correcta (si solo quieres desactivar todas)

Usa `forEach`:

`penalties.forEach(p -> p.deactivate());`

o con referencia a método (más limpio):

```
penalties.forEach(Penalty::deactivate);
```

Si luego quieres guardarlas en el repositorio

Si usas Spring Data JPA:

```
penalties.forEach(Penalty::deactivate);  
penaltyRepository.saveAll(penalties);
```

Si quieres hacerlo en un stream completo

No es necesario, pero sería:

```
penalties.stream()  
.forEach(Penalty::deactivate);
```



Recomendación:

Si solo ejecutas una acción sobre cada elemento, **no necesitas** stream(), forEach directo sobre la lista es más claro.

```
penalties.forEach(Penalty::deactivate);
```

en el código que te he pasado donde se está generando una sanción

Los archivos que me habías subido antes **han caducado**, así que no puedo volver a abrirlos para señalar la línea exacta. Si quieres que lo localice exactamente, vuelve a subir:

- LoanTest.java
- ReturnTest.java
- ReservationTest.java
- BaseE2ETest.java

Aun así, por lo que vimos en tus tests anteriores, **la sanción no se genera en el test, sino en la lógica de negocio del backend cuando haces la devolución.**

Concretamente ocurre cuando el test ejecuta este flujo:

```
click(By.cssSelector(".three > .card:nth-child(2)"));  
type(By.name("copyId"), "2");  
click(By.cssSelector(".primary"));  
click(By.cssSelector(".btn:nth-child(5)"));  
waitAlert().accept();
```

Ese botón llama a tu endpoint de **devolución** de **ejemplar**.
En el backend ocurre algo así:

```
Controller
↓
LoanService.returnCopy(...)
↓
si (dueDate < today)
↓
PenaltyService.createPenalty(...)
↓
penaltyRepository.save(...)
```

Es decir, **la sanción se genera en el servicio que gestiona la devolución del préstamo**, no en el test.

Normalmente está en algo como:

```
loanService.returnLoan(copyId);
```

o

```
catalogService.returnCopy(...)
```

y dentro de ese método verás algo parecido a:

```
if (loan.getDueDate().isBefore(LocalDate.now())) {
    Penalty penalty = new Penalty(...);
    penaltyRepository.save(penalty);
}
```

💡 Cómo localizarlo rápido en tu proyecto

Busca en el código:

```
penaltyRepository.save
```

o

```
new Penalty(
```

o

```
createPenalty
```

Ahí estará el punto exacto donde se genera.

Si vuelves a subir los archivos del backend (LoanService, CatalogService, PenaltyService, etc.), puedo **señalarte exactamente la línea donde se crea la sanción** y explicarte por qué tus tests se pisan entre sí.

que diferencia hay entre save all y save and flush

La diferencia entre saveAll y saveAndFlush en Spring Data JPA está en **cuándo se sincronizan los cambios con la base de datos**.

1 saveAll()

```
repository.saveAll(lista);
```

Qué hace

- Persiste o actualiza **varias entidades**.
- **NO ejecuta inmediatamente el SQL** en la base de datos.
- Los cambios quedan en el **Persistence Context (Hibernate)**.

Cuándo se ejecuta realmente el SQL

- cuando se hace flush
- cuando se hace commit de la transacción
- cuando Hibernate necesita sincronizar

Ejemplo

```
penalties.forEach(Penalty::deactivate);  
penaltyRepository.saveAll(penalties);
```

Aquí puede que **todavía no exista ningún UPDATE en la BD**.

2 saveAndFlush()

```
repository.saveAndFlush(entity);
```

Qué hace

1. Guarda la entidad
2. **Hace flush inmediatamente**

Flush significa que Hibernate ejecuta el SQL contra la BD.

Ejemplo

```
penaltyRepository.saveAndFlush(penalty);
```

Aquí Hibernate ejecuta inmediatamente algo como:

```
update penalty set active=false where id=...
```

Diferencia clave

Método	Guarda	Ejecuta SQL inmediatamente
save()	✓	✗
saveAll()	✓	✗
saveAndFlush()	✓	✓
flush()	✗	✓

Por qué esto te afecta en tus tests

En tus E2E haces algo como:

```
click(...); // devolver libro
var penalties = penaltyRepository.findByIdAndActiveIsTrue(2L);
```

Si el backend hizo:

```
penaltyRepository.save(penalty);
```

puede que:

- el test consulte la BD
- **antes de que Hibernate haya hecho flush**

y por eso te fallaba el assert.

Con:

```
penaltyRepository.saveAndFlush(penalty);
```

te aseguras de que el test **ya vea la sanción**.

Regla práctica

Usa normalmente:

```
save()
saveAll()
```

y solo usa:

```
saveAndFlush()
```

cuando:

- necesitas que el cambio esté **inmediatamente en la BD**
 - estás en **tests**
 - necesitas que otra consulta lo vea ya
 - necesitas validar constraints SQL en el momento.
-

✅ En tu caso (tests E2E) es muy común usar `saveAndFlush` o `await()` para evitar condiciones de carrera.

Si quieres, también puedo explicarte **por qué tus E2E fallan con mvn clean install pero no cuando los ejecutas individualmente**, que casi seguro está relacionado con **transacciones + flush + estado compartido**.